M1-Introduction to Java

PBCST304 - OBJECT ORIENTED PROGRAMMING

Course Overview

Course Code	PBCST304	CIE Marks	60
Teaching Hours/Week (L:T:P:R)	3:0:0:1	ESE Marks	40
Credits	4	Exam Hours	2 Hrs. 30 Min.
Prerequisites (if any)	None	Course Type	Theory

Course Objectives:

1. To teach the core object-oriented principles such as abstraction, encapsulation, inheritance, and polymorphism, robust error-handling using exception mechanisms to ensure program reliability.
2. To equip the learner to develop object oriented programs encompassing fundamental structures, environments, and the effective utilization of data types, arrays, strings, operators, and control statements for program flow in Java.
3. To enable the learner to design and develop event-driven graphical user interface (GUI) database applications using Swing and database connection components.

Course Assessment Method: **(CIE: 60 marks, ESE: 40 marks)**

Continuous Internal Evaluation Marks (CIE):

Attendance	Project	Internal Ex-1	Internal Ex-2	Total
5	30	12.5	12.5	60

End Semester Examination Marks (ESE)

In Part A, all questions need to be answered and in Part B, each student can choose any one full question out of two questions

Part A	Part B	Total
• 2 Questions from each module. • Total of 8 Questions, each carrying 2 marks **(8x2 =16 marks)**	• 2 questions will be given from each module, out of which 1 question should be answered. • Each question can have a maximum of 2 subdivisions. E • ach question carries 6 marks. **(4x6 = 24 marks)**	**40**

PBL Course Elements

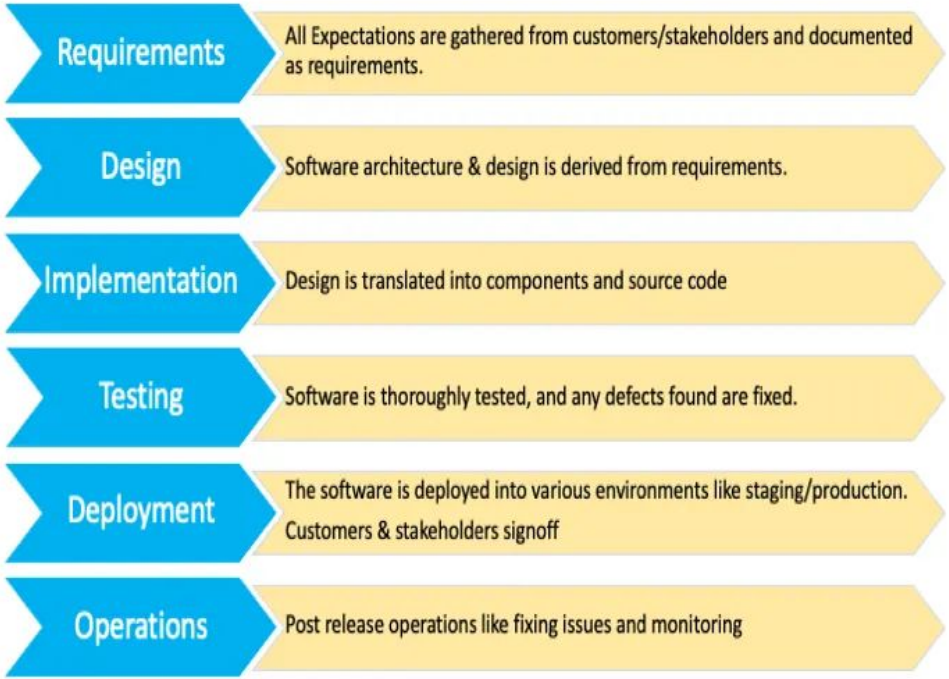
L: Lecture (3 Hrs.)	R: Project (1 Hr.), 2 Faculty Members		
	Tutorial	**Practical**	**Presentation**
Lecture delivery	Project identification	Simulation/ Laboratory Work/ Workshops	Presentation (Progress and Final Presentations)
Group discussion	Project Analysis	Data Collection	Evaluation
Question answer Sessions/ Brainstorming Sessions	Analytical thinking and self-learning	Testing	Project Milestone Reviews, Feedback, Project reformation (If required)
Guest Speakers (Industry Experts)	Case Study/ Field Survey Report	Prototyping	Poster Presentation/ Video Presentation: Students present their results in a 2 to 5 minutes video

Suggestion on Project Topics

Mandatory

- UI – Swing
- Database connectivity using JDBC

Software Development Life Cycle & Deliverables



Software Requirements Specification

Design Document

Source Code

Test Report

User Guide

Expected Deliverables

Design Document: Describe the classes, their attributes, methods and relationships.

Source Code: Well-documented Java code implementing the described functionalities.

Test Cases: A suite of test cases demonstrating the functionality of the system.

User Manual: Instructions on how to set up, run and use the system.

Assessment and Evaluation for Project Activity

Sl. No	Evaluation for	Allotted Marks
1	Project Planning and Proposal	5
2	Contribution in Progress Presentations and Question Answer Sessions	4
3	Involvement in the project work and Team Work	3
4	Execution and Implementation	10
5	Final Presentations	5
6	Project Quality, Innovation and Creativity	3
Total		**30**

Evaluation points

1. Project Planning and Proposal (5 Marks)
- Clarity and feasibility of the project plan
- Research and background understanding
- Defined objectives and methodology

2. Contribution in Progress Presentation and Question Answer Sessions (4 Marks)
- Individual contribution to the presentation
- Effectiveness in answering questions and handling feedback

3. Involvement in the Project Work and Team Work (3 Marks)
- Active participation and individual contribution
- Teamwork and collaboration

4. Execution and Implementation (10 Marks)
- Adherence to the project timeline and milestones
- Application of theoretical knowledge and problem-solving
- Final Result

5. Final Presentation (5 Marks)
- Quality and clarity of the overall presentation
- Individual contribution to the presentation
- Effectiveness in answering questions

6. Project Quality, Innovation, and Creativity (3 Marks)
- Overall quality and technical excellence of the project
- Innovation and originality in the project
- Creativity in solutions and approaches

M1-Syllabus

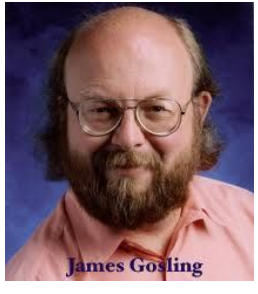
Introduction to Java:-
Structure of a simple java program; Java programming Environment and Runtime Environment (Command Line & IDE); Java compiler; Java Virtual Machine; Primitive Data types and Wrapper Types; Casting and Autoboxing; Arrays; Strings; Vector class; Operators - Arithmetic, Bitwise, Relational, Boolean Logical, Assignment, Conditional (Ternary); Operator Precedence; Control Statements - Selection Statements, Iteration Statements and Jump Statements; Functions; Command Line Arguments; Variable Length Arguments; Classes; Abstract Classes; Interfaces. [Use proper naming conventions]

OOP Concepts :-
Data abstraction, encapsulation, inheritance, polymorphism, Procedural and object oriented programming paradigm; Microservices.

Object Oriented Programming in Java :-
Declaring Objects; Object Reference; Introduction to Methods; Constructors; Access Modifiers; this keyword.

History of Java

- Java is a **general-purpose** , **O**bject **O**riented **P**rogramming language developed by Sun Micro System of USA in 1991.
- Developed by **James Gosling** and his team (Green Team) at Sun.
- First name of Java was “Oak,” later it was renamed “Java” in 1995.
- Initially designed for small, embedded systems in electronic appliances like set-top boxes.
- The target of Java is to write a program once and then run this program on multiple operating systems [**PLATFORM INDEPENDENCE**].
- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc.



Sun microsystems

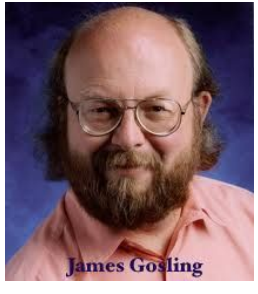
Java ORACLE

History of Java

It is owned by Oracle, and more than 3 billion devices run Java.

It is used for:

- Mobile applications (especially Android apps)
- Desktop applications
- Web applications
- Web servers and application servers
- Games
- Database connection, etc.

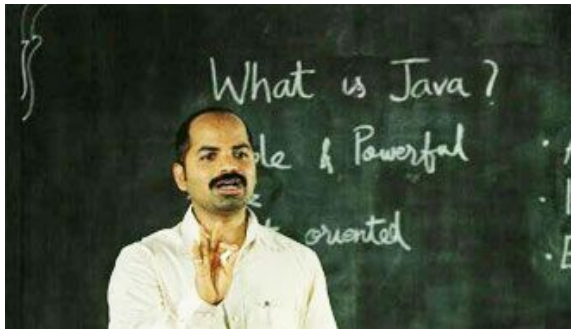


Sun microsystems

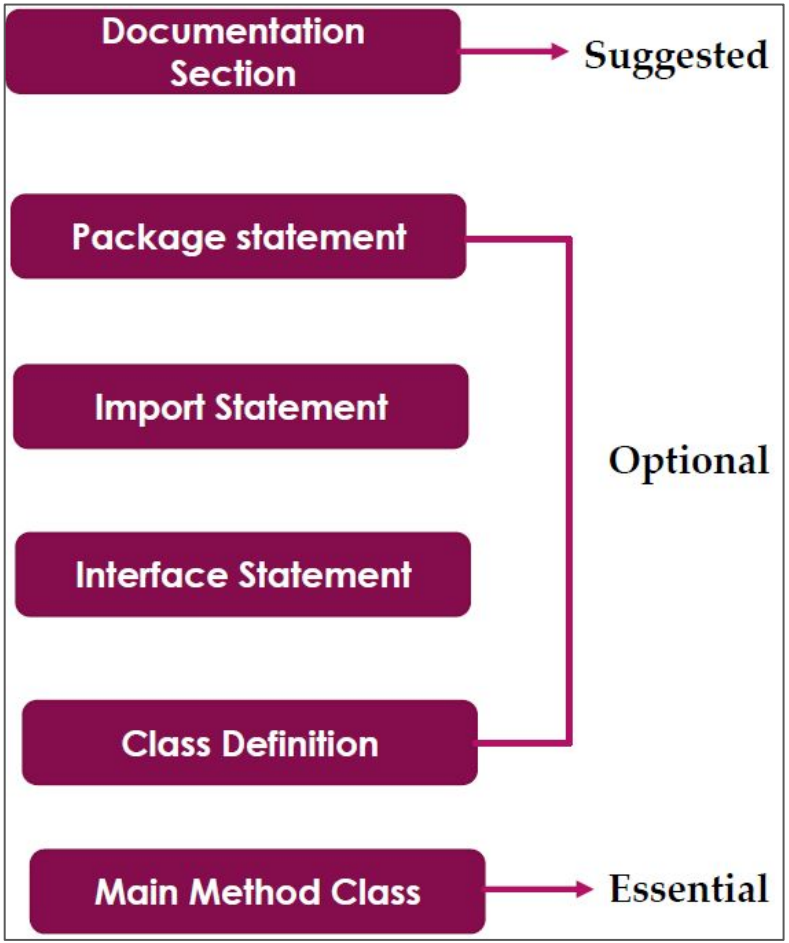
Java ORACLE

Why Use Java?

- **Platform Independent**: Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- **Simple**: It is easy to learn and simple to use
- It is **open-source** and free
- It is **secure**, fast and powerful
- **Robust**: Java makes an effort to eliminate error prone situations
- **Multithreaded**: It is possible to write programs that can perform many tasks simultaneously.
- **Distributed**: Java is designed for the distributed environment of the internet.



Structure of a simple java program




```
/*****First Java Program: HelloWorld.java *****/
//* Author: Mary

public class HelloWorld
{
  public static void main( String[ ] args )
  {
        System.out.println( "Hello, world!" );
  }
}
```

"main" method

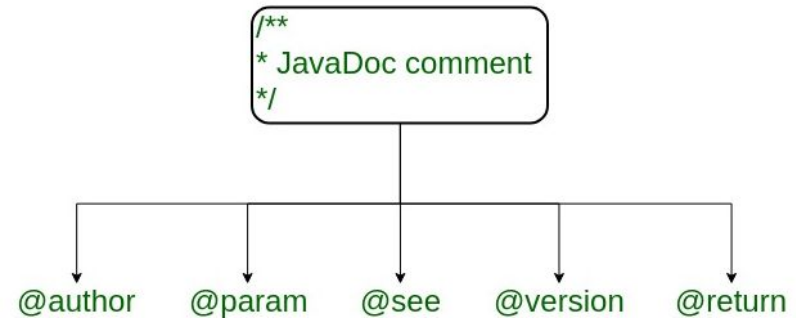
Comments

There are three types of comments defined by Java.

1. **Single-line comment :**Java single line comment starts from `//` and ends till the end of that line.
2. **Multiline comment:** Java multiline comment is between `/* and */`
3. **Documentation comment :** Documentation comment is used to produce an HTML file that documents your program. The documentation comment begins with a `/** and ends with a */`

Comments...

```
1  public class CommentsInJava {
2      /** Javadoc comment...appropriate for documenting a class or method for javadoc utility.
3          Example: The main method is run automatically by the Java Virtual Machine */
4      public static void main (String[] args) {
5          int i=100;
6          // Single line comment...appropriate for documenting a statement.
7          // Example: The following statement prints a string to the console:
8          System.out.println("This is the first statement after the comment!");
9
10         /* Multi-line comment...appropriate for commenting code that is
11            not needed currently but may be
12            necessary in the future.
13         Example: System.out.println("This is the integer variable I created: " + i);
14         System.out.println("This is the last statement in the block!");
15         */
16
17
```



Java program structure

Document section (optional)

- Add comments describing the details of author, purpose, licence, etc

```
/**
*  This file contains the implementation of sample program.
* @author Jinesh Jose
* @version 1.0
* @since 13-09-2020
*/
```

Package Statement (optional)

- You can create a package with any name to organize the classes of that file.
- A package is to group classes that are defined by a name.
- It is like a folder in which all the classes present in that file are kept.
- It is declared as: **package package_name;**

Java program structure

Import Statement (optional)

- This line indicates that if you want to use a class of another package, then you can do this by importing it directly into your program.
- E.g. **import java.io.*; // importing all the classes from java.io package**

Interface statement (optional)

- Interfaces are like a class that includes a group of method declarations.
- It's an optional section and can be used when programmers want to implement multiple inheritances within a program.

Class Definition (optional)

- A Java program file may contain several class definitions.
- But only one public class is allowed in a file
- File name and the public class name should be same.

Java program structure

Main Method Class

- Every Java **stand alone program requires the main method** as the starting point of the program.
- There may be many classes in a Java program, but only one class defines the main method .

public static void main(String args[])

- It is declared public , it means that it can also be used by code outside of its class.
- static indicates classified scope, ie we want to access a method without creating its object, as we call the main method, before creating any class objects.
- void indicates that a method does not return a value.
- main is a method; this is a starting point of a Java program.
- String args[] - method receives an array of strings, it is used to receive the command line arguments to your program.

Sample Program

```java
/** Java program to read some values using Scanner
 * class and print their mean.
 * @author Jinesh Jose
 * @version 1.0
 */
import java.util.Scanner;

public class HelloWorld
{
    public static void main(String[] args)
    {
        // Declare an object and initialize with
        // predefined standard input object
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name:");
        String name = sc.nextLine();
        System.out.println("Hello " + name);
    }
}
```

File | D:/GECI/CST205-OOPJ/Java/basics/HelloWorld.html

PACKAGE CLASS TREE DEPRECATED INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

Class HelloWorld

java.lang.Object
HelloWorld

public class **HelloWorld**
extends java.lang.Object

Constructor Summary

Constructors

Constructor
`HelloWorld()`

```
D:\GECI\CST205-OOPJ\Java\basics>javac HelloWorld.java

D:\GECI\CST205-OOPJ\Java\basics>java HelloWorld.class
Error: Could not find or load main class HelloWorld.class
Caused by: java.lang.ClassNotFoundException: HelloWorld.class

D:\GECI\CST205-OOPJ\Java\basics>java HelloWorld
Enter your name:Jinesh
Hello Jinesh

D:\GECI\CST205-OOPJ\Java\basics>javadoc HelloWorld.java
Loading source file HelloWorld.java...
Constructing Javadoc information...
```

Method Summary

All Methods | **Static Methods** | **Concrete Methods**

Modifier and Type	Method	Description
static void	`main(java.lang.String[] args)`	

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

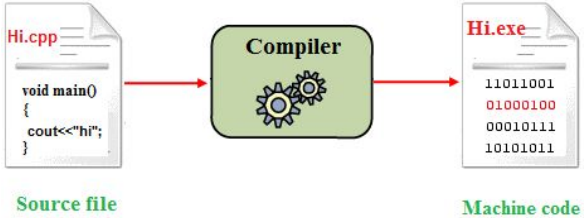
Main Method Class

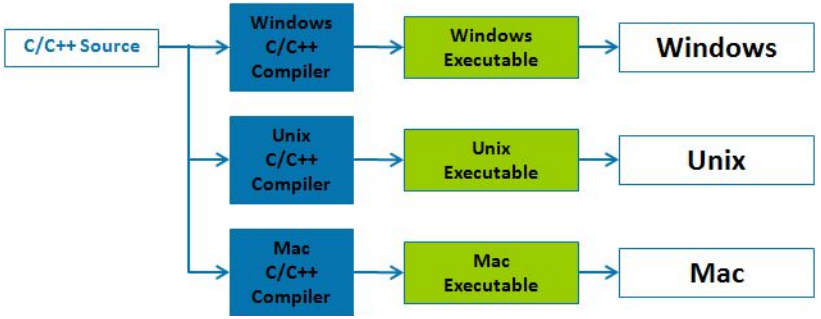
- Every Java program requires the main method as the starting point of the program.
- There may be many classes in a Java program,and only one class defines the main method.
- Methods contain data type declaration and executable statements.

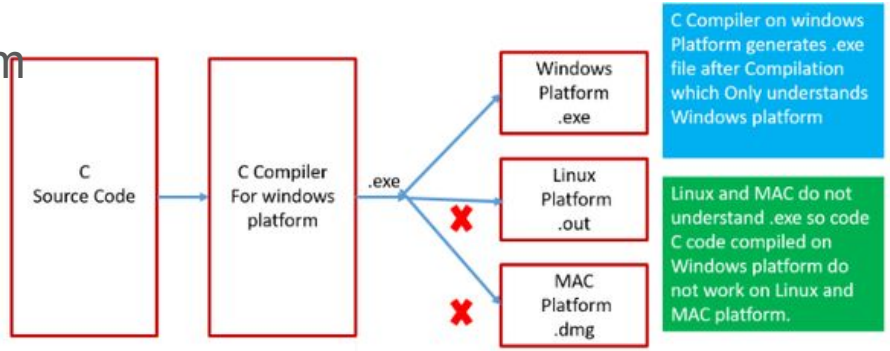
```
class Classname
{
      public static void main(String args[])
      {
      ……
      ……
      }
}
```

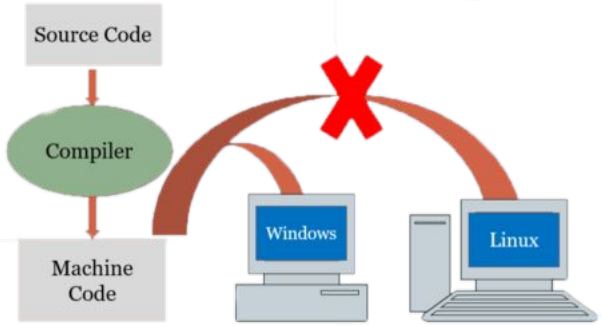
Platform dependency issue

- Platform dependency refers to case where the applications **run under only one operating system** in one series of computers (platform => OS + H/w).
- C/C++ **executable** created for one platform will not run on another, this is nothing but **platform dependence.**
- Solution is **platform independence** which describes a technology to **implement things on one platform** and **use them on another platform** without changes





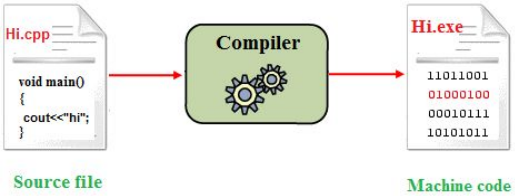




Java Compiler

- Compiler is a software which converts a program written in high level language (Source Language) to low level language (Object/Target/Machine Language)
- Java compilers include the Java Programming Language Compiler (**javac**), the GNU Compiler for Java (**GCJ**), the Eclipse Compiler for Java (**ECJ**) and **Jikes**.
- A Java compiler is a software that takes the java program file(.java) and compiles it to produce a Java **class file**(.class).
- The content of class file is not machine code. It contains **Java bytecode** which can be executed only on **Java Virtual Machine (JVM)**.

- Syntax: **javac HelloWorld.java**

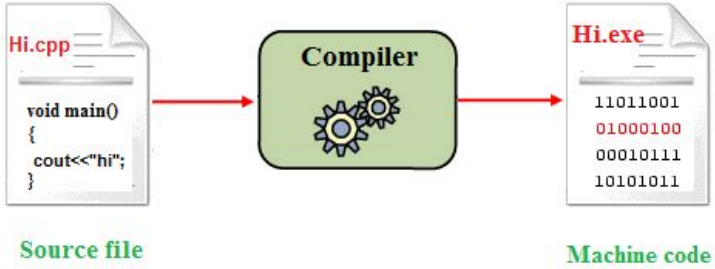


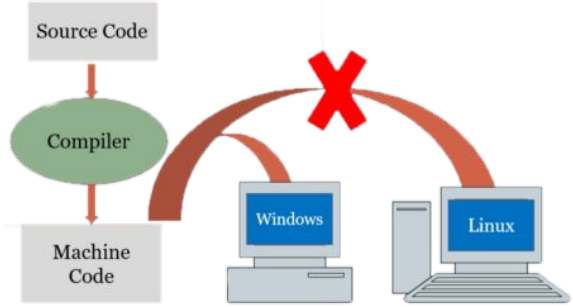


Java Byte Code

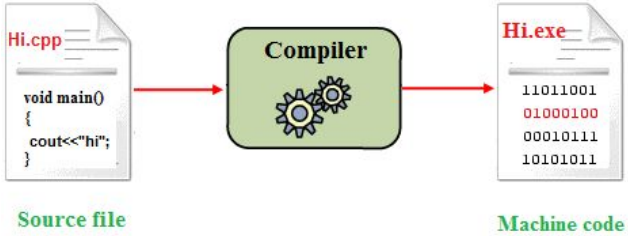


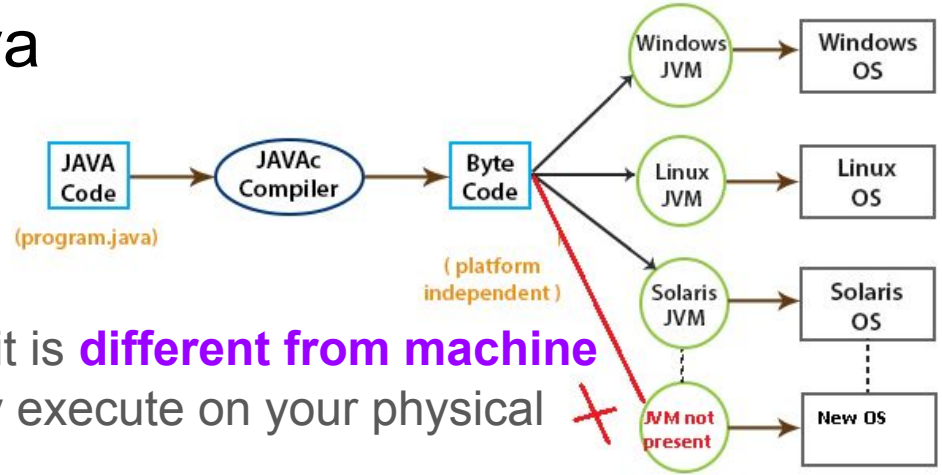
- On compiling java program, we get **class file** that is nothing but **bytecode**. Java **bytecode** is an intermediate representation of the java program which is **machine independent**. ie, it is **not the conventional Machine Code**.
- Byte code cannot directly run on your machine like executable file from C/C++ program, it can run only on Java Virtual Machine.
- Java bytecode contains the instruction set for the Java Virtual Machine.
- JVM is responsible to convert bytecode to machine code and execute on your machine.
- **java** interpreter is used to execute class file [**java ClassFileName**]



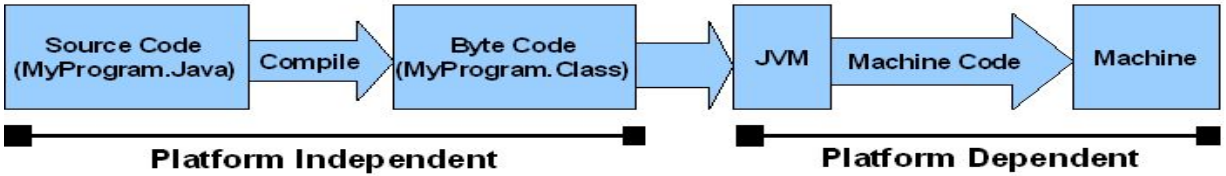


Platform independence in Java



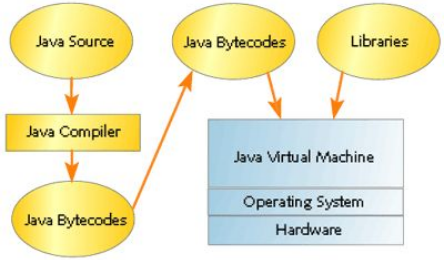


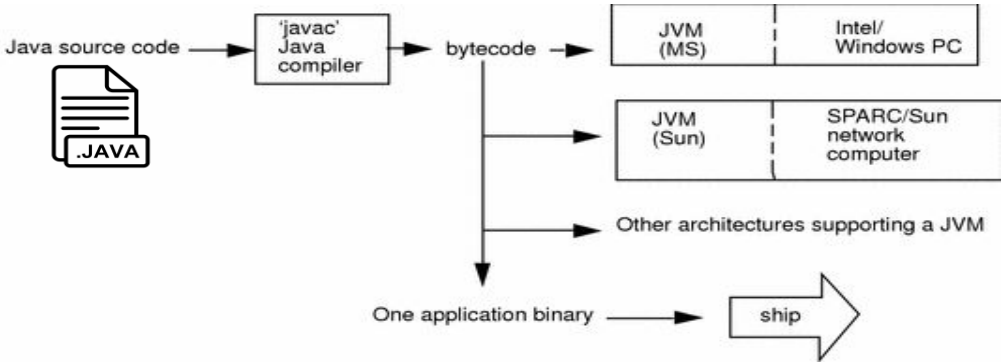
- Java compiler output is class file and it is **different from machine code**. Only machine code can directly execute on your physical machine.
- Class file contains **byte code**, which can execute only on **Java Virtual Machine(JVM)**.
- Java is platform independent because java class files **can run on all operating systems** without any modifications.
- If JVM is available for a platform, then Java can run on that platform



Java Programming Environment

- Java is a concurrent, class based, object oriented programming and runtime environment, consisting of
  1. A programming language (Java)
     Used by programmers to write the application.
  1. An API specification (Java Libraries)
     Huge number of libraries and components for Java program
  1. A virtual machine specification (JVM)
     It is a program which provides the runtime environment to execute Java programs



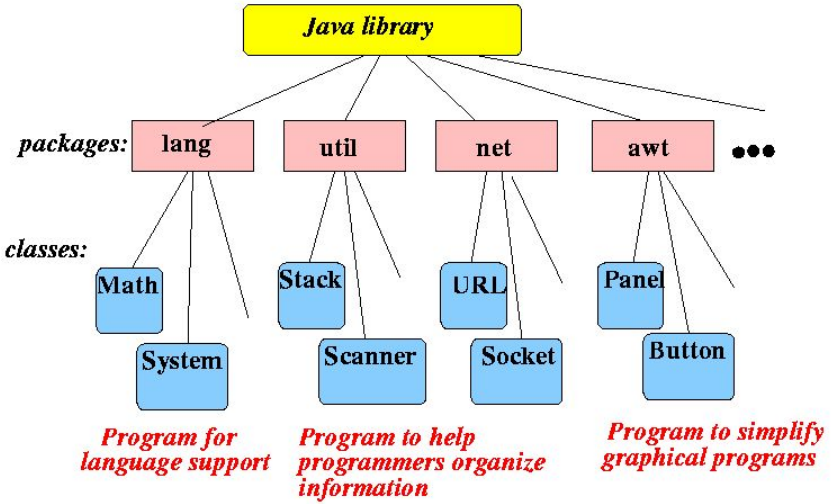


Java technology: Language specification

- The Java Language Specification (**JLS**) defines how a confirming Java application must behave.
- JLS describes the technical details of **how** the language is implemented.
- Java is a high level (human-readable) language which is generally considered easy to read and write.
- Java is class-based and strictly object-oriented in nature.
- Java is intended to be easy to learn and to teach.
- There are many different implementations of JLS available, both proprietary and open source.
- Fulfils the goal Write Once, Run Anywhere (WORA). ie, the same program can be highly portable and run on many different platforms

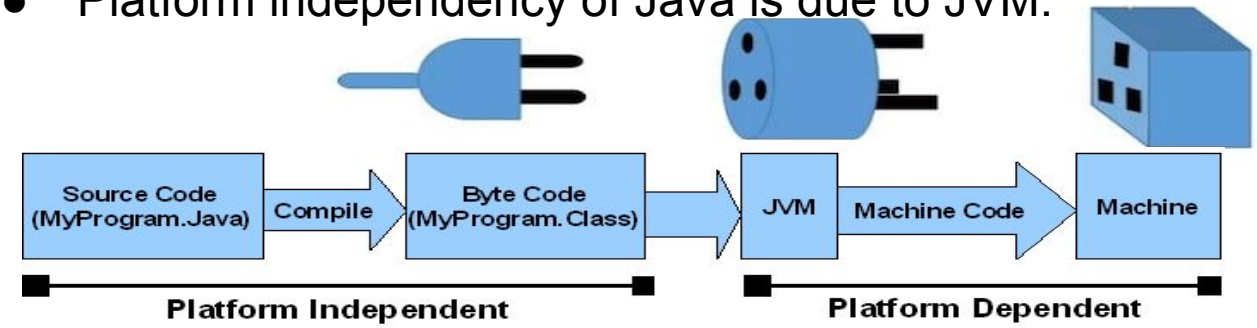
Java technology: Library specification

- Specifies the standard libraries in Java language.
- The Java Class Library is a set of standard libraries that is available to any application running on the JVM.
- This set of libraries is composed of classes that allow programs to handle commons tasks like: network communication, collection manipulation, file operations, user interface creation, etc.



Java technology: Virtual Machine Specification

- A virtual machine(VM) is a software implementation of an **abstract computer**.
- Java has a **VM specification** that formally describes what is required in a Java VM implementation.
- Java Virtual Machine(JVM) is implemented as per Java VM specification to execute the **Java bytecode (**ie, instruction set understandable to the VM**)**.
- Most of the JVMs are implemented using C language, JVM developed for one platform will not work on another. ie, JVM is platform dependent.
- It provides the **runtime environment** to execute Java programs (**bytecode**) on a particular platform.
- Platform independency of Java is due to JVM.



JVM – Java Virtual Machine

JVM is the Java run-time system.

Java Virtual Machine is called virtual because it provides a machine interface that does not depend on the operating system and machine hardware architecture.

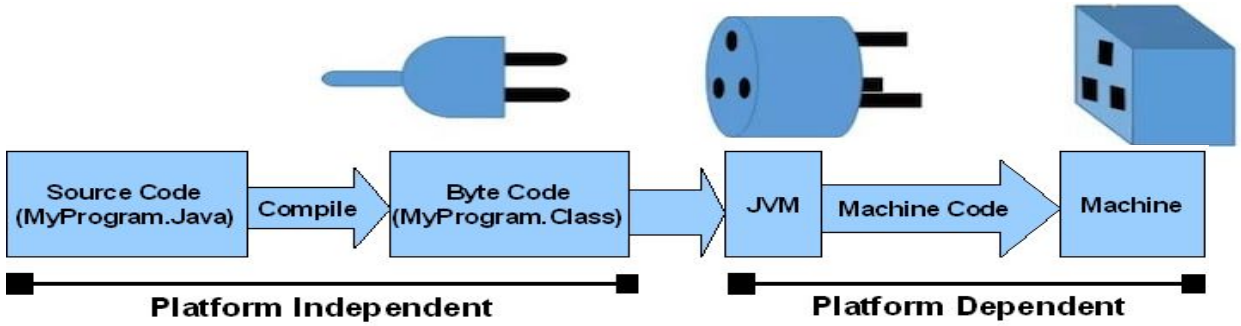
Java programs are WORA(Write Once Run Anywhere) programs.

When we compile a Java program, we get .class file (byte code)which is not executable.

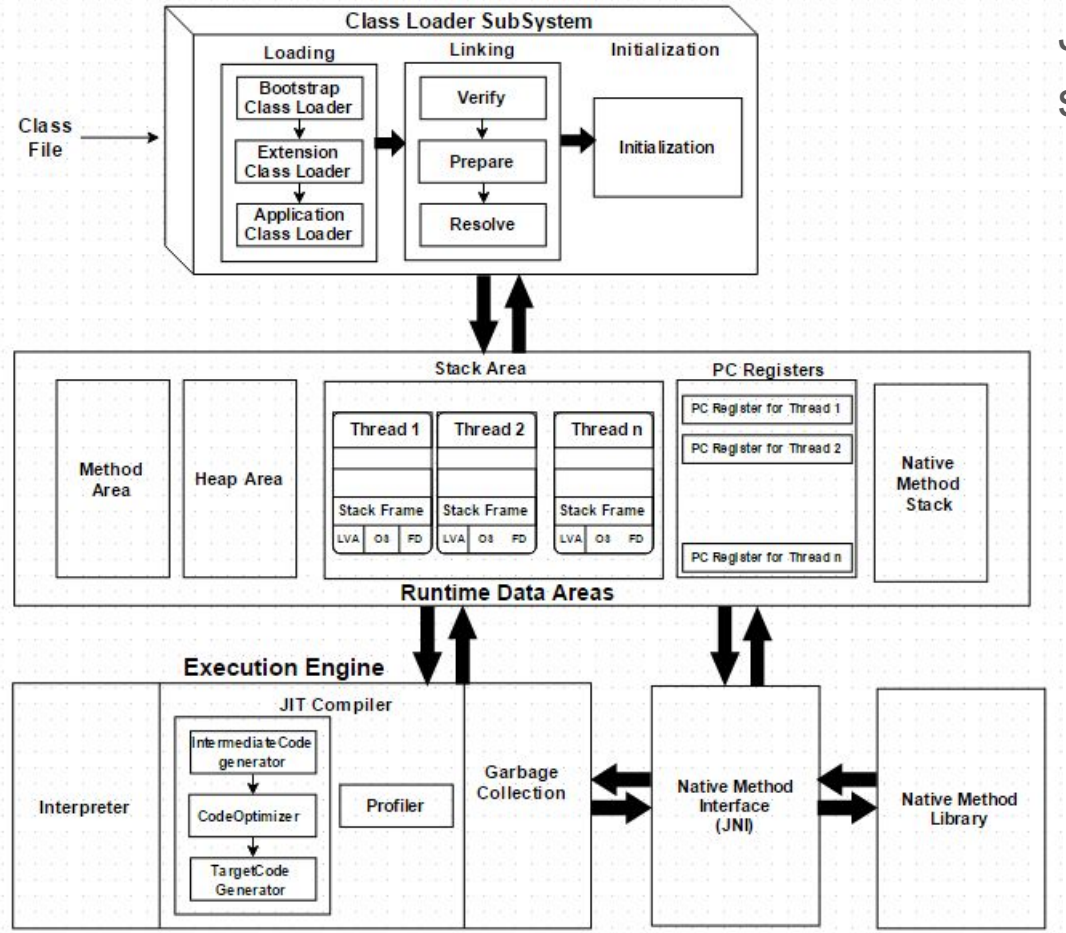
JVM interprets the .class file in to machine code depending on the operating system and hardware.

JVM executes java programs like a machine.

JVM is also responsible for garbage collection, array bound checking etc.



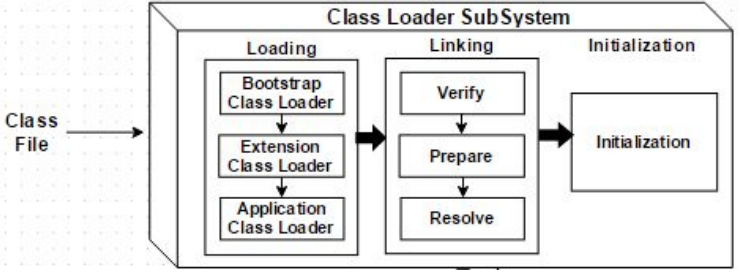
JVM Architecture



JVM is divided into three main subsystems:

1. ClassLoader Subsystem
2. Runtime Data Area
3. Execution Engine

JVM - ClassLoader Subsystem



ClassLoader subsystem loads, links. and initializes the class file when it refers to a class for the first time **at runtime**, not compile time.

1.1 Loading - Classes will be loaded by this component. BootStrap ClassLoader, Extension ClassLoader, and Application ClassLoader are the three

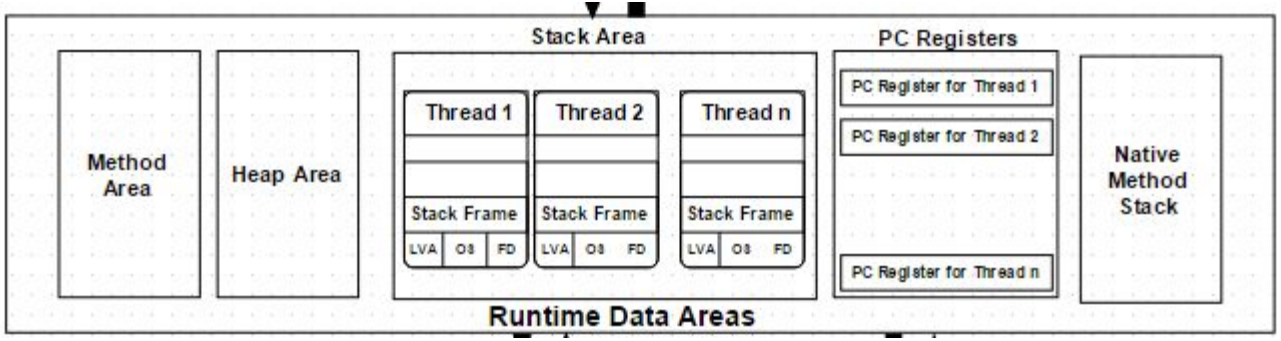
1.2 Linking - Performs verification , preparation

- Verify – Bytecode verifier will verify the generated bytecode
- Prepare – For all static variables memory will be allocated and assigned with defaults.
- Resolve – All symbolic memory references are replaced with the original references from Method Area.

1.3 Initialization

This is the final phase of ClassLoading; here, all static variables will be assigned with the original values, and the static block will be executed.

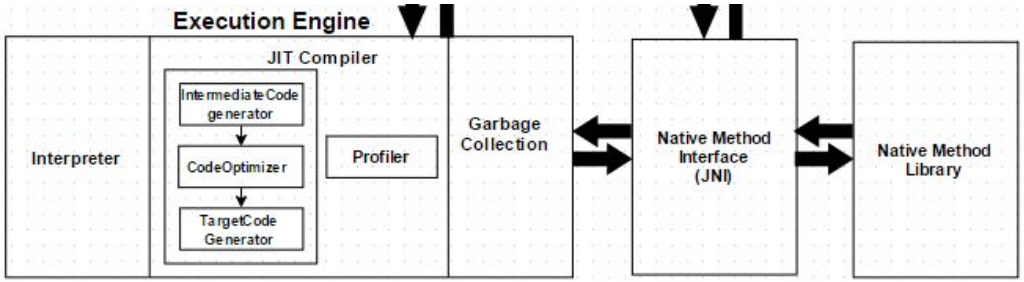
JVM - Runtime Data Area



These are **storage areas** needed during the execution of the program.

1. **Method Area** – Class-level data will be stored here, including static variables.
2. **Heap Area** – Objects and their corresponding instance variables and arrays
3. **Stack Area** – For every thread, a separate runtime stack will be created. For every method call, one entry will be made in the stack memory which is called Stack Frame.
    a. **Local Variable Array** – Related to the method, how many local variables are involved and the corresponding values will be stored here.
    b. **Operand stack** – acts as runtime workspace to perform operation.
    c. **Frame data** – To store all symbols corresponding to the methods.
4. **PC Registers** – Each thread will have separate PC Registers.
5. **Native Method stacks** – holds native method information.

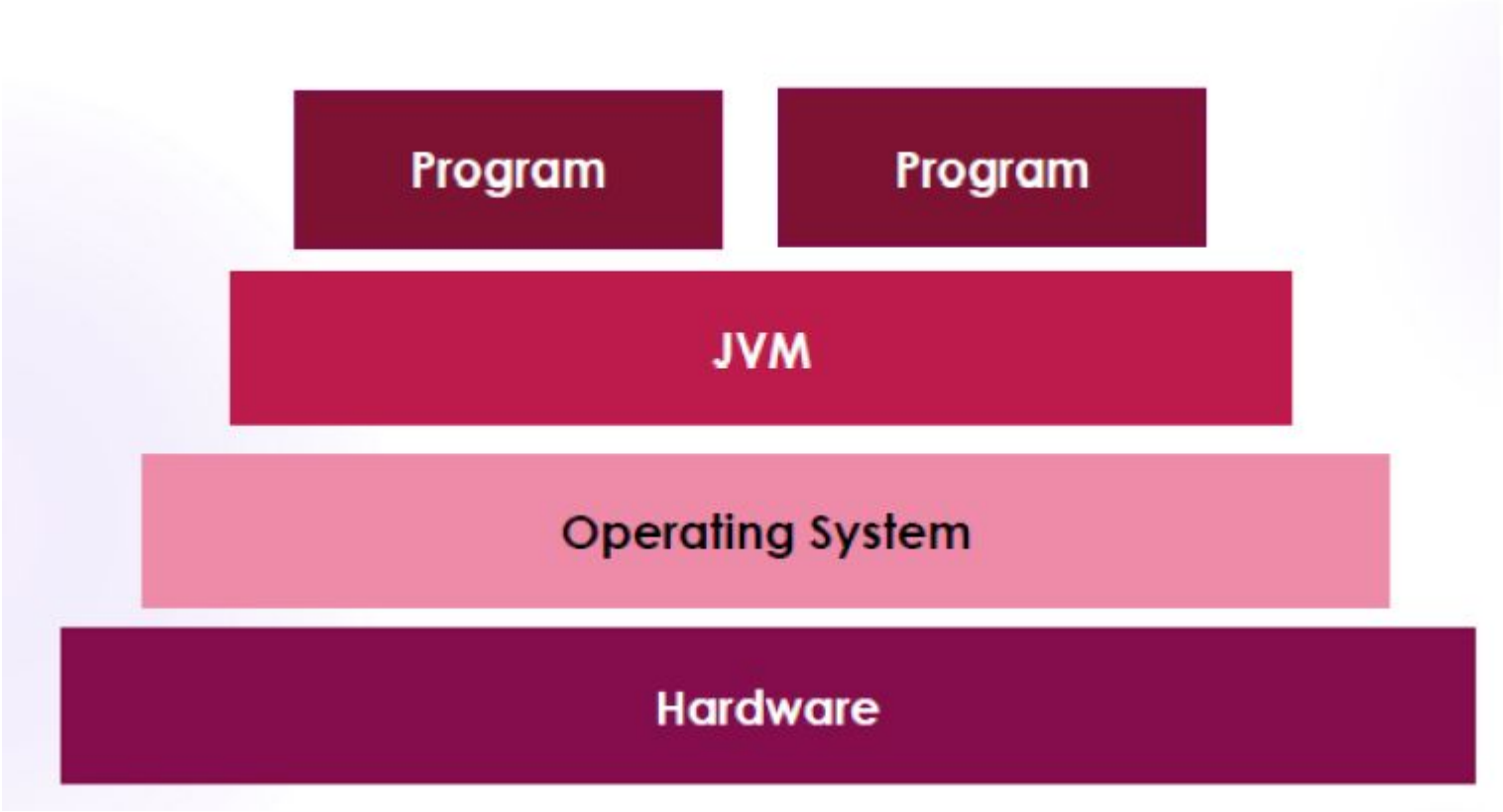
JVM - Execution Engine



The bytecode, which is assigned to the Runtime Data Area, will be executed by the Execution Engine(EE). It reads the bytecode and executes it piece by piece.

1. **Interpreter** – The interpreter interprets the bytecode faster but executes slowly.
2. **JIT Compiler** – When EE finds repeated code, it uses the JIT compiler, which compiles the entire bytecode and changes it to native code. This native code will be used directly.
    a. **Intermediate Code Generator** – Produces intermediate code
    b. **Code Optimizer** – Responsible for optimizing the intermediate code.
    c. **Target Code Generator** – Responsible for Generating Native Code
    d. **Profiler** – Responsible for finding hotspots in the code.
3. **Garbage Collector:** Collects and removes unreferenced objects.
4. **Java Native Interface (JNI)**: Interacts with the Native Method Libraries
5. **Native Method Libraries:** Native Libraries for the EE.

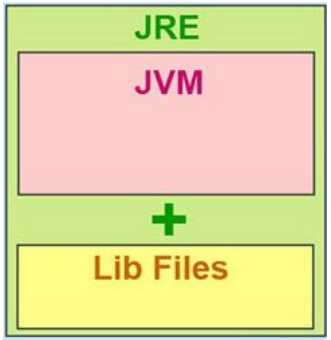
Java Programming and Runtime Environment

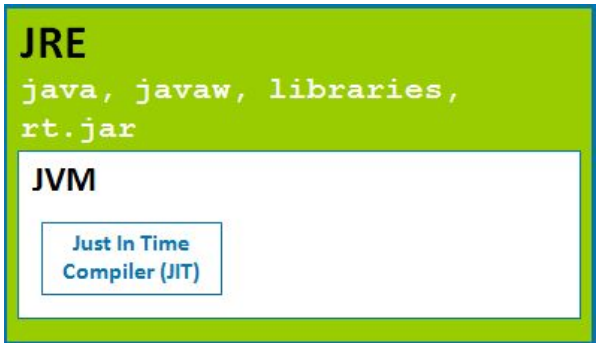


Java Runtime Environment (JRE)

- JRE provides the **minimum requirements** for **executing a Java** application programs.
- JRE is an **installation package** which provides environment to **only run**(not develop) the java program(or application)onto your machine.
- JRE can be view as a subset of Java Development Kit.
- JRE contains Java Virtual Machine (JVM),core classes, and supporting files
- JVM is a software implementation of a computer that executes programs like a real machine.

Java SE Runtime Environment 8 Downloads

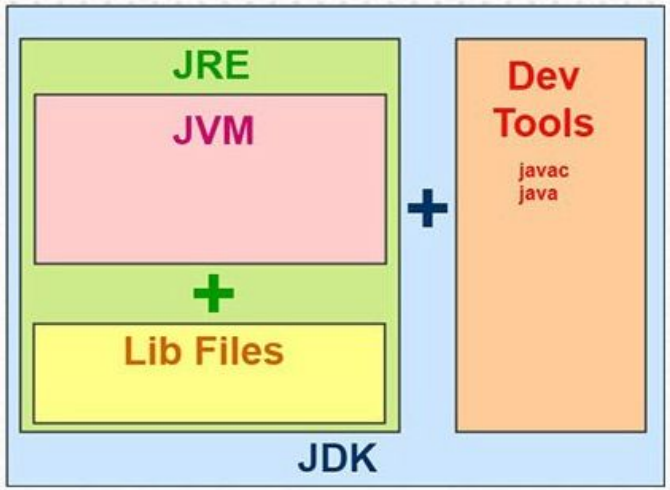


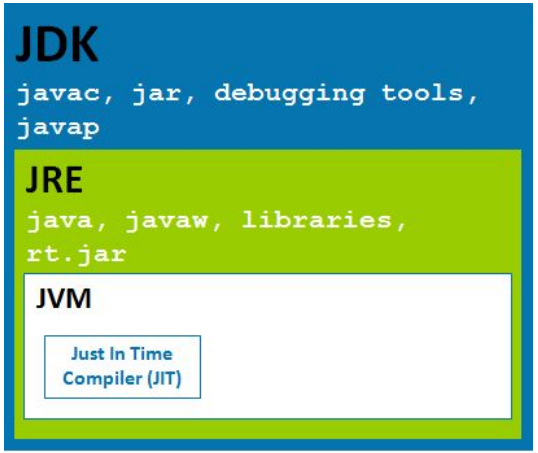


Java Development Kit (JDK)

- JDK is a software development environment used for developing and executing Java applications and applets
- It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development.
- JDK is only used by Java Developers.

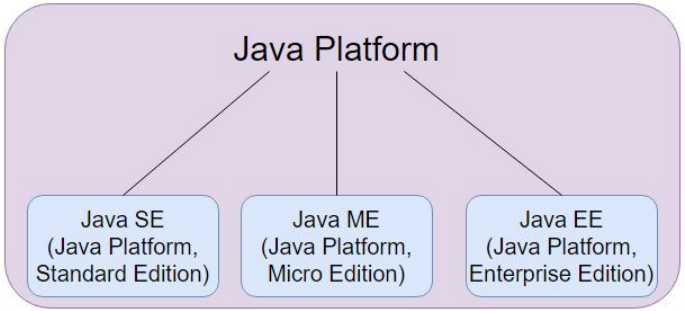


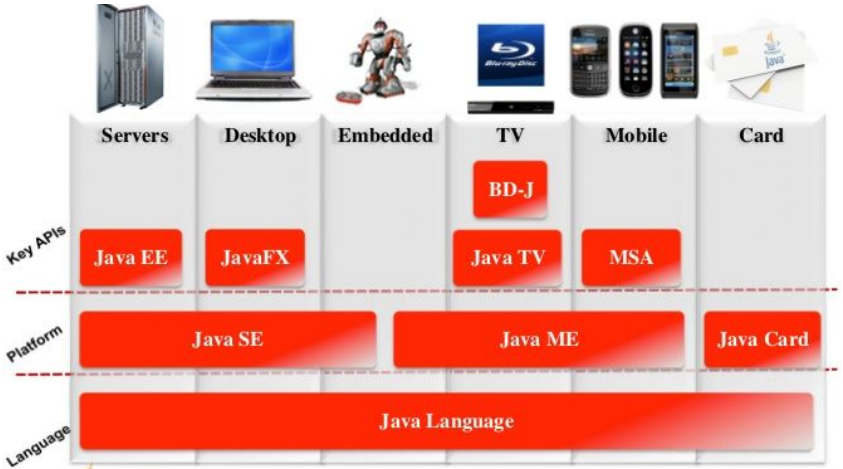




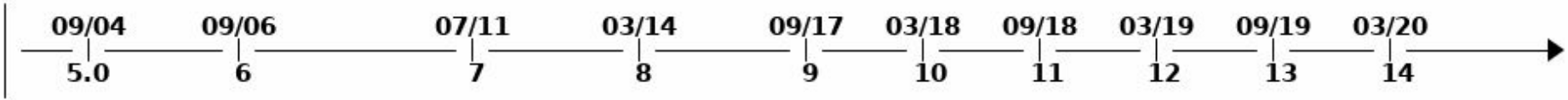
Development Platforms

- All Java platforms consist of a Java Virtual Machine (JVM) and an application programming interface (API).
- Allows applications written for that platform to run on any compatible system with all the advantages of the Java. Major platforms are
  1. Java Platform, Standard Edition (Java SE)
  2. Java Platform, Enterprise Edition (Java EE)
  3. Java Platform Micro Edition (Java ME)





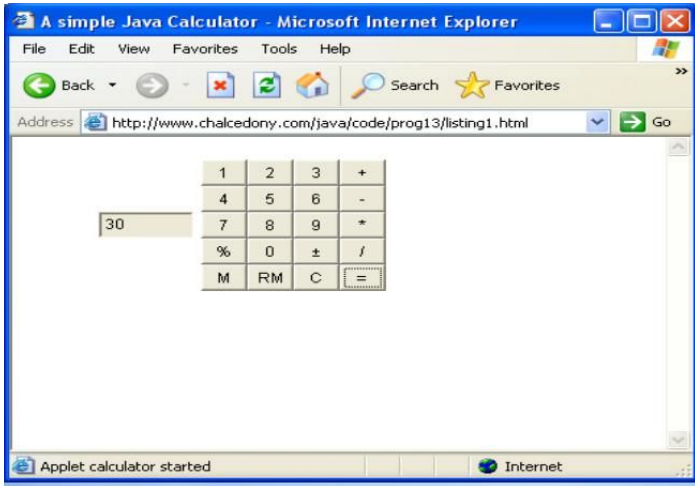
Java Platform, Standard Edition (Java SE)



- Java SE lets you develop and deploy Java applications on desktops and personal computers.
- Java SE's API provides the core functionality of the Java programming language.
- It defines everything from the basic types and objects of the Java programming language to high level classes that are used for networking, security, database access, GUI development, and XML
- In addition to the core API, the Java SE platform consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.
- **Latest version is - Java SE 26**

Java Applet

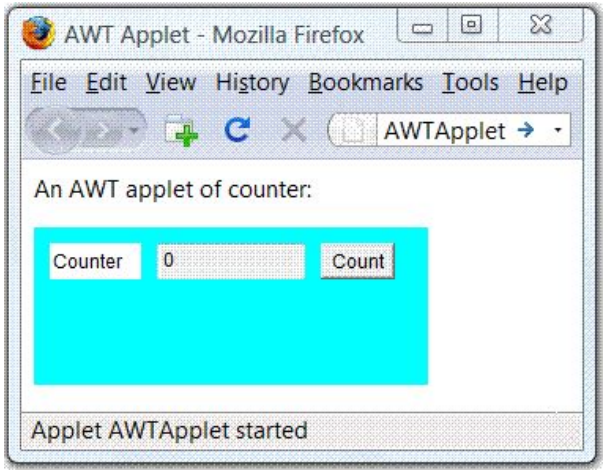




An applet is a Java program that usually runs a in a **Web browser**.

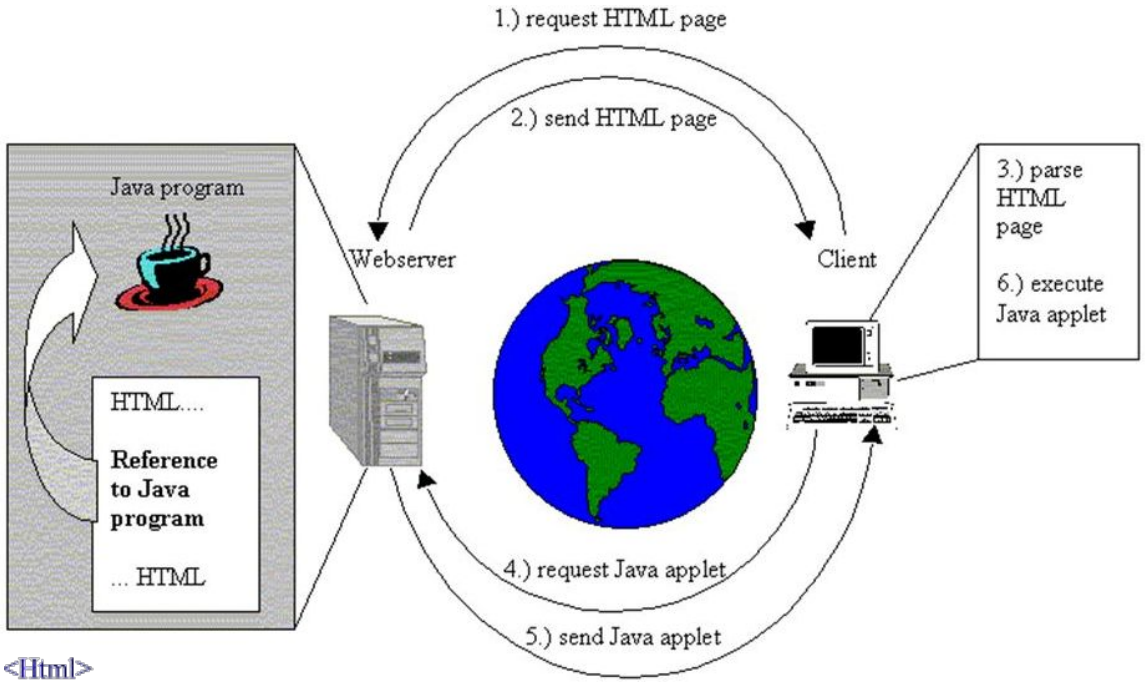
To make interactive web pages with better GUI





Java Applet

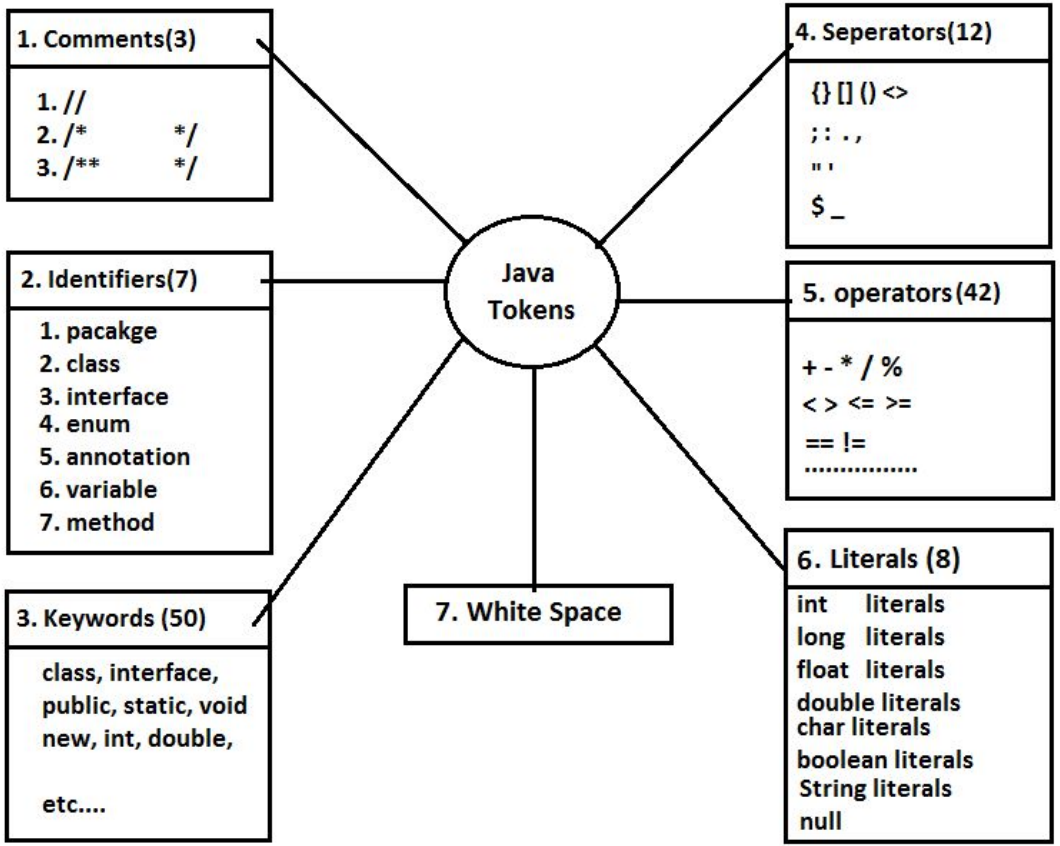
- An applet is a special kind of Java program that is designed to be **transmitted over the Internet** and automatically executed by a Java-compatible **web browser**.
- It runs inside the web browser and works at client side
- Applets are used to make the web pages more dynamic and entertaining




```
<Html>
<Head>
<Title>Java Applet Example>

<Body>
Below is the sample applet<br>
<br>
<Applet Code="SampleApplet.class" width=75 height=50
</Applet>
</Body>
</Html>
```

Java Tokens



- Tokens are the smallest individual building block or smallest unit of a program.
- Java compiler uses it for constructing expressions and statements.
- Java program is a collection of different types of **tokens**, comments, and white spaces.

Tokens - Keywords

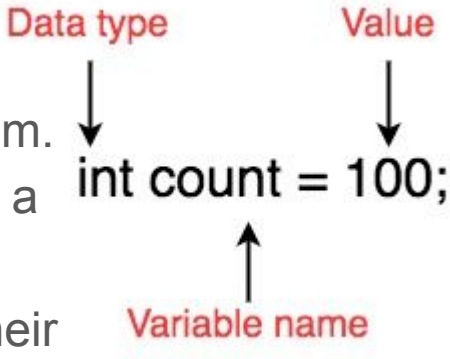
- A keyword is a reserved word. In Java there are 50 keywords.
- The meaning of the keywords has already been described to the java compiler. These meaning cannot be changed.
- Thus, the keywords cannot be used as variable names because that would try to change the existing meaning of the keyword, which is not allowed.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

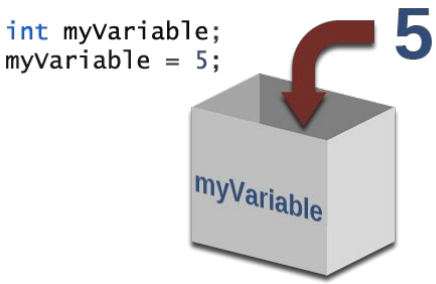
Tokens - Identifiers

- Identifiers are the names of variables, methods, classes, packages and interfaces
- Allowed characters for identifiers are all alphanumeric characters(**[A-Z],[a-z],[0-9]**), **‘$‘**(dollar sign) and **‘_‘** (underscore).
- All identifiers must start with either a letter(a to z or A to Z) or currency character($) or an underscore.
- They must not begin with a digit
- After the first character, an identifier can have any combination of allowed characters.
- Identifiers in Java are case sensitive, foo and Foo are different.
- A Java keywords cannot be used as an identifier.
- They can be any length
- Eg: **int a;** , **char name;**

Variables



- The variable is the basic unit of storage in a Java program.
- A variable is defined by the combination of an identifier , a type , and an optional initializer .
- In addition, all variables have a **scope** , which defines their visibility, and a **lifetime** which indicates how long the variable stays alive in the memory
- Here are several examples of variable declarations of various types. Note that some include an initialization
  - int a, b, c; // declares three ints , a, b, and c
  - int d = 3, e, f = 5; // declares three more ints , initializing d and f.
  - byte z = 22; // initializes
  - String Name = “Samia” ;





Variable naming rule

- Variable names are case sensitive.
- A variable's name can be any legal **identifier**
- The only allowed characters for identifiers are all alphanumeric characters (characters([A Z a z 0 9]), ‘$‘(dollar sign) and ‘_‘
- For example “geek@” is not a valid java identifier as it contain ‘@’ special character, geek$ is valid.
- Identifiers should not start with digits([0 9])9]).
- Reserved Words can’t be used as an identifier.
- For example “ int while = 20;” is an invalid statement as while is a reserved word.

Variable naming convention

- If the name you choose, consists of only one word, spell that word in all lowercase letters.
- If it consists of more than one word, capitalize the first letter of each subsequent word. This style is known as camel case.
- Examples
- gearRatio , currentGear
- If your variable stores a constant value, such as static final int
- NUM_GEARS = 6 , the convention changes slightly, capitalizing every letter and separating subsequent words with the underscore character.



Tokens - Constants or Literals

- Constants are fixed values of a particular type of data,
- Java language specifies five major type of literals.
- In Java, a character preceded by a backslash (\) is meaning.

Java Literals	Java Literal Examples			
Integer Literal	100	0	27530	18
Floating Point Literal	3.14	-0.536	26783.087	0.9
String Literal	"India"	"838365"	"Good"	"cat"
Character Literal	'A'	'a'	'#'	')'
Boolean Literal	true	false		

whicl

an

\t	Inserts a tab
\b	Inserts a backspace
\n	Inserts a newline
\r	carriage return. ()
\f	form feed
\'	Inserts a single quote
\"	Inserts a double quote
\\	Inserts a backslash

Literals

- Primitive types are special data types built into the language; they are not objects created from a class.
- A literal is the source code representation of a fixed value, that do not change during the execution; literals are represented directly in your code without requiring computation.
- As shown below, it's possible to assign a literal to a variable of a primitive type
  - boolean result = true;
  - char capitalC = 'C'
  - byte b = 100;
  - short s = 10000;
  - int i = 10 ;

Integer Literal	100	0	27530	18
Floating Point Literal	3.14	-0.536	26783.087	0.9
String Literal	"India"	"838365"	"Good"	"cat"
Character Literal	'A'	'a'	'#'	')'
Boolean Literal	true	false		

Integer literals

- Integer literal are used to represent a **decimal** , **binary** , or **hexadecimal** value.
- Integer literals are used to initialize variables of integer data types **byte, short, int** and **long.**
- Integer literal that ends with **l** or **L** is of type **long** .
- In Java, a binary literals starts with **0b** , and hexadecimal literals starts with
- **0x** and rest every integer literal is a decimal literals
- int decInt = 18; // 18 in decimal notation
- int binInt = 0b10010; // 18 in binary notation
- int hexInt = 0x12; // 18 in hexadecimal notation
- long longValue = 123456789L ;

Floating point literals

- A floating point literal is of type float if it ends with the letter F or f
- Otherwise its type is double and it can optionally end with the letter D or d
- The floating point types (float and double) can also be expressed using E or e (for scientific notation
- **double myDouble = 123.4 ;**
- **double myDoubleScientific = 1.234e2 ;// same value as d1, but in scientific notation**
- **float myFloat = 123.4f ;**

Character and String literals

- Literals of types char and String may contain any Unicode (UTF 16) characters.
- String literals are represented as a sequence of characters surrounded by double quotes
- character literal is represented as a single character surrounded by single quotes.
- Java also allows use of **escape sequences** in string and character literals. [\b (backspace), \t (tab), \n (line feed), \f (form feed), \r (carriage return), \” (double quote), \’ (single quote), and \\(backslash]
- char myChar = 'A '; // Character Literal
- char newLine = n ';';// Character Literal Escape sequence
- String myString = "Java Tutorial "; // String Literal

Underscore characters in numeric literals

- In Java SE 7 and later, any number of underscore characters (_) can appear anywhere between digits in a numerical literal.
- This feature enables you, for example. to separate groups of digits in numeric literals, which can improve the readability of your code
- The following example shows other ways you can use the underscore in numeric literals:
  - long creditCardNumber = 1234_5678_9012_8906L ;
  - long socialSecurityNumber = 999_99_990 ;
  - float pi = 3.14_15F;
  - long hexBytes = 0xFF_EC_DE_AB ;
  - long hexWords = 0xCAFE_AABB ;
  - long maxLong = 0x7fff_ffff_ffff_ffffL ;
  - byte nybbles = 0b0010_0101 ;
  - long bytes = 0b11010010_01101001_10010100_10010010;

Underscore characters in numeric literals

- You can place underscores only between digits; you cannot place underscores in the following places
- At the beginning or end of a number
- Adjacent to a decimal point in a floating point literal
- Prior to an F or L suffix
- In positions where a string of digits is expected

```
// Invalid: cannot put underscores adjacent to a decimal point
float pi1 = 3_.1415F;
// Invalid: cannot put underscores prior to an L suffix
long socialSecurityNumber1 = 999_99_9999_L;
// Invalid: cannot put underscores in the 0x radix prefix
int x4 = 0_x52;
// Invalid: cannot put underscores at the beginning of a number
int x5 = 0x_52;
// OK (hexadecimal literal)
int x6 = 0x5_2;
// OK (decimal literal)
int x3 = 5_______2;
```

Tokens - Separators

- There are few symbols in java that are used as separators.
- A separator is a sequence of one or more characters used to specify the boundary between separate, independent regions in plain text or other data stream.
- The most commonly used separator in java is the semicolon ' ; '.

Separators	Comments	Usage
( )	Method declaration and arguement passing	main(String[] args)
{ }	Block of code	try{ for(int i : j){ } }
[ ]	Defining array and accessing its element	String[] args
;	End of an statement	int i =0 ;
,	Argument Separation	int i , j , k ;
.	Calling Operator (method,objects)	System.out.println("");
:	For-each loop	for(ArrayList a : list)

Tokens - Operators

- An operator is a symbol that takes one or more arguments and operates on them to produce a result.

1. Unary Operator
2. Arithmetic Operator
3. Shift Operator
4. Relational Operator
5. Bitwise Operator
6. Logical Operator
7. Ternary Operator
8. Assignment Operator

Operator	Category	Precedence
Unary Operator	postfix	expression++ expression--
	prefix	++expression --expression +expression -expression ~!
Arithmetic Operator	multiplication	* / %
	addition	+ -
Shift Operator	shift	<< >> >>>
Relational Operator	comparison	< > <= >= instanceof
	equality	== !=
Bitwise Operator	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical Operator	logical AND	&&
	logical OR	||
Ternary Operator	ternary	? :
Assignment Operator	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Tokens - Comments & White space

White space

- Java is a free-form language. This means that you do not need to follow any special indentation rules
- White space in Java is used to **separate tokens** in the source file.
- It is also used to **improve readability** of the source code. Eg: int i = 0;
- White spaces are required in some places. For example between the int keyword and the variable name.
- In java whitespace is a space, tab, or newline

Comments

- Java supports 3 types of comments.

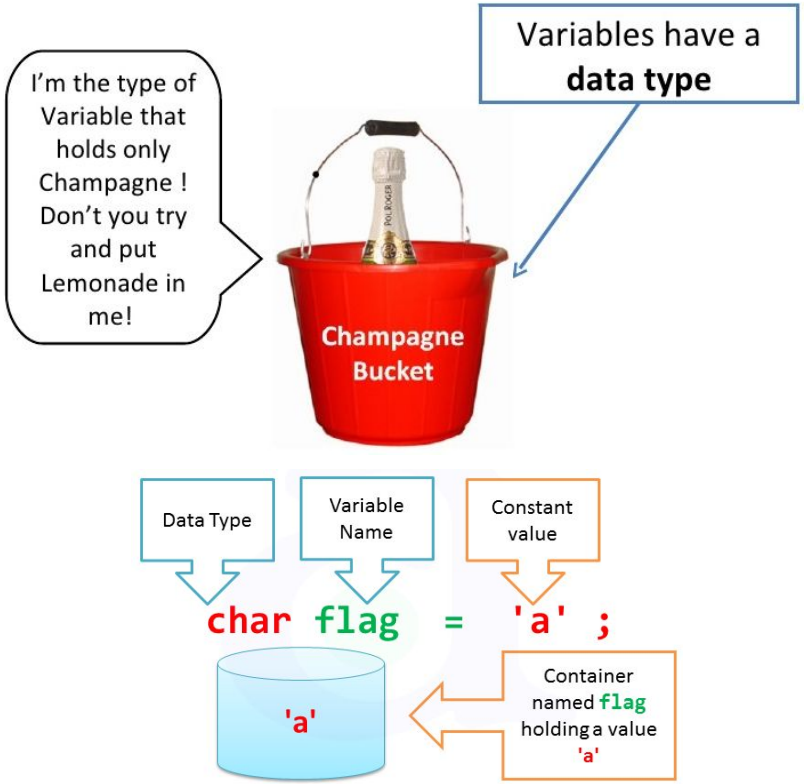


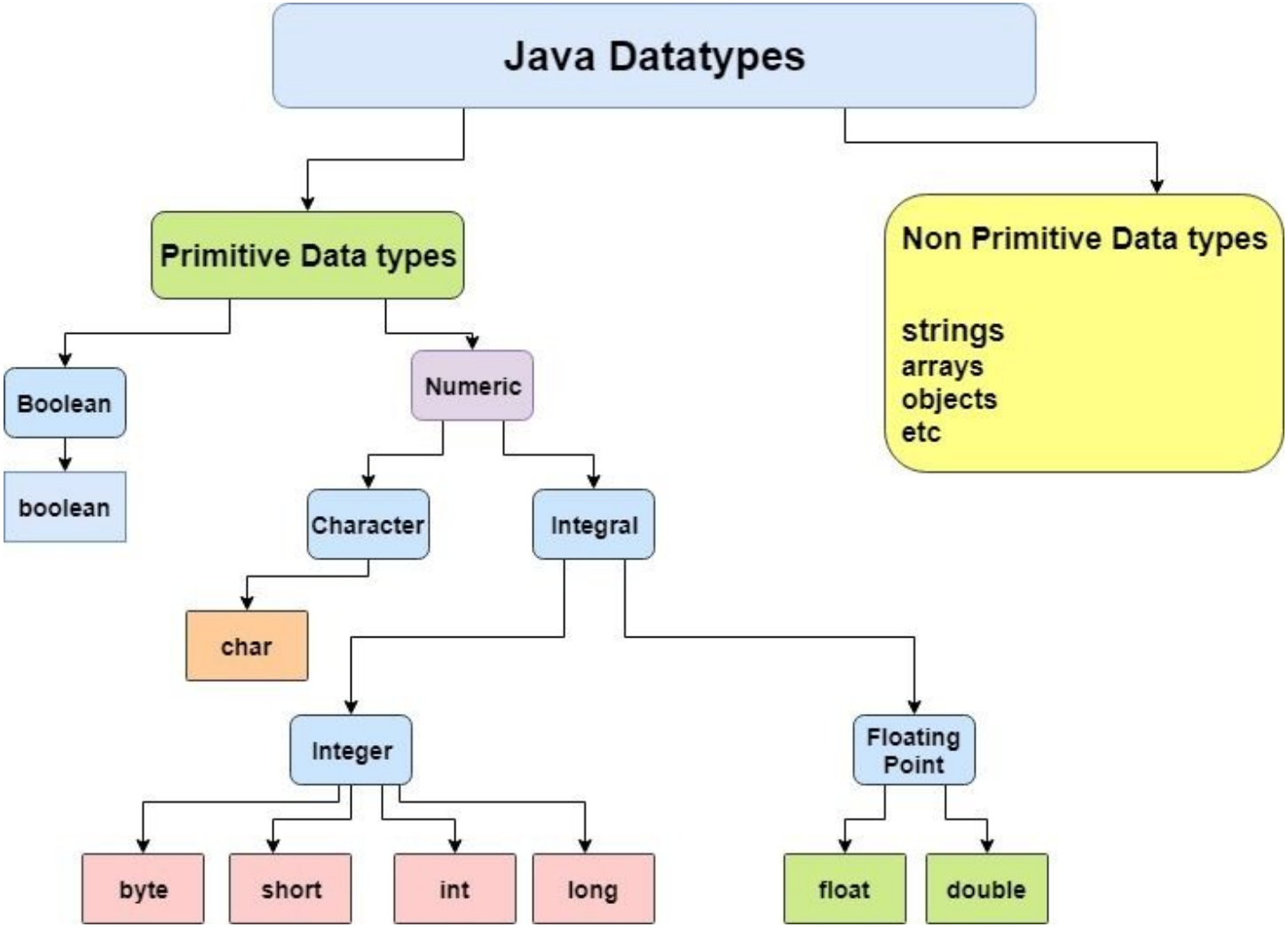




Data type

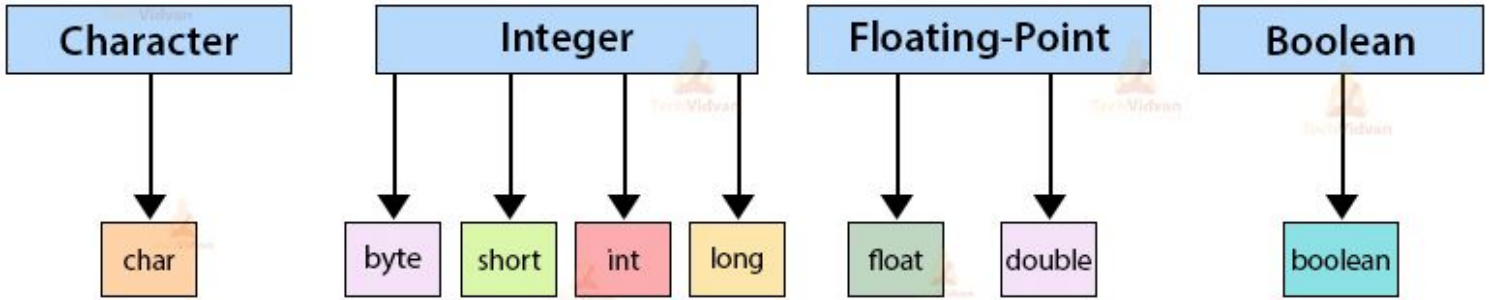
- Data type defines the type of **values that a variable can hold**
- Java is **statically typed**, which means that all variables must first be declared before they can be used.
- Java is **strongly typed**, every variable/expression has a strictly defined data type.
- The Java compiler checks all expressions and parameters to ensure that the types are compatible.
- Any type **mismatches are errors** that must be corrected by the programmer.




Java Datatypes
Primitive Data types
Non Primitive Data types
strings
arrays
objects
etc
Boolean
boolean
Numeric
Character
char
Integral
Integer
byte
short
int
long
Floating Point
float
double

Primitive data type

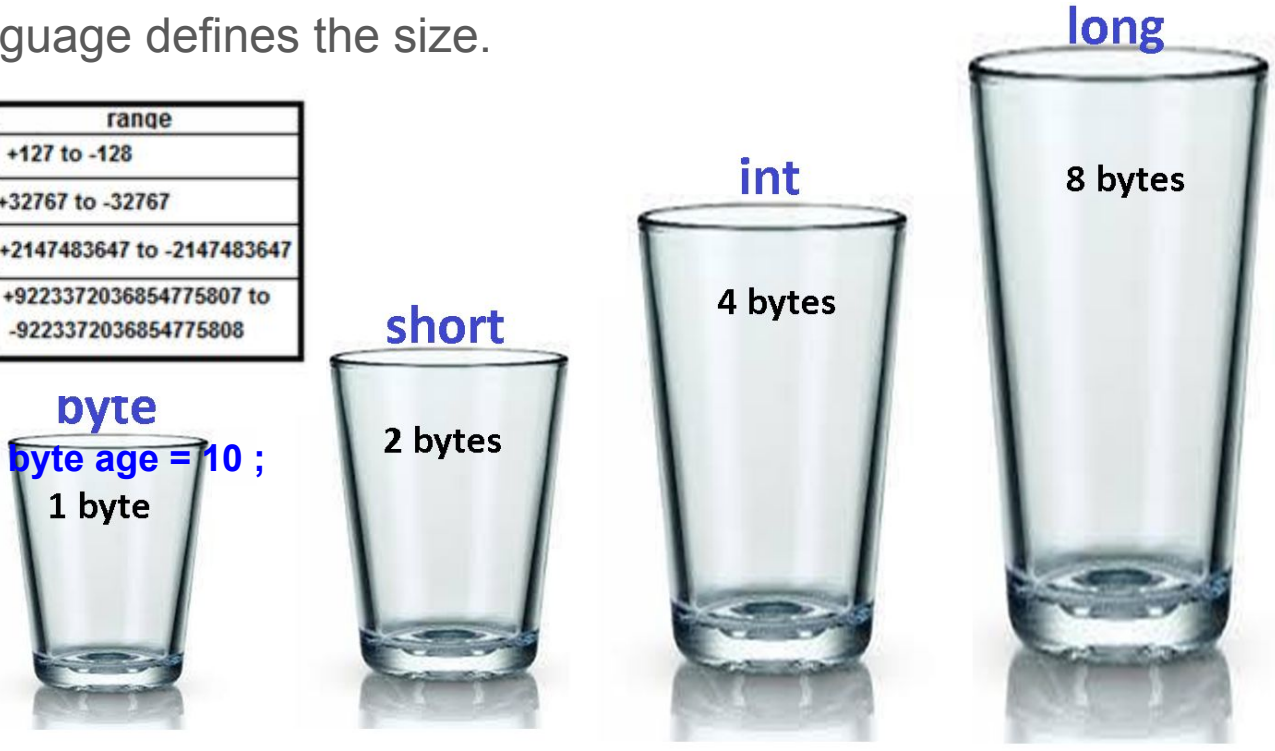
- A primitive type is predefined by the language and type name is a keyword.
- In Java there are 8 primitive data types: byte, short, int , long, char, float, double, and boolean
- Integers : byte, short, int , and long , which are for whole valued signed numbers.
- Floating point numbers : float and double , which represent numbers with fractional precision.
- Characters char , which represents symbols in a character set.
- Boolean boolean , represents true/false values.



Primitive data type - Integer

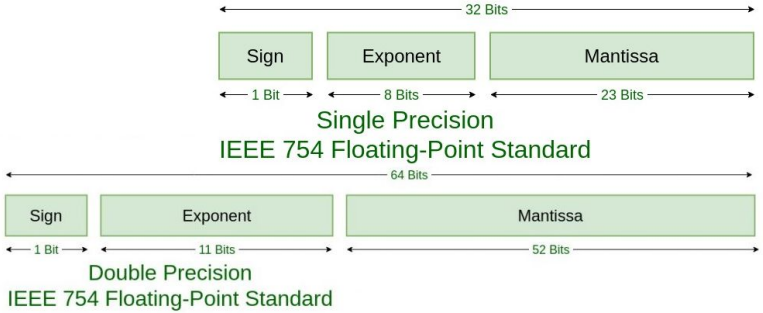
- The integer data types are used to store **whole numbers** like 2, 3 -1245 etc.
- There are 4 types to handle signed whole numbers
- In C/C++, size of int is dependant upon the platform.
- In Java, language defines the size.

sno	Data type	Size(in bytes)	range
1	byte	1	+127 to -128
2	short	2	+32767 to -32767
3	int	4	+2147483647 to -2147483647
4	long	8	+9223372036854775807 to -9223372036854775808



Primitive data type - Floating point

- Floating-point numbers: This group includes **float** and **double**, which represent numbers with fractional precision
- Java implements the standard (IEEE–754) set of floating-point types
- **float** - single-precision 32-bit
- **double** - precision 64-bit format



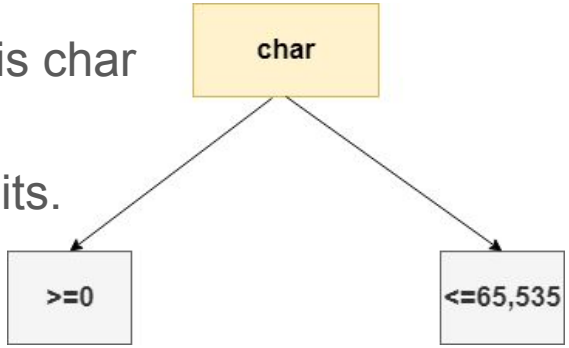

```
float f  = 10.111111111;
```

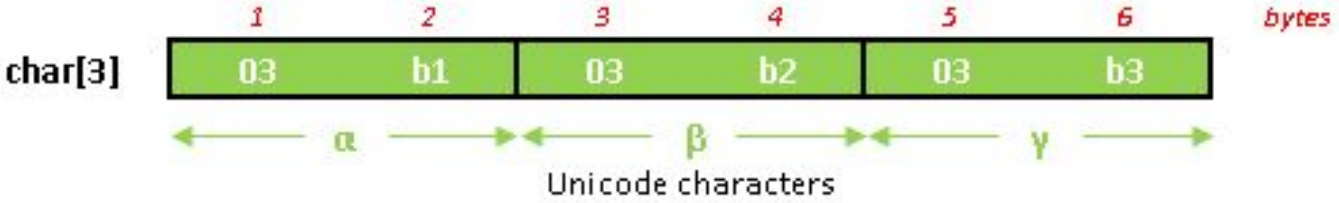
```
float f  = 10.111111111f;
System.out.println(f);
output : - 10.111111 (6)
```

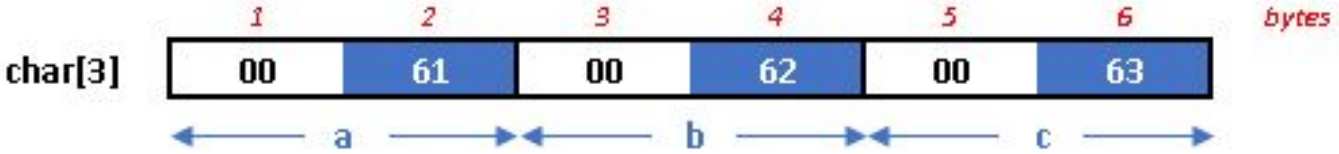
```
double d = 10.111111111;
System.out.println(d);
output : - 10.111111111(15)
```

Primitive data type - Character

- In Java, the char data type used to store characters is char
- Java uses Unicode to represent characters
- At the time of Java's creation, Unicode required 16 bits. Thus, in Java char is a 16 bit type.
- Example: **char letter = 'A';**

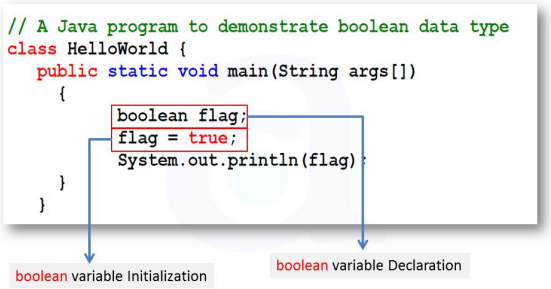






Primitive data type - Boolean

- boolean: The boolean data type has only two possible values: true and false. Use this data type for simple flags that track true/false conditions.
- This data type represents one bit of information, but its "size" isn't something that's precisely defined
- Size of boolean data type is virtual machine-dependent. Usually 1 byte.
- Values of type boolean are not converted implicitly or explicitly (with casts) to any other type.
- This is the type returned by all relational operators, as in the case of a < b .
- Example: boolean b;



Java Wrapper Classes

- Primitive type variables and not objects.
- Wrapper classes provide a way to use primitive data types (int, boolean, etc..) as objects.
- Primitive wrapper classes are used to create an Object that needs to represent primitive types in java

Datatypes	Size	Default value	Wrapper class
byte	8 bits	0	Byte
boolean	1 bit	False	Boolean
int	32 bits	0	Integer
char	16 bits	\u0000	Character
float	32 bits	0.0f	Float
double	64 bits	0.0d	Double
long	64 bits	0L	Long
short	16 bits	0	Short

Wrapper Types

- The type wrappers or Wrapper types are
- **Character**
- **Double**
- **Float**
- **Long**
- **Integer**
- **Short**
- **Byte**

- **Boolean**.

Character wrapper

- **Character** is a wrapper around a **char**. The constructor for **Character** is

  Character(char *ch*)

- To obtain the **char** value contained in a **Character** object, call **charValue( )**:

  char charValue( )

  - It returns the encapsulated character.

Boolean wrapper

- **Boolean** is a wrapper around **boolean** values. It defines these constructors:

Boolean(boolean *boolValue*)

Boolean(String *boolString*)

- To obtain a **boolean** value from a **Boolean** object, use **booleanValue( )**, :

boolean booleanValue( )

  - It returns the **boolean** equivalent of the invoking object.

The Numeric Type Wrappers

- **Byte**, **Short**, **Integer**, **Long**, **Float**, and **Double**
- All of the numeric type wrappers inherit the abstract class **Number.**
- All of the numeric type wrappers define constructors that allow an object to be constructed from a given value, or a string representation of that value
- Functions are:

byte byteValue( ) // returns the value of an object as a **byte**

double doubleValue( ) // returns the value of an object as a **double**

float floatValue( )

int intValue( )

long longValue( )

short shortValue( )

Integer wrapper

Integer(int *num*)

Integer(String *str*)

Example-Integer wrapper

```
class Wrap
 {
     public static void main(String args[])
      {
      Integer ob = new Integer(100);
      int i = ob.intValue();
      System.out.println(i + " " + ob);
      }
}
```

OUTPUT:

100 100

Wrapper-boxing and unboxing

- **BOXING:**The process of encapsulating a value within an object is called ***boxing.***
  - The following statement boxes the value 100 into an **Integer**:

    Integer ob = new Integer(100);
- **UNBOXING:** The process of extracting a value from a type wrapper is called ***unboxing***.
  - the following statement unboxes the value in **ob** :

    int i = ob.intValue();

AutoBoxing

- AutoBoxing is the process by which a primitive type is **automatically encapsulated** (boxed)into its equivalent type wrapper whenever an object of that type is needed.
  - There is no need to explicitly construct an object.

- Auto-unboxing is the process by which the value of a boxed object is **automatically extracted** (unboxed) from a type wrapper when its value is needed.
  - There is no need to call a method such as **intValue( )** or **doubleValue( )**.

AutoBoxing|UnBoxing -Advantages

- removes tedious manually boxing and unboxing values.
- Helps to prevent errors.

AutoBoxing|Autounboxing

- To construct an **Integer** object that has the value 100:

```
Integer ob = 100;
```

(No need to use construtor as in:- Integer ob = new Integer(100);)

- To unbox an object, simply assign that object reference to a primitive-type variable.
  - For example, to **unbox** the **ob** object in the above code,

```
int i = ob;
```

Example-Autoboxing|unboxing

```
// Demonstrate autoboxing/unboxing.
class Sample
{
public static void main(String args[])
{
    Integer ob = 100;           // autobox an int
    int i = ob;                 // auto-unbox
    System.out.println(i + " " + ob);
}
}
```

Output:
100
100

Autoboxing and Methods

- Autoboxing/unboxing might occur
  - when an argument is passed to a method, or
  - when a value is returned by a method.

Autoboxing/Unboxing Occurs in Expressions

- Within an expression, a numeric object is automatically unboxed.
- The outcome of the expression is reboxed, if necessary,

Autoboxing/Unboxing Boolean and Character Values

- Autoboxing/unboxing applies to these **Boolean** and **Character** wrappers, too

Autoboxing/Unboxing Helps Prevent Errors

- Autoboxing/unboxing can also help prevent errors.

```
// An error produced by manual unboxing.
class UnboxingErrorExample
 {
public static void main(String args[])
{
        Integer ob = 1000;              // autobox the value 1000
        int i = ob.byteValue();         // manually unbox as byte !!!
        System.out.println(i);          //  THIS DOES NOT DISPLAY THE VALUE 1000
}
}
```

Variable scope & life time

```
public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}
```

Lifetime of the parameter variables **a** *and* **b**

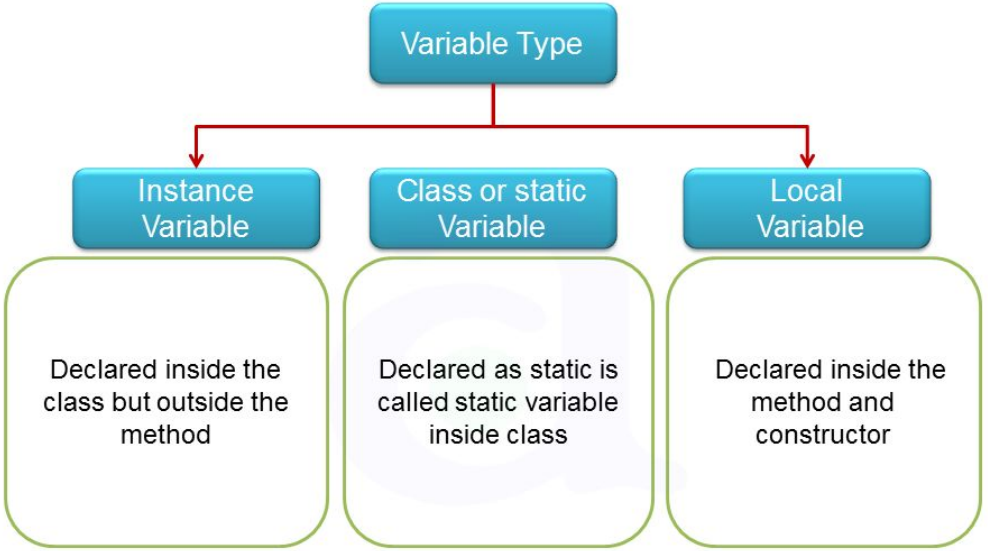
```
public class Scope2
{
   public static void main ( String[] args )
   {
      {
         double r;
         r = 3.14;
      }
      System.out.println( r );
   }
}
```

Scope of the variable **r**

use variable OUTSIDE its scope Error !!

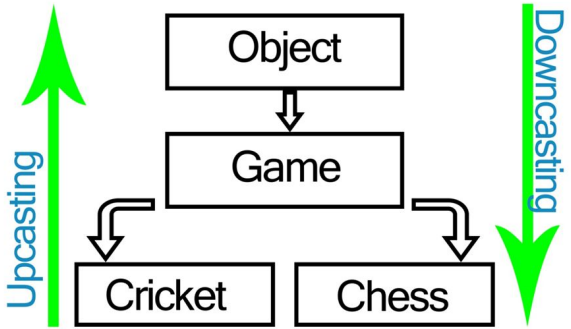
Types of Variables

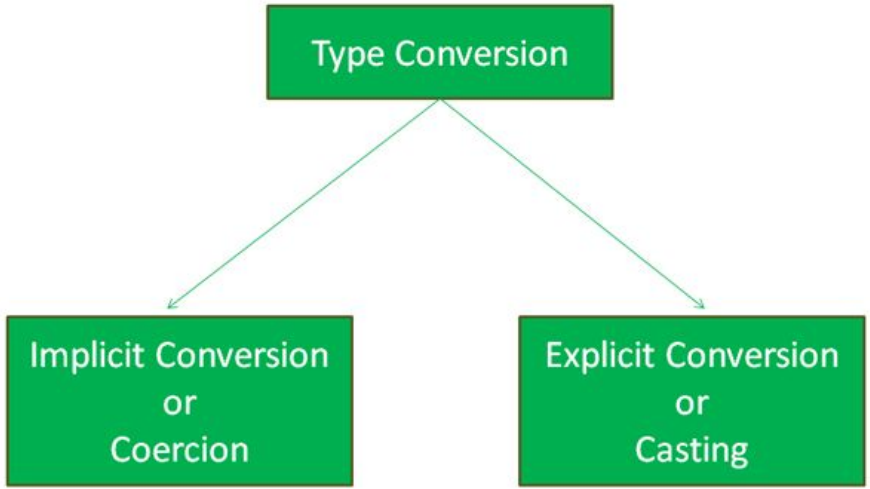
```
class A
{
    int data=50;   //instance variable
    static int m=100;  //static variable
    void method()
    {
        int n=90;  //local variable
    } //end of method
} //end of class
```



Type conversion and casting

- When a data type is converted into another type is called type conversion
- When a variable is converted into a different type, the compiler basically treats the variable as of the new data type.
- Type of Type Conversion In Java
- Implicit Conversion
- Explicit Conversion

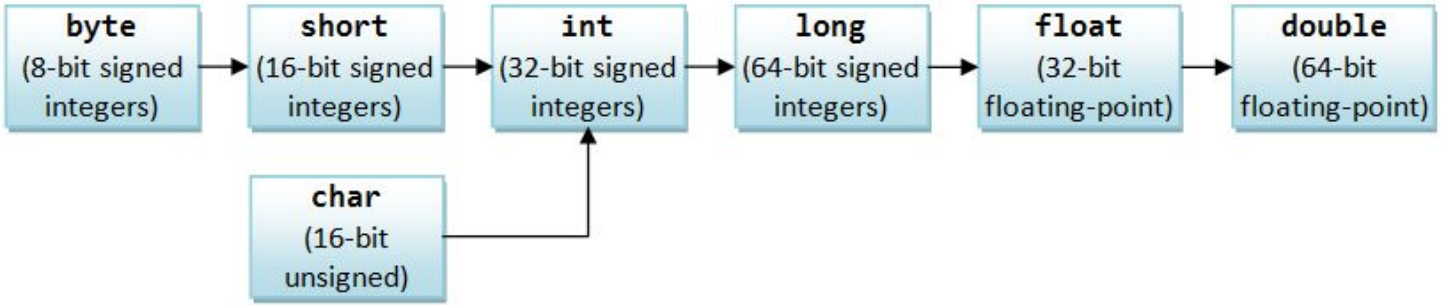




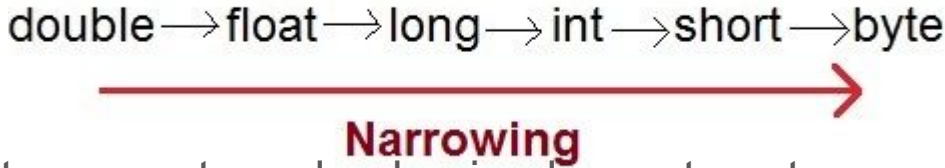
Implicit conversion.



- Implicit or automatic conversion compiler will automatically change one type of data into another without data loss.
- Implicit or automatic conversion can happen if both type are compatible and target type is larger than source type.
- If you assign an integer value to a floating point variable, the compiler will insert code to convert the int to a float.
- In implicit conversion a smaller type is converted into a larger thus it is also known as widening conversion.



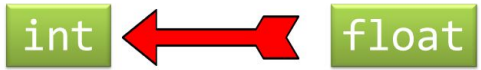
Explicit conversion.



- There may be situations when we want to convert a value having larger type to a smaller type.
- In this case casting needs to be performed explicitly.
- Explicit casting allows you to make this type conversion explicit, or to force it when it wouldn't normally happen.
- Syntax: (type) expression
- To perform type casting, put the desired type including modifiers inside parentheses to the left of the variable or constant you want to cast.

```
double d   = 10;
int i;
i = (int) d
```

Type Cast Operator



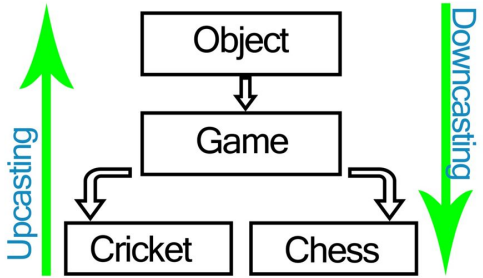

```
int number;
float fval= 32.33f;
number= (int)fval;
```

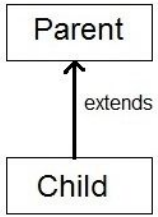
Type in which you want to convert

Variable name Which you want to convert

Upcasting & Downcasting.

- Just like the data types, the objects can also be type casted.
- In objects, there are only two types of objects (i.e.) parent object and child object.
- One type of object (i.e.) child or parent can be converted to another.
- There are two types of typecasting. They are:
- **Upcasting**: Upcasting is the typecasting of a child object to a parent object. Upcasting can be done implicitly.
- **Downcasting**: Similarly, downcasting means the typecasting of a parent object to a child object. Downcasting cannot be implicitly.





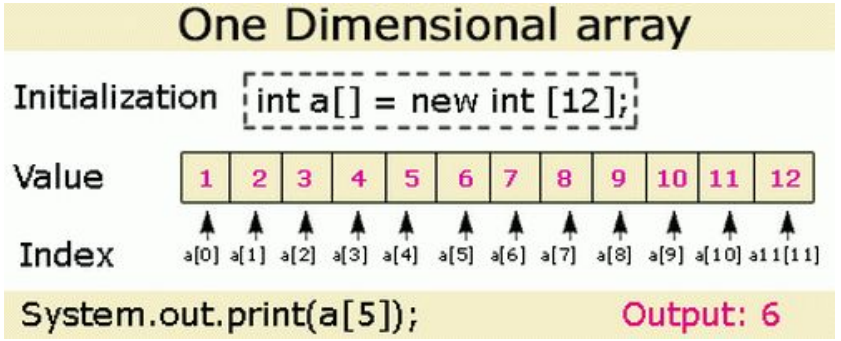

```
Parent p = new Child( );
```
Upcasting

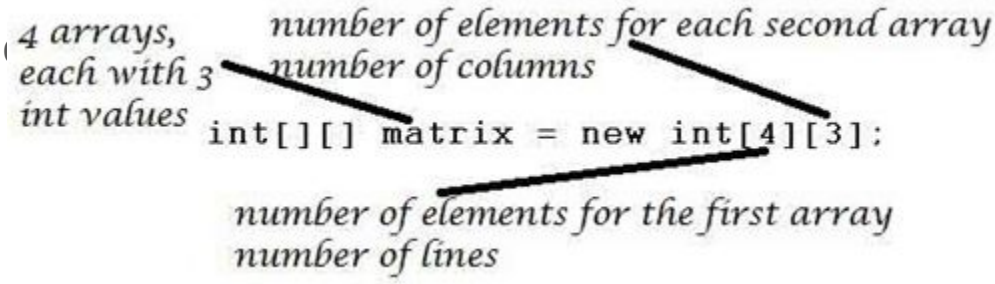
```
Child c = new Parent( );
```
Compile time error

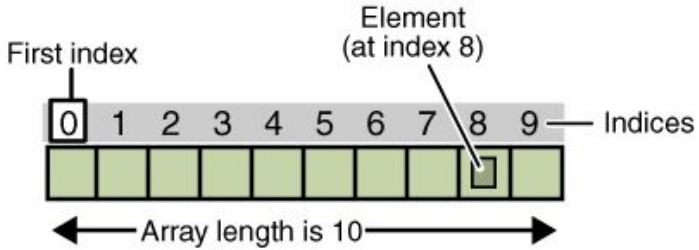
```
Child c = ( Child ) new Parent( );
```
Downcasting but throws ClassCastException at runtime.

Arrays in Java

- Arrays are used to store multiple values of same type, in a single variable
- Arrays are objects which store multiple values of the **same type**
- It can hold primitive types as well as references.
- Features of Array
- Arrays are objects
- They can even hold the reference variables of other objects
- They are created during **runtime (**new**)**
- They are dynamic, created on the **heap**
- The Array length is fixed.

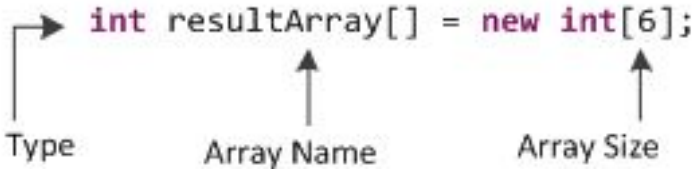


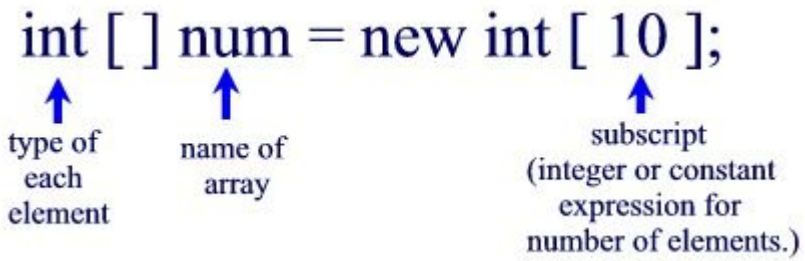




Array declaration

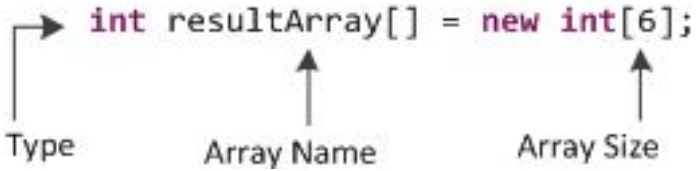
- To declare an array, define the variable type with square brackets
- Eg: int[] anArray;
- You can also place the brackets after the array's name:
- // this form is discouraged
- float anArrayOfFloats[];
- However, convention discourages this form; the brackets identify the array type and should appear with the type designation.

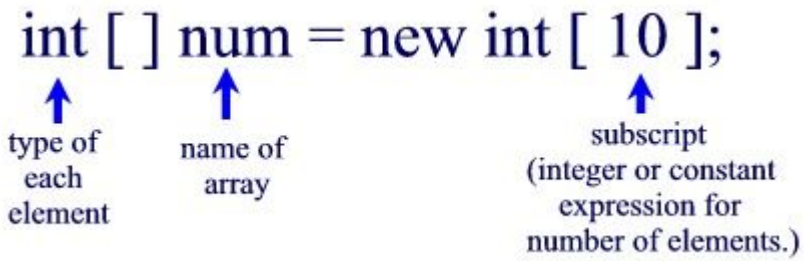




Array declaration

- To declare an array, define the variable type with square brackets
- Eg: int[] anArray; // Creates array reference
- You can also place the brackets after the array's name:
- // this form is discouraged
- float anArrayOfFloats[]; // Creates array reference
- However, convention discourages this form; the brackets identify the array type and should appear with the type designation.





Definition, Initializing and Accessing arrays

One way to create an array is with the new operator.

```
// declares an array of integers
int [] anArray // create an array of integers
anArray = new int [10] ;
```

To assign values to each element of the array

```
anArray[0] = 100; // initialize first element
anArray[1] = 200; // initialize second element
anArray[2] = 300; // and so forth
```

Each array element is accessed by its numerical index

```
System.out.println("Element 1 at index 0: " + anArray [0]);
System.out.println("Element 2 at index 1: " + anArray [1]);
System.out.println("Element 3 at index 2: " + anArray [2]);
```

Array length

Alternatively, you can use the shortcut syntax to create and initialize an array

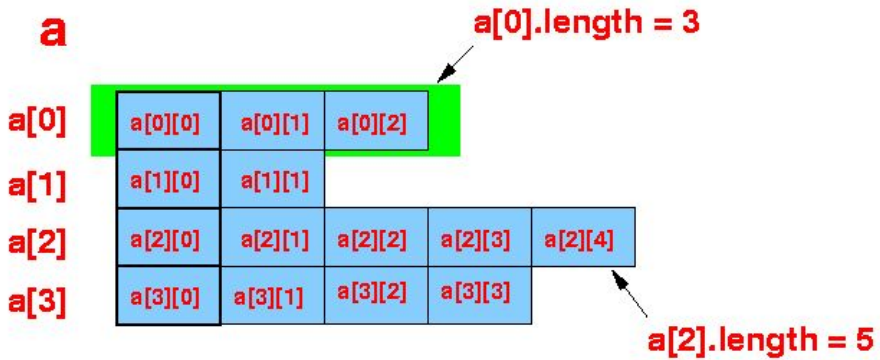
int[] anArray = {100, 200, 300};

Here the length of the array is determined by the number of values provided between braces and separated by commas.

To find out how many elements an array has, use the length property:

String[] cars = {"Volvo", "BMW", "Ford"};

System.out.println(cars.length);// Outputs 3



Multidimensional Arrays

- A multidimensional array is an array of arrays. Each element of a multidimensional array is an array itself. For example int[][] anArray = new int [3][4];
- To create a two dimensional array, add each array within its own set of curly braces:
- int [][] anArray ={ {1, 2, 3}, {4, 5, 6, 9} };
- //calculate the length of each row
- System.out.println("Length of row 1: " + anArray [0].length);
- System.out.println("Length of row 2: " + anArray [1].length);

Iterating array elements

- We can use for...each loop or for loop to access elements of the multidimensional array.

```
public class MultidimensionalArrayDemo1 {

    public static void main(String[] args) {
        int[][] anArray = {
                {1, -2, 3},
                {-4, -5, 6, 9},
                {7},
            };

            for (int i = 0; i < anArray.length; ++i) {
                for(int j = 0; j < anArray[i].length; ++j) {
                    System.out.print(anArray[i][j]+"\t");
                }
                System.out.print("\n");
            }

    }

}
```

```
public class MultidimensionalArrayDemo {

    public static void main(String[] args) {
        // create a 2d array
        int[][] anArray = {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        // first for...each loop access the individual array
        // inside the 2d array
        for (int[] innerArray: anArray) {
            // second for...each loop access each element inside the row
            for(int element: innerArray) {
                System.out.print(element+"\t");
            }
            System.out.print("\n");
        }

    }
```

Copying array

- The System class has an arraycopy method that you can use to efficiently copy data from one array into another

public static void arraycopy (Object src , int srcPos, Object dest , int destPos , int length) ;

```
public class ArrayCopyDemo {

    public static void main(String[] args) {
        char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
                'i', 'n', 'a', 't', 'e', 'd' };
        char[] copyTo = new char[7];

        System.arraycopy(copyFrom, 2, copyTo, 0, 7);
        System.out.println(new String(copyTo));

    }

}
```

Output: caffein

Sorting array

```
import java.util.Arrays;

public class SortArrayDemo {

    public static void main(String[] args) {
        int intArr[] = { 10, 20, 15, 22, 35 };

        Arrays.sort(intArr);
        System.out.println("The sorted Array is:");
        for(int element:intArr) {
            System.out.print(element+" ");
        }

    }

}
```

Output

```
The sorted Array is:
10 15 20 22 35
```

Searching array

```
import java.util.Arrays;

public class SearchArrayDemo {

    public static void main(String[] args) {
        int intArr[] = { 10, 20, 15, 22, 35 };

        //Binary Search requires Arrays to be sorted first
        Arrays.sort(intArr);

      //Element to search
        int intKey = 22;

        System.out.println(intKey + " found at index = "
                           + Arrays.binarySearch(intArr, intKey));
    }

}
```

Output

22 found at index = 3

Strings

- In Java, string is basically an object of String class, that represents sequence of char values. An array of characters works same as Java string
- Strings are treated as objects of class String which represents character strings.
- All string literals in Java programs, such as "abc", are implemented as instances of this class.

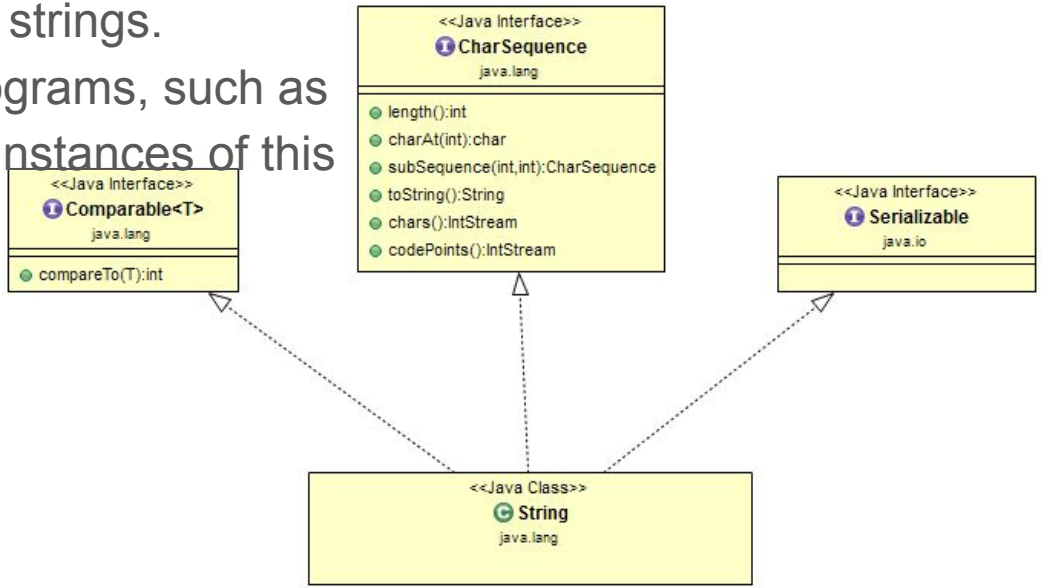
java.lang

Class String

java.lang.Object
    java.lang.String

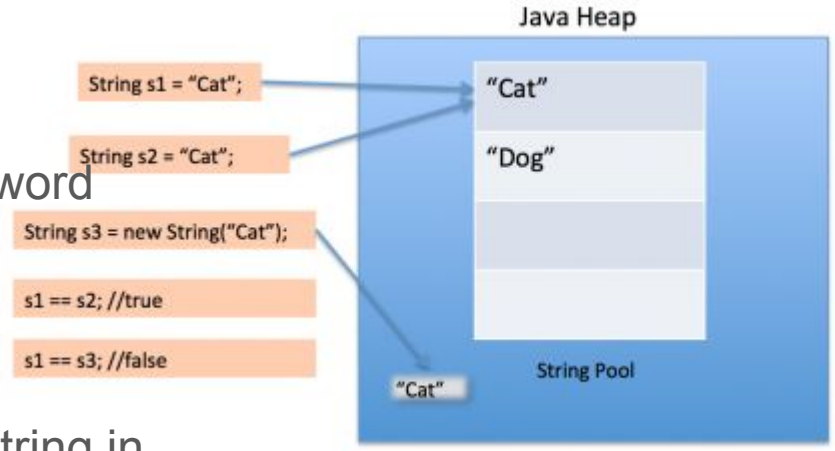
All Implemented Interfaces:

Serializable, CharSequence, Comparable<String>



char[] ch={'h','e','l','l','o'};
String s=new String(ch);
This is same as that of
String s="hello";

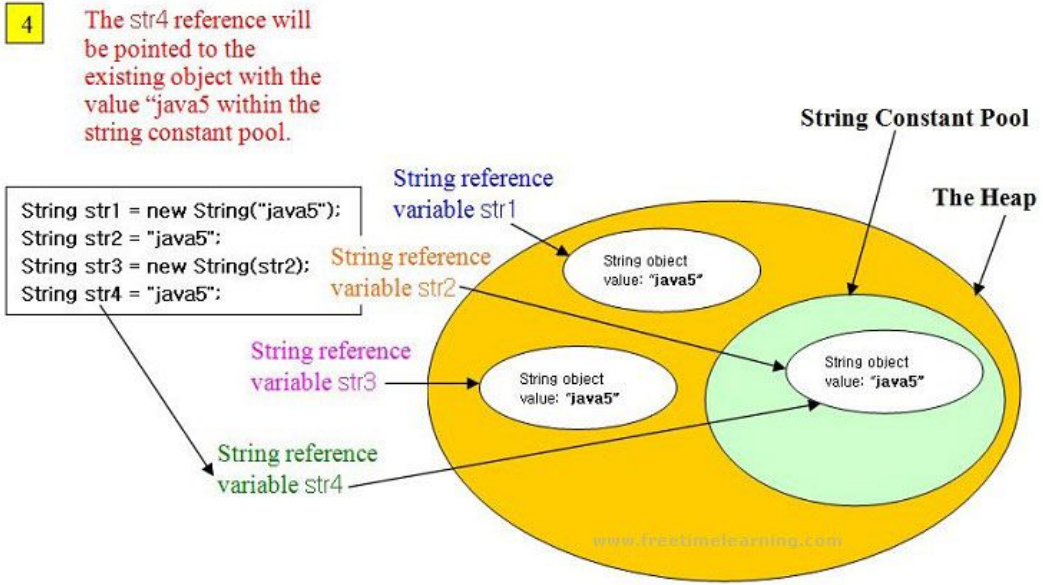
String literals vs objects



- String objects are created using **new** keyword
- String s1 = new String("Java");
- String s2 = new String("Java");
- System.out.println(s1 == s2); //**false**
- String literals are created by placing the string in double quotes.
- String s3 = "Java";
- String s4 = "Java";
- System.out.println(s3 == s4);//**true**
- Here **==** (equality) is not checking the content of the stings.
- String literals are stored in a special area called **String Constant Pool** in program heap.

Immutability of string

- Strings are constant; their values cannot be changed(**immutable**) after they are created. **StringBuffer** support mutable strings.
- String objects are immutable, so they can be shared.
- In Java, String is a final and immutable class.
- It cannot be inherited, and once created, we can not alter the object.

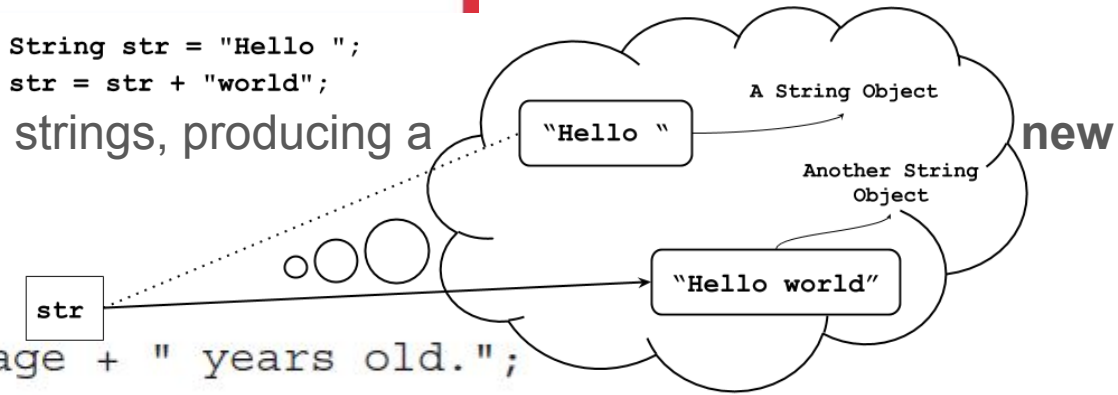


String length and String concatenation

- The length of a string is the number of characters that it contains.
- To obtain this value, call the length( ) method.

```
String str = "This is a string";
int length = str.length();
System.out.println (length);
```

+ operator concatenates two strings, producing a **new** String object as the




```
int age = 9;
String s = "He is " + age + " years old.";
System.out.println(s);
```

String comparison

- equals( ) and equalsIgnoreCase()
- These functions produces boolean output.
- It has this general form: boolean equals(Object str)

```
String s1 = "Hello";
String s2 = "Hello";
s1.equals(s2); // true

String s1 = "Hello";
String s2 = "HELLO" ;
s1.equals(s2); // false
s1.equalsIgnoreCase(s2); // true
```

String comparison

```
String s1="java string split method”;
System.out.println(s1.startsWith("ja")); // true
System.out.println(s1.startsWith("java string")); // true
```

startsWith( ) and endsWith ()

- The startsWith( ) method determines whether a given String begins with a specified string.
- endsWith( ) determines whether the String in question ends with a specified string.
- They have the following general forms
  - boolean startsWith (String str)
  - boolean endsWith (String str)

```
String s1="java string split method”;
System.out.println(s1. endsWith(“d")); // true
System.out.println(s1. endsWith(" method ")); // true
```

String comparison: compareTo() method

- To find less than, equal to, or greater than relations between strings
- A string is less than another if it comes before the other in dictionary order
- A string is greater than another if it comes after the other in dictionary order

int compareTo(String *str*)

Here, *str* is the **String** being compared with the invoking **String**. The result of the comparison is returned and is interpreted as shown here:

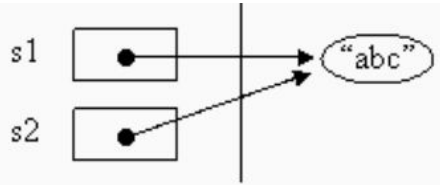
Value	Meaning
Less than zero	The invoking string is less than *str*.
Greater than zero	The invoking string is greater than *str*.
Zero	The two strings are equal.

```
String s1="hello";
String s2="hello";
String s3="meklo";
String s4="hemlo";
String s5="flag";
System.out.println(s1.compareTo(s2));//0 because both are equal
System.out.println(s1.compareTo(s3));//-5 because "h" is 5 times lower than "m"
System.out.println(s1.compareTo(s4));//-1 because "l" is 1 times lower than "m"
System.out.println(s1.compareTo(s5));//2 because "h" is 2 times greater than "f"
```

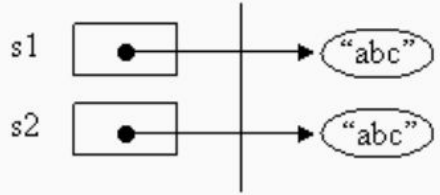
String comparison: equals() vs ==

- equals( ) method compares the characters inside a String object.
- == operator compares two object references to see whether they refer to the same instance

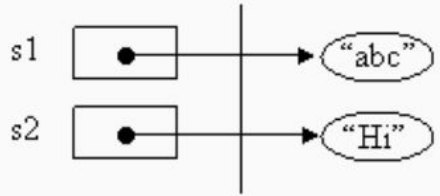
```
String s1 = new String("HELLO");
String s2 = new String("HELLO");
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2));// true
```



s1 == s2
(also implies s1.equals(s2) is true)



s1 != s2, but
s1.equals(s2) is true



s1 != s2, and
s1.equals(s2) is false

equals()
vs
==
in Java
String

String searching

- The String class provides two methods that allow you to search a string for a specified character or substring
- **indexOf( )** Searches for the first occurrence of a character or

```
String s1="this is index of example";
//passing substring
int index1=s1.indexOf("is");//returns the index of is substring
int index2=s1.indexOf("index");//returns the index of index substring
System.out.println(index1+"  "+index2);
```

Output: 2 8

```
String s1="this is index of example";//there are 2 's' characters in this Sentence
int index1=s1.lastIndexOf('s');//returns last index of 's' char value
System.out.println(index1);//6
```

substring() method

- Used to get the substring of a given string based on the passed indexes
  - String substring(int beginIndex)
  - Returns the substring starting from the specified index i.e beginIndex and extends to the character present at the end of the string
- **String substring(int beginIndex , int endIndex)**
  - The substring begins at the specified beginIndex and extends to the character at index endIndex - 1.
  - Thus the length of the substring is endIndex - beginIndex .

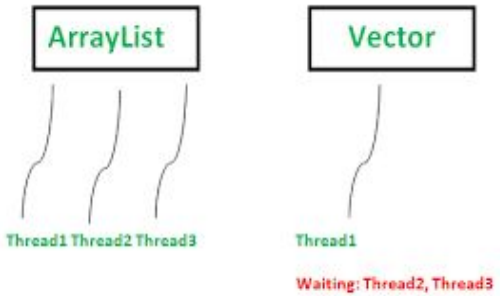
```
public class SubStringExample{
   public static void main(String args[]) {
       String str= new String("quick brown fox jumps over the lazy dog");
       System.out.println("Substring starting from index 15:");
       System.out.println(str.substring(15));
       System.out.println("Substring starting from index 15 and ending at 20:");
       System.out.println(str.substring(15, 20));
   }
}
```

String methods

char	charAt(int index) Returns the char value at the specified index	char first = str.charAt (0)
boolean	equals(Object object) Compares string to another string	bool isEqual = str.equals (other)
boolean	equalsIgnoreCase(String other) Compares to another string, ignoring case	bool isEqual = str. equalsIgnoreCase(other)
int	indexOf(String str) Returns the index of str in the given string	int index = str.indexOf ("Hello")
int	length()	int length = str.length()
String	substring(int beginIndex)	int rest = str.substring (5)
String	substring(int beginIndex, int endIndex)	int firstTwo = str. substring(0, 2)

Vector Class in Java

- Vector implements a **dynamic array**, ie growable array of objects.
- Like an array, it contains elements that can be accessed using an integer index .
- It is found in the *java.util* package and implements the List interface , so we can use all the methods of List interface.
- It is synchronized and due to which it gives poor performance in searching, adding, delete and update of its elements.
- Contains many legacy methods that are not part of the collections framework.
- Vector proves to be very useful if you don't know the size of the array in advance or you just need one that can change sizes over the lifetime of a program.



Creating a Vector

java.util

Class Vector<E>

java.lang.Object
    java.util.AbstractCollection<E>
        java.util.AbstractList<E>
            java.util.Vector<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess

Direct Known Subclasses:

Stack

Here is how we can create vectors in Java

- Import Vector class. (import java.util.Vector ;)

Vector<Type> vector = new Vector<>();

For example

// create Integer type linked list

Vector<Integer> vector= new Vector<>();

// create String type linked list

Vector<String> vector= new Vector<>();

Add elements to vector

- `add(element)` - adds an element to vectors
- `add(index, element)` - adds an element to the specified position
- `addAll(vector)` - adds all elements of a vector to another vector

```java
import java.util.Vector;

public class VectorAddElementsDemo {

    public static void main(String[] args) {
        Vector<String> mammals= new Vector<>();

        // Using the add() method
        mammals.add("Dog");
        mammals.add("Horse");

        // Using index number
        mammals.add(2, "Cat");
        System.out.println("Vector: " + mammals);

        // Using addAll()
        Vector<String> animals = new Vector<>();
        animals.add("Crocodile");

        animals.addAll(mammals);
        System.out.println("New Vector: " + animals);

    }
}
```

Output

```
Vector: [Dog, Horse, Cat]
New Vector: [Crocodile, Dog, Horse, Cat]
```

Access vector elements

- `get(index)` – returns an element specified by the index
- `iterator()` – returns an iterator object to sequentially access vector elements

```
import java.util.Iterator;
import java.util.Vector;

public class VectorAccessElementDemo {

    public static void main(String[] args) {
        Vector<String> animals= new Vector<>();
        animals.add("Dog");
        animals.add("Horse");
        animals.add("Cat");

        // Using get()
        String element = animals.get(2);
        System.out.println("Element at index 2: " + element);

        // Using iterator()
        Iterator<String> iterate = animals.iterator();
        System.out.print("Vector: ");
        while(iterate.hasNext()) {
            System.out.print(iterate.next());
            System.out.print(", ");
        }//end of while

    }//end of main

}//end of class
```

Output

```
Element at index 2: Cat
Vector: Dog, Horse, Cat,
```

Remove vector elements

- `remove(index)` – removes an element from specified position
- `removeAll()` – removes all the elements
- `clear()` – removes all elements. It is more efficient than `removeAll()`

```
import java.util.Vector;

public class VectorRemoveElementDemo {

    public static void main(String[] args) {
        Vector<String> animals= new Vector<>();
        animals.add("Dog");
        animals.add("Horse");
        animals.add("Cat");

        System.out.println("Initial Vector: " + animals);

        // Using remove()
        String element = animals.remove(1);
        System.out.println("Removed Element: " + element);
        System.out.println("New Vector: " + animals);

        // Using clear()
        animals.clear();
        System.out.println("Vector after clear(): " + animals);

    }

}
```

Output

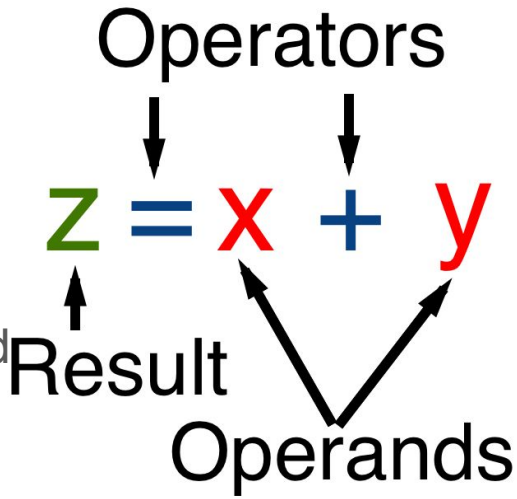
```
Initial Vector: [Dog, Horse, Cat]
Removed Element: Horse
New Vector: [Dog, Cat]
Vector after clear(): []
```

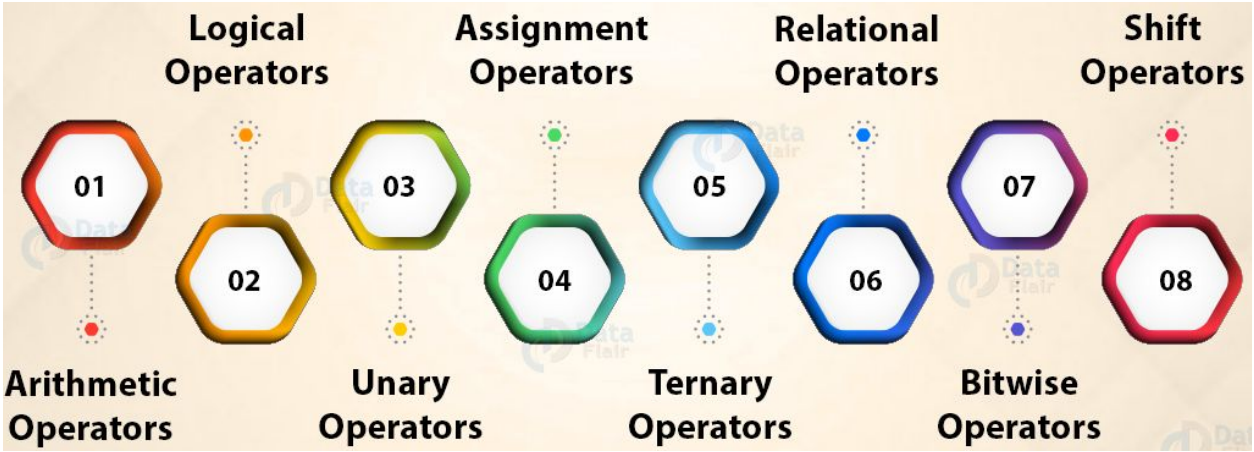
Important Vector methods

Method	*Description*
v.add(o)	adds Object o to Vector v
v.add(i, o)	Inserts Object o at index i, shifting elements up as necessary.
v.clear()	removes all elements from Vector v
v.contains(o)	Returns true if Vector v contains Object o
v.firstElement(i)	Returns the first element.
v.get(i)	Returns the object at int index i.
v.lastElement(i)	Returns the last element.
v.size()	Returns the number of elements in Vector v.

Operators in Java

- Operators are special symbols that perform specific operations on one, two, or three operands, and then return a result.
- An operator is a token that tells the computer to perform certain mathematical or logical operations on variables and literals.





Arithmetic operators



Operator	Meaning	Example	Result
+	Addition	10 + 2	12
-	Subtraction	10 – 2	8
*	Multiplication	10 * 2	20
/	Division	10 / 2	5
%	Modulus (remainder)	10 % 2	0
++	Increment	a++ (consider a = 10)	11
--	Decrement	a-- (consider a = 10)	9
+=	Addition Assignment	a += 10 (consider a = 10)	20
-=	Subtraction assignment	a -= 10 (consider a = 10)	0
*=	Multiplication assignment	a *= 10 (consider a = 10)	100
/=	Division assignment	a /= 10 (consider a = 10)	1
%=	Modulus assignment	a %= 10 (consider a = 10)	0

Arithmetic operators - Compound operators

- a+=10 is same as that of a = a + 10
- += is an example for Compound Assignment Operators, also known as Shorthand Assignment Operators.
- It's called shorthand because it provides a short way to assign an expression to a variable.
- This operator can be used to connect Arithmetic operator with an Assignment operator

Modulus operator (%)

A = 5, B =2

C = A%B

C = ?

```
import java.util.*;

public class Demo{
      public static void main(String[] args){

            double x = 4.3;
            double y = 1.1;

            System.out.println(x%y);


      }
}
```

```
0.9999999999999996
```

++ / --

Initial x	Expression	Final y	Final x
3	y=x++	3	4
3	y=++x	4	4
3	y=x--	3	2
3	y=--x	2	2

X++ is Use –Then - Change

++x is Change – Then - Use

- A++ is same as A = A + 1
- A-- is same as A = A - 1
- ++A is pre increment, change first then use the value when there is an assignment.
- A++ post increment
- Use the value then change (when there is an assignment)

Assignment operator

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

- Assignment operators are used to assign values to variables.
- Eg: int A **=** C + 10 ;
- When the target variable and one of the operand are same then we can use shorthand / compound operators. (Not applicable for ternary operators)

Unary Operators

- Java unary operators are the types that need only one operand to perform any operation.
- It consists of various arithmetic, logical and other operators that operate on a single operand

+	Unary plus; indicates positive value
-	Unary minus; negates an expression
++	Increment operator; increments a value by 1
--	Decrement operator; decrements a value by 1
!	Logical complement operator; inverts the value of a boolean

```
int x = 5;
int a, b;
a = x++;        // a = 5 x = 6
x--;            // x = 5
b = ++x;        // b = 6 x = 6
++x;            // x = 7
boolean bool = true;
// true
System.out.println(bool);
// false
System.out.println(!bool);
```

Java Comparison/Relational Operators

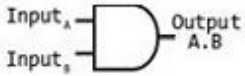
- Comparison operators are used to compare two values
- Produces Boolean result

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Java Boolean Logical Operators

- Logical operators are used to determine the logic between variables or values.
- Produces boolean result.
- In Java (&&) AND and (||) OR supports **short circuiting**

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)



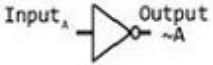
Truth table: AND

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1



Truth table: OR

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1



Truth table: NOT

A	~A
0	1
1	0

Java Bitwise Operators

- Bitwise operators are used to perform binary logic with the bits of an **integer** or **long** integer.

Operator	Description	Example	Same as	Result	Decimal
&	AND - Sets each bit to 1 if both bits are 1	5 & 1	0101 & 0001	0001	1
|	OR - Sets each bit to 1 if any of the two bits is 1	5 | 1	0101 | 0001	0101	5
~	NOT - Inverts all the bits	~ 5	~0101	1010	10
^	XOR - Sets each bit to 1 if only one of the two bits is 1	5 ^ 1	0101 ^ 0001	0100	4
<<	Zero-fill left shift - Shift left by pushing zeroes in from the right and letting the leftmost bits fall off	9 << 1	1001 << 1	0010	2
>>	Signed right shift - Shift right by pushing copies of the leftmost bit in from the left and letting the rightmost bits fall off	9 >> 1	1001 >> 1	1100	12
>>>	Zero-fill right shift - Shift right by pushing zeroes in from the left and letting the rightmost bits fall off	9 >>> 1	1001 >>> 1	0100	4

Note: The Bitwise examples above use 4-bit unsigned examples, but Java uses 32-bit signed integers and 64-bit signed long integers. Because of this, in Java, ~5 will not return 10. It will return -6. ~00000000000000000000000000000101 will return 11111111111111111111111111111010

In Java, 9 >> 1 will not return 12. It will return 4. 00000000000000000000000000001001 >> 1 will return 00000000000000000000000000000100

Java Shift Operators (Unsigned right shift)

- >>> Right shift with zero fill or unsigned right shift is a newly introduced operator in Java

```
int a =  3; // ...00000011 =  3
int b = -4; // ...11111100 = -4
```

>>> Right 0

```
a         00000000000000000000000000000011      3
a >>> 2   00000000000000000000000000000000      0

b         11111111111111111111111111111100     -4
b >>> 2   00111111111111111111111111111111     +big
```

<< Left

```
a         00000000000000000000000000000011      3
a << 2    00000000000000000000000000001100     12

b         11111111111111111111111111111100     -4
b << 2    11111111111111111111111111110000    -16
```

>> Right

```
a         00000000000000000000000000000011      3
a >> 2    00000000000000000000000000000000      0

b         11111111111111111111111111111100     -4
b >> 2    11111111111111111111111111111111     -1
```

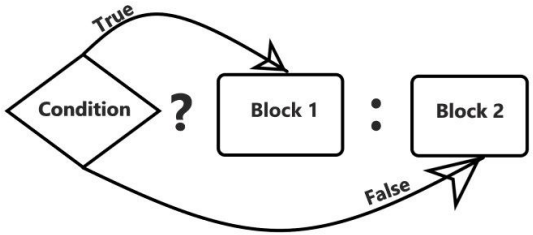
Java Ternary Operato

```java
int age = 10;

String result = (age > 18) ? "Yes, you can vote!" : "No, you can't vote!";

System.out.println(result);
```

- As the name ternary suggests, it is the only operator in Java consisting of three operands.
- The ternary operator can be thought of as a simplified version of the if-else statement with a value to be returned.
- Syntax
  variable = (condition) ? block1 : block2 ;




```java
public class LargestOfThree {

    public static void main(String[] args) {

        int a = 55;
        int b = 95;
        int c = 75;

        int result = ( a > b ) ? (a > c ? a : c) : ( b > c ? b : c);

        System.out.println(result);

    }

}
```

Type Comparison Operator

- In Java, the **instanceof** keyword is a binary operator. Result will be boolean
- It is used to check whether an object is an instance of a particular class or not.
- The operator also checks whether an object is an instance of a class that implements an interface. Syntax: **object instanceof ClassName/InterfaceName;**

```
class Animal {
}

// Dog class is a subclass of Animal
class Dog extends Animal {
}

class Main {
    public static void main(String[] args){
        Dog d1 = new Dog();

        // checks if d1 is an object of Dog
        System.out.println("Is d1 an instance of Dog: "+ (d1 instanceof Dog));

        // checks if d1 is an object of Animal
        System.out.println("Is d1 an instance of Animal: "+ (d1 instanceof Animal));
    }
}
```

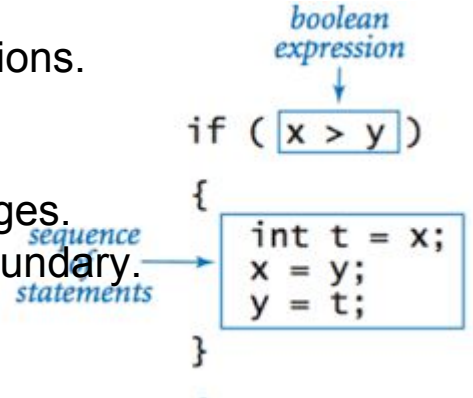
Output:

```
Is d1 is an instance of Dog: true
Is d1 an instance of Animal: true
```

Expression, Statement, Block

Expressions

- An expression is a construct made up of variables, operators, and method invocations.
- Constructed according to the syntax of the language, that evaluates to a single value.
  - int result = 1 + 2 // result is now 3
- Compound expressions are created from various smaller expressions.
  - Eg: a = x + y / 100 // ambiguous



Statements

- Statements are roughly equivalent to sentences in natural languages.
- A statement forms a complete unit of execution. **;** indicates the boundary.
- Expression statement
  - Bicycle myBike = new Bicycle();
- Declaration statements - A declaration statement declares a variable.
  - double aValue = 8933.234;
- Control flow statements regulate the order in which statements get executed.

Block

A block is a group of zero or more statements between balanced braces and can be used anywhere a single statement is allowed.

Expression evaluation in Java

- When your Java program executes and encounters a line with an expression, the expression is *evaluated* (its value is computed).
  - The expression `3 * 4` is evaluated to obtain `12`.
  - `System.out.println(3 * 4)` prints `12`, not `3 * 4`.

Evaluate following expressions.

- A = 10 + 5 ;
- C = 10 * 2 - 3 ;

Evaluate 2*x-3*y ?

- (2x)-(3y) or 2(x-3y) which one is correct?

Evaluate A / B * C

- A / (B * C) or (A / B) * C Which one is correct?

To answer these questions satisfactorily one has to understand the priority or the order in which these operators are evaluated.

Operator Precedence

- **Operator precedence** determines the order in which the operators in an expression are evaluated.
- **Precedence order**. When two operators share an operand the operator with the higher precedence goes first.
  - For example, 1 + 2 * 3 is treated as 1 + (2 * 3), whereas 1 * 2 + 3 is treated as (1 * 2) + 3 since multiplication has a higher precedence than addition.
- **Associativity**. When an expression has two operators with the same precedence, the expression is evaluated according to its associativity.
  - For example x = y = z = 17 is treated as x = (y = (z = 17)), leaving all three variables with the value 17, since the **=** operator has **right-to-left** associativity (and an assignment statement evaluates to the value on the right hand side).
  - On the other hand, 72 / 2 / 3 is treated as (72 / 2) / 3 since the **/** operator has **left-to-right** associativity.
  - Some operators are not associative: for example, the expressions (x <= y <= z) and x++-- are **invalid**.

Operator Precedence

Order of evaluation of subexpressions.

- Associativity and precedence determine in which order Java applies operators to expressions but they do not determine in which order the **subexpressions** are evaluated.
- In Java, subexpressions are evaluated from **left to right** (when there is a choice).
- So, for example in the expression **A() + B() * C(D(), E())**, the subexpressions are evaluated in the order A(), B(), D(), E(), and C().
- It is considered poor style to write code that relies upon this behavior (and different programming languages may use different rules).

Short circuiting.

- When using the conditional and and or operators (&& and ||), Java does not evaluate the second operand unless it is necessary to resolve the result.
- This allows statements like if (s != null && s.length() < 10) to work reliably.
- Programmers rarely use the non short-circuiting versions (& and |) with boolean expressions.

Precedence table

Highest level to lowest

Level	Operator	Description	Associativity

16	`[]`
	`.`
	`()`
		access array element
		access object member
		parentheses
			left to right

15	`++` `--`	unary post-increment unary post-decrement	not associative

14	`++`
	`--`
	`+`
	`-`
	`!`
	`~`
		unary pre-increment
		unary pre-decrement
		unary plus
		unary minus
		unary logical NOT
		unary bitwise NOT
			right to left

13	`()` `new`	cast object creation	right to left
12	`* / %`	multiplicative	left to right
11	`+ -` `+`	additive string concatenation	left to right
10	`<< >>` `>>>`	shift	left to right

9	`< <=`
	`> >=`
	`instanceof`
		relational	not associative

8	`==` `!=`	equality	left to right
7	`&`	bitwise AND	left to right
6	`^`	bitwise XOR	left to right
5	`|`	bitwise OR	left to right
4	`&&`	logical AND	left to right
3	`||`	logical OR	left to right
2	`?:`	ternary	right to left

1	`=` `+=` `-=`
	`*=` `/=` `%=`
	`&=` `^=` `|=`
	`<<=` `>>=` `>>>=`
		assignment	right to left

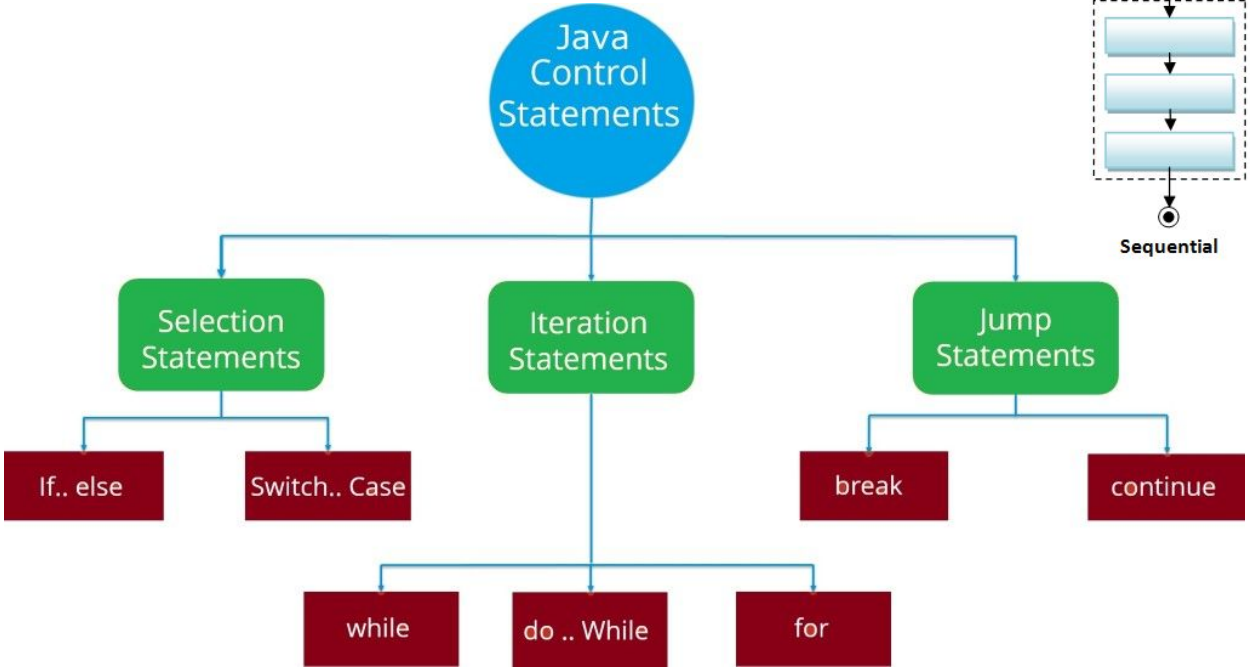
Operator Precedence

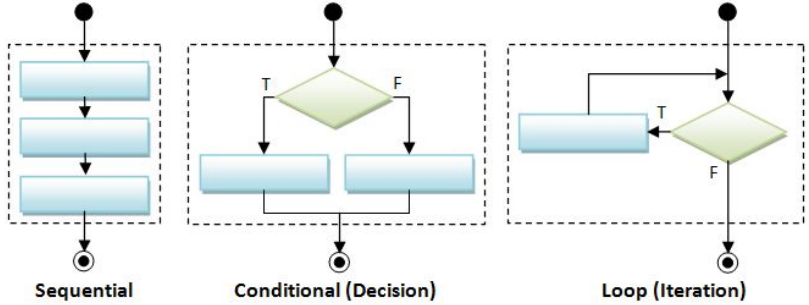
```java
public class OperatorPrecedence {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        int z = ++x * y--;
        int a = 7 * 3 + 24 / 3 - 5;
        int b = (7 * 3) + 24 / (3 - 5);
        System.out.println("1 + 2 = " + 1 + 2);
        System.out.println("1 + 2 = " + (1 + 2));
        System.out.print("a :" +a + " b:" + b + " z:" +z);
    }
}
```

```
D:\GECI\CST205-OOPJ\Java\basics>java Operato
1 + 2 = 12
1 + 2 = 3
a :24 b:9 z:60
D:\GECI\CST205-OOPJ\Java\basics>
```

Control Statements

- Default execution order of statements in a program is sequential, ie one after the other from top to bottom.
- Using control statement, this execution sequence of the program can be altered.
- A control statement in java is a statement that determines whether the other statements will be executed or not.





Selection Statements

- In java, the selection statements are also known as decision making statements or branching statements or conditional control statements.
- The selection statements are used to select a part of the program to be executed based on a condition.
- Java provides the following selection statements.

1. if statement
2. if-else statement
3. nested if statement
4. if-else if statement (if else if ladder)
5. switch statement

Selection if statement

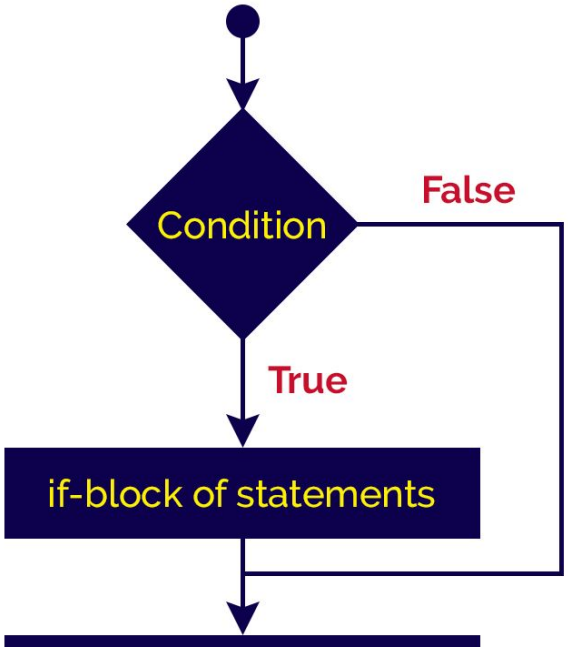
- In java, we use the if statement to test a condition and decide the execution of a block of statements based on that condition result.

Syntax

```
if(condition){
        if-block of statements;
        ...
}
statement after if-block;
...
```

```
if((num % 5) == 0) {
    System.out.println("We are inside the if-block!");
    System.out.println("Given number is divisible by 5!!");
}
```

Flow of execution



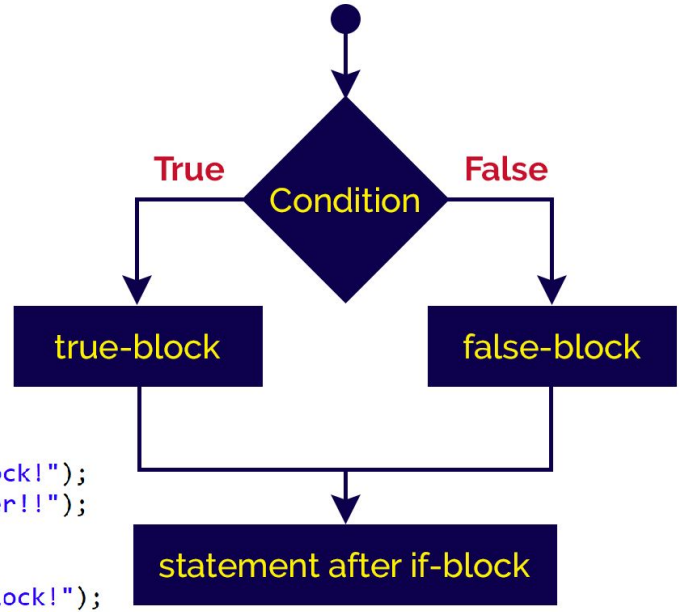
Selection: if-else statement

- if-else statement to test a condition and pick the execution of a block of statements out of two blocks based on that condition result

Syntax

```
if(condition){
        true-block of statements;
        ...
}
else{
        false-block of statements;
        ...
}
statement after if-block;
```

Flow of execution

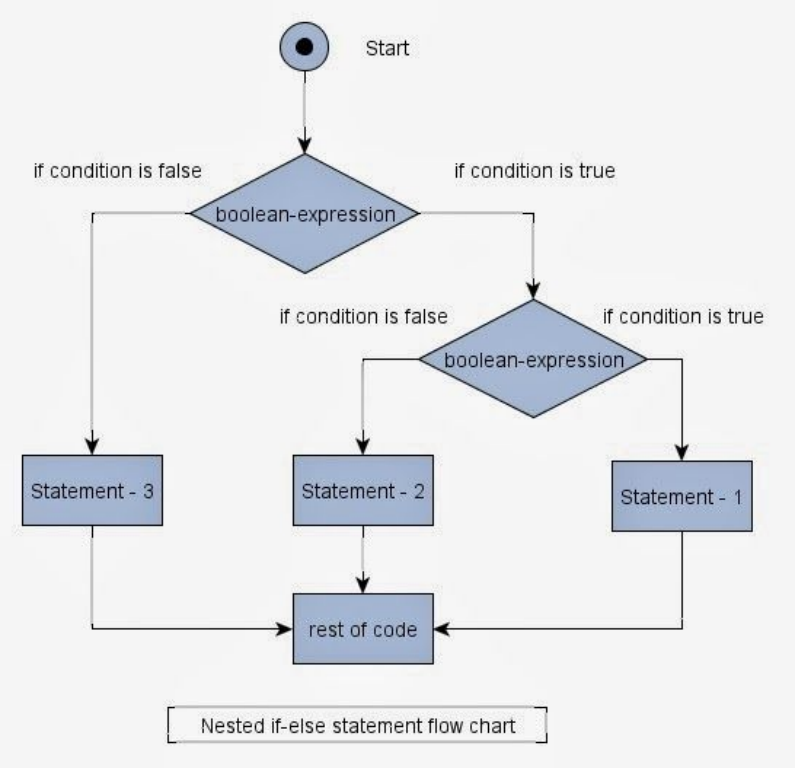



```
if((num % 2) == 0) {
    System.out.println("We are inside the true-block!");
    System.out.println("Given number is EVEN number!!");
}
else {
    System.out.println("We are inside the false-block!");
    System.out.println("Given number is ODD number!!");
}
```

Selection: nested if statement

- Writing an if statement inside another if-statement is called nested if statement.

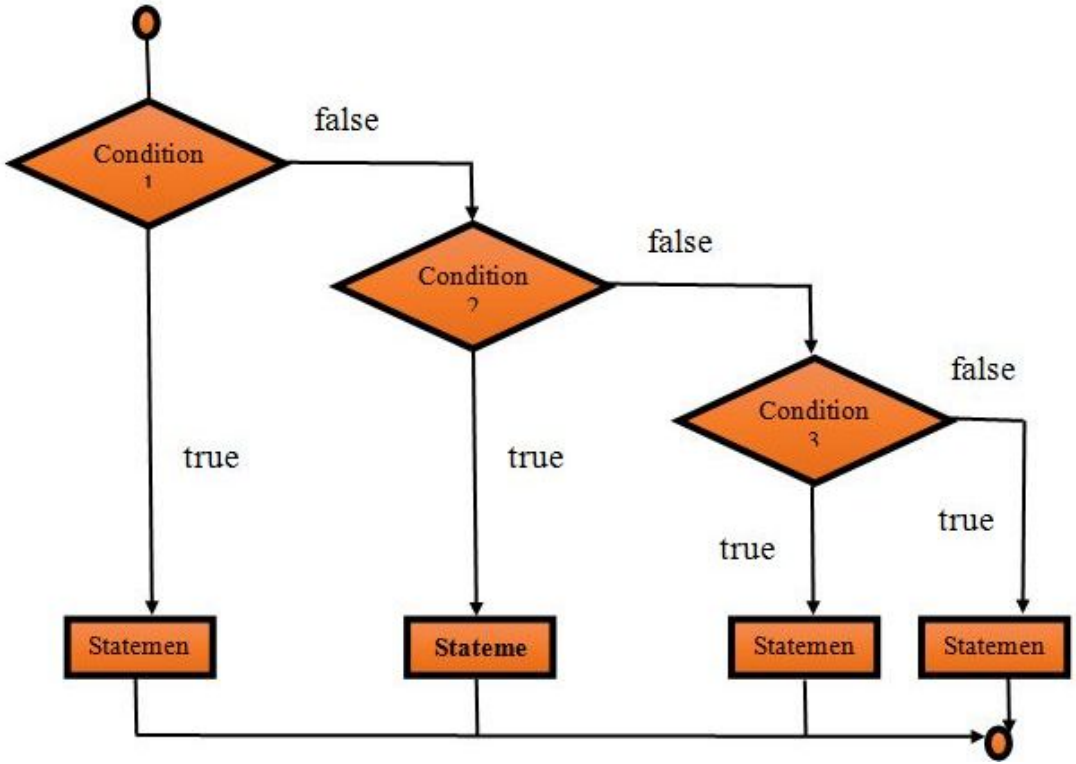
```
Java keyword   Test expression
if (test_condition 1)   <-- Outer if
    {
      //statement(s)
        if (test_condition 2)   <-- Inner if
            {
              //statement(s)
            }
        else{   <-- Inner else
            //statement(s)
            }
    }
else{   <-- Outer else
    //statement(s)
}
```



Selection: if else if (if else if ladder)

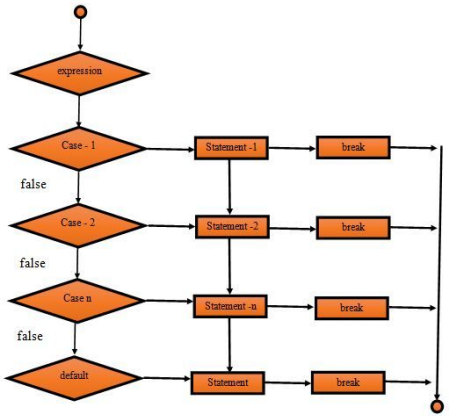
- The if-else-if ladder statement executes one condition from multiple statement block.

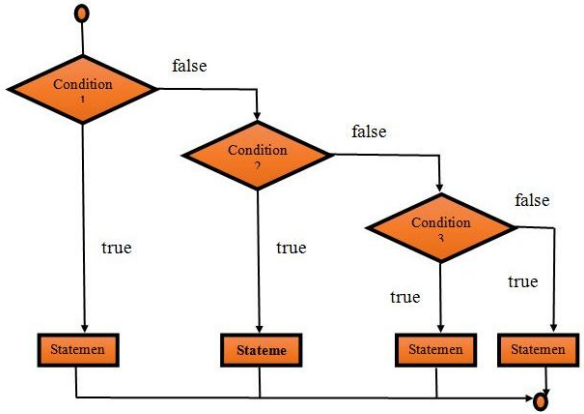
```
if(condition1)
{
//code for if condition1 is true
}
else if(condition2)
{
//code for if condition2 is true
}
else if(condition3)
{
//code for if condition3 is true
}
...
else
{
//code for all the false conditions
}
```



Selection: switch statement

- Switch statement is used for executing one statement from multiple conditions.
- It is similar to an if-else-if ladder.
- In a switch statement,the expression can be of **byte, short, char** and **int** data types.
- From JDK7 **enum**, **String** class and the **Wrapper** classes can also be used.
- Following are some of the rules while using the switch statement:
  - There can be one or N numbers of cases.
  - The values in the case must be unique.
  - Each statement of the case can have a break statement. It is optional.

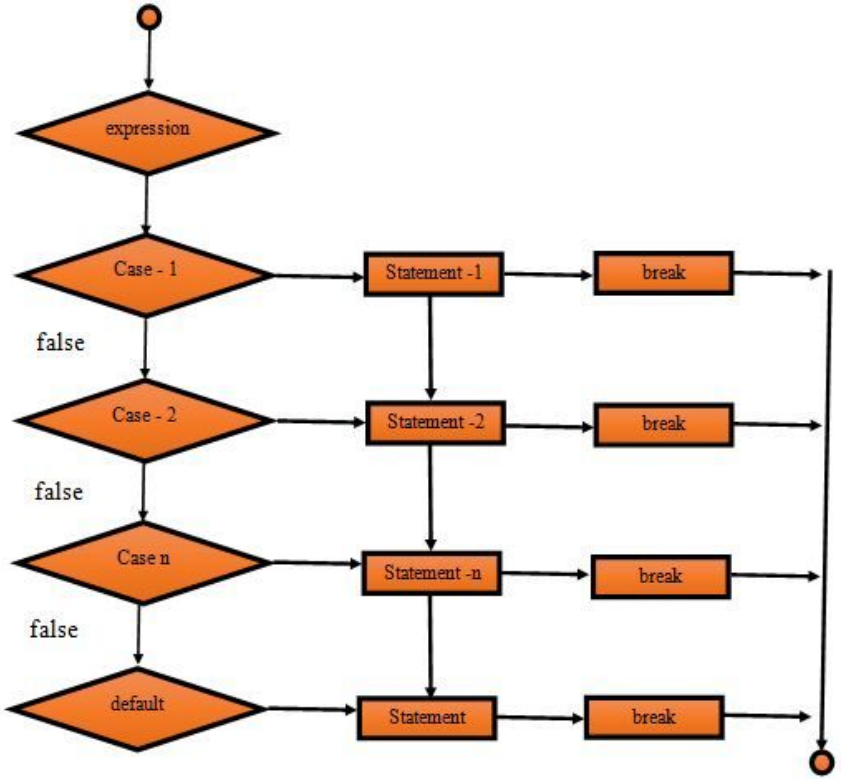




Selection: switch statement

```
switch(expression)
{
case value1:
            //code for execution;
            break;  //optional
case value2:
 // code for execution
 break;  //optional
......
......
......
......
Case value n:
// code for execution
 break;  //optional

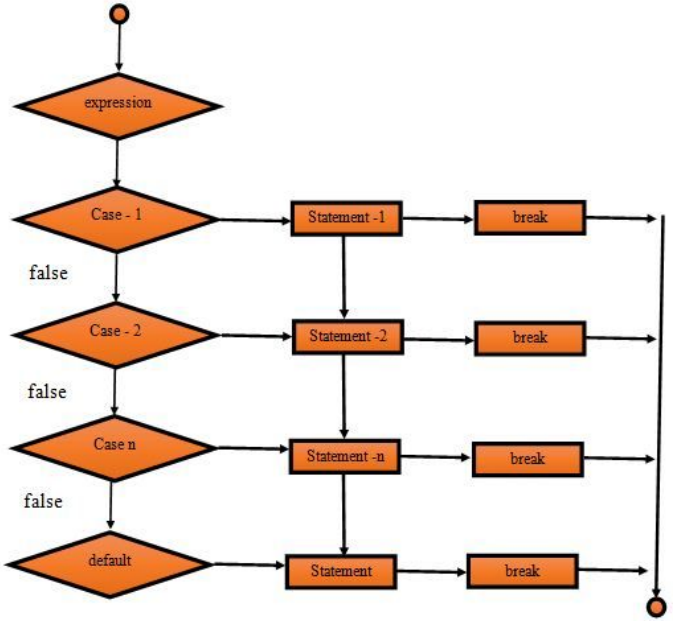
default:
 code for execution when none of the case is true;
}
```



Selection: switch statement

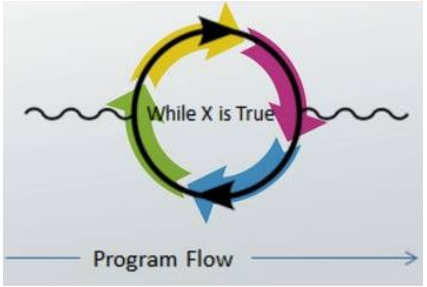
```
public class SwitchExample {

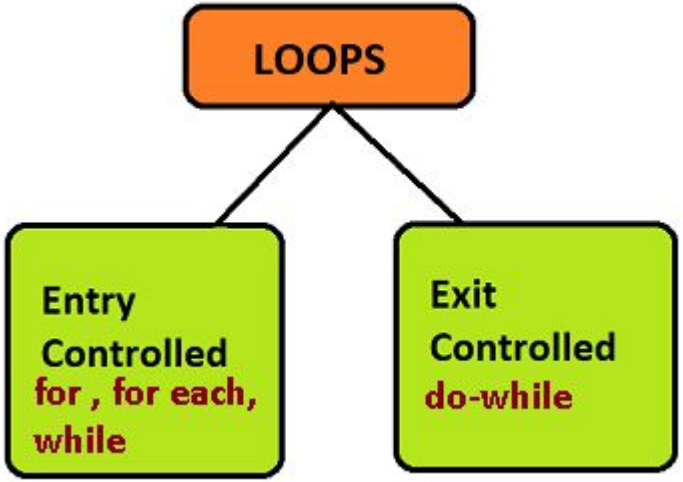
    public static void main(String[] args) {
        int n = 1;
        switch (n) {
            case 1:
                System.out.println("Khanh");
                break;
            case 2:
                System.out.println("Huong Dan Java");
                break;
            case 3:
                System.out.println("Lap Trinh Java");
                break;
            default:
                break;
        }
    }
        Khanh
}
```

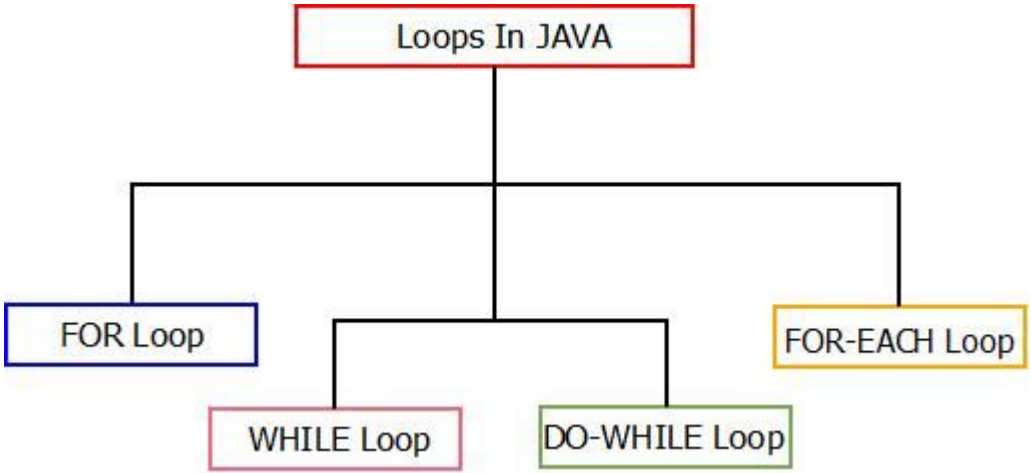


Iteration Statements (Loops)

- A loop statement allows us to execute a statement or group of statements multiple times





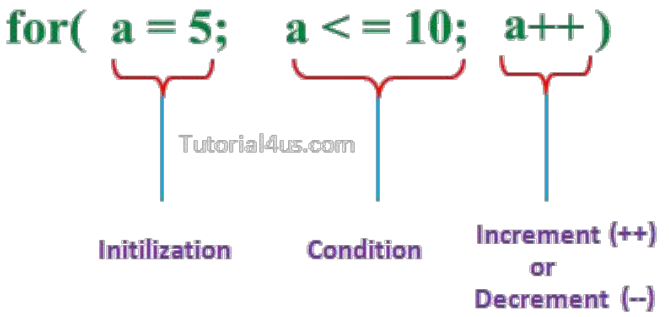


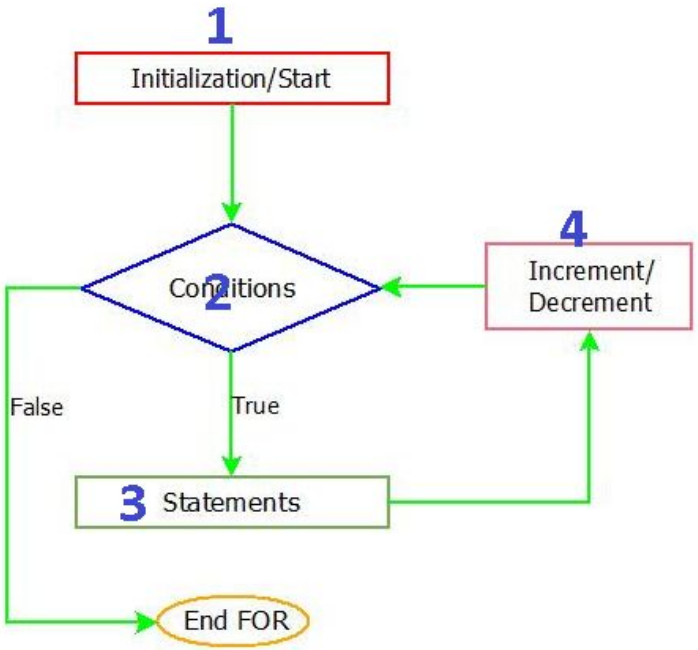
Iteration:for loop

```
public static void main(String args[]) {
                1      2      4
    for (int i = 0; i < 10; i++) {

     3  System.out.println("The value of i is: " + i);

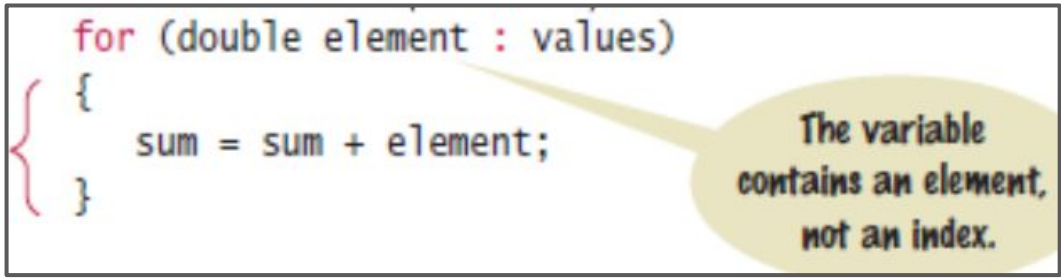
    }
}
```





Enhanced for loop (for each)

- As of Java 5, the enhanced for loop was introduced.
- This is mainly used to traverse collection of elements including arrays
- Syntax of enhanced for loop.
  - **for(declaration : expression)**
  - **{**
  - **// Statements**
  - **}**



- **Declaration** − The newly declared block variable, is of a type compatible with the elements of the array you are accessing.
- This variable will be available only within the for block and its value would be the same as the **current array element**.
- **Expression** − This evaluates to the array you need to loop through.
- The expression can be an array variable or method call that returns an array.

Enhanced for loop (for-each)

```
public class Test {

   public static void main(String args[]) {
      int [] numbers = {10, 20, 30, 40, 50};

      for(int x : numbers ) {
         System.out.print( x );
         System.out.print(",");
      }
      System.out.print("\n");
      String [] names = {"James", "Larry", "Tom", "Lacy"};

      for( String name : names ) {
         System.out.print( name );
         System.out.print(",");
      }
   }
}
```

Output

```
10, 20, 30, 40, 50,
James, Larry, Tom, Lacy,
```

Notes on for-each

- For-each loops are not appropriate when you want to modify the array.
- For-each loops do not keep track of index. So we can not obtain array index using For-Each loop.
- For-each only iterates forward over the array in single steps (ie, i+2 is not possible)
- For-each cannot process two decision making statements at once

```
// cannot be easily converted to a for-each loop
for (int i=0; i<numbers.length; i++)
{
   if (numbers[i] == arr[i]) { ...
   }
}
```

```
for (type var : array)
{
    statements using var;
}
```

is equivalent to:

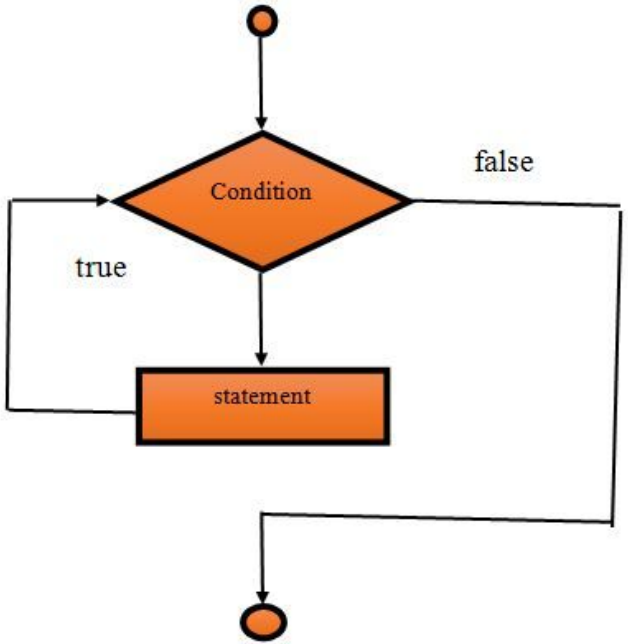
```
for (int i=0; i<arr.length; i++)
{
    type var = arr[i];
    statements using var;
}
```

hidden

while loop

- Entry controlled loop to repeat the loop body.
- When the number of iteration is not fixed then while loop is used

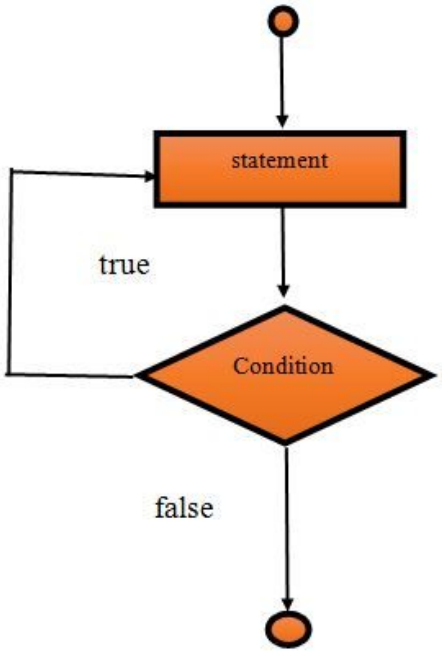
```
public class WhileDemo1
{
    public static void main(String[] args)
    {
        inti=1;
        while(i<=10)
        {
            System.out.println(i);
            i++;
        }
    }
}
```



do while loop

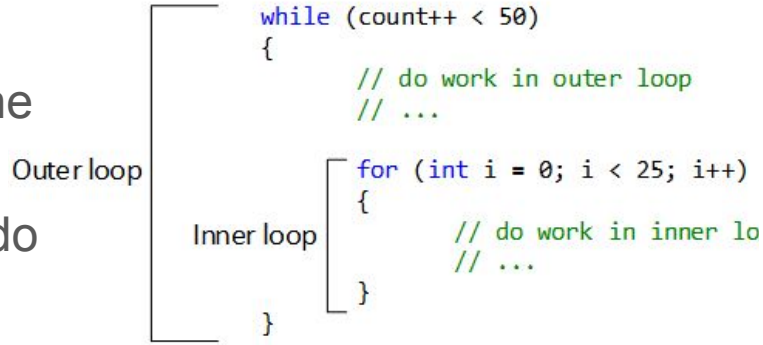
- Exit controlled loop to repeat the loop body.
- Irrespective of the loop condition, loop body will be executed at least once.

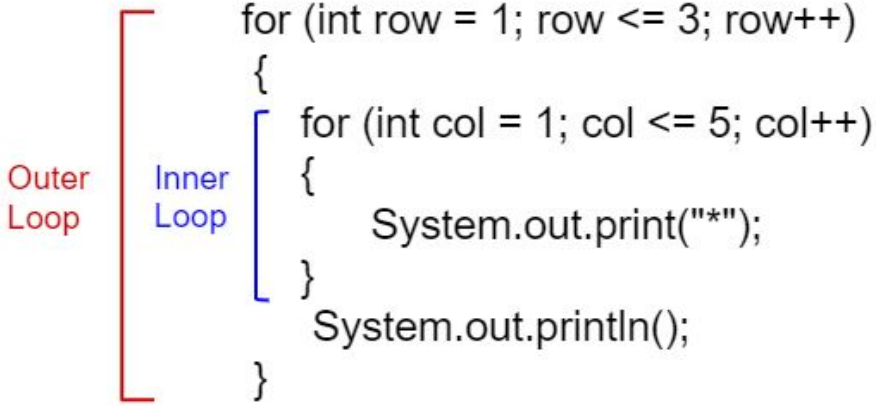
```
public class DoWhileDemo1
{
public static void main(String[] args)
{
inti=1;
do
{
System.out.println(i);
i++;
}while(i<=10);
        }
}
```



Nested loops

- A nested loop is a (inner) loop that appears in the loop body of another (outer) loop.
- The inner or outer loop can be any type: while, do while, or for.
- For example, the inner loop can be a while loop while an outer loop can be a for loop. Of course, they can be the same kind of loops too.





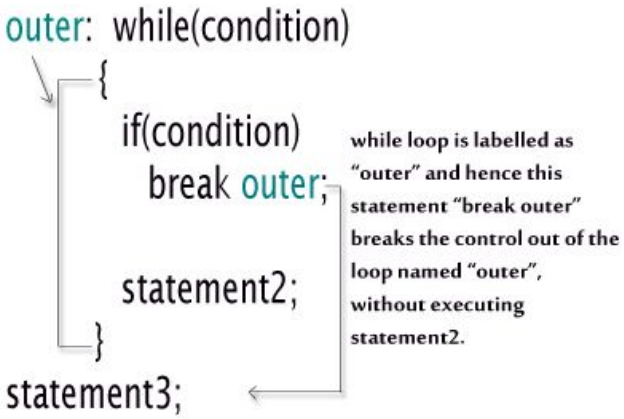
Labeled statements

- A label is a name attached to a statement. [**labele_name :** statement…]
- A **goto** statement in C/C++ programming provides an unconditional jump from the 'goto' to a **labeled** statement in the same function
- Java does not have goto statement, but java supports label.
- The only place where a label is useful in Java is **right before nested loop**.
- You can attach a label name just before the beginning of a loop.
- Label name can be used with break/continue statement for unconditional jump.

```
#include<stdio.h>
#include <conio.h>
int main()
{
printf ("Hello World\n");
goto Label;
printf("How are you?");
printf("Are you Okey?");
Label:
    printf("Hope you are fine");
getch();
}
```

Jumped

Skipped



Jump statements

- Loop control statements change execution from its normal sequence.
- When execution leaves a scope, all automatic objects that were created in that scope are destroyed.

break statement

- Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
- Has two forms: labeled and unlabeled.

continue statement

- Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
- Has two forms: labeled and unlabeled.

return statement

break;

```
while (testExpression) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

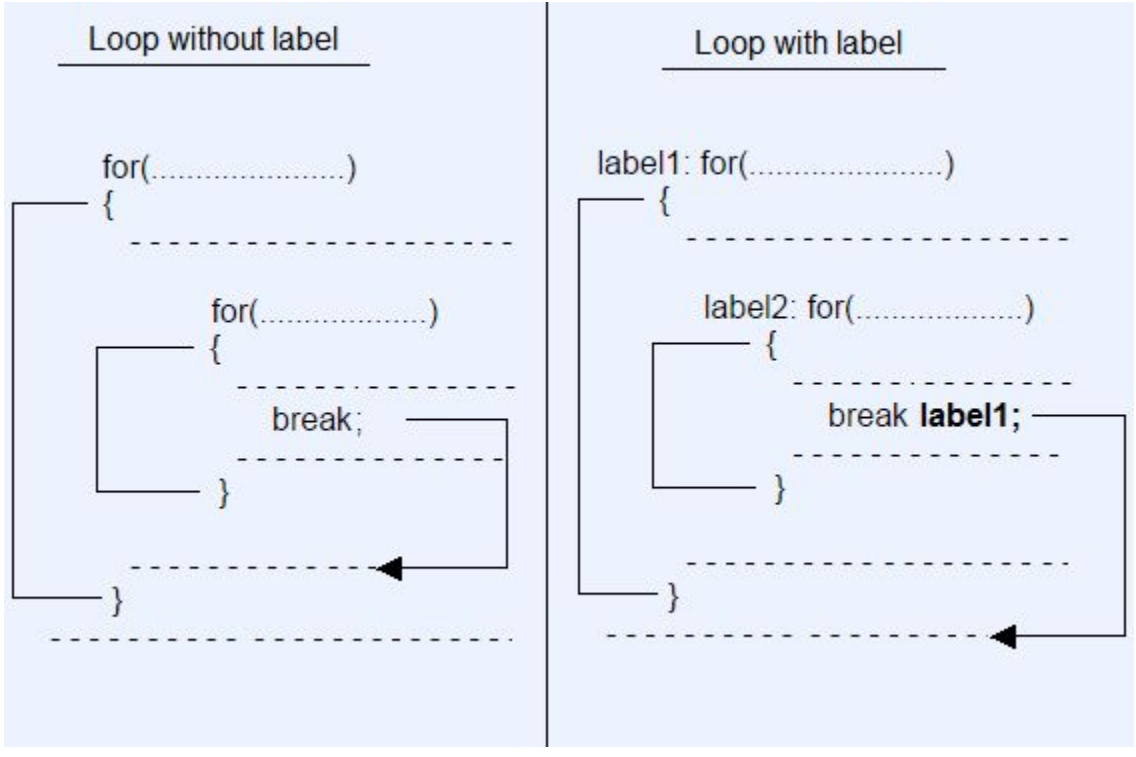
```
do {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
while (testExpression);
```

```
for (init; testExpression; update) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

```
while (testExpression) {
    // codes
    while (testExpression) {
        // codes
        if (condition to break) {
            break;
        }
        // codes
    }
    // codes
}
```

Labeled break;

- According to nested loop, if we put break statement in inner loop, compiler will jump out from inner loop and continue the outer loop again.
- To jump out from the outer loop, we can use labeled break.



continue;

```
while (testExpression) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

```
do {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
} while (testExpression);
```

```
for (init; testExpression; update) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

```
while(testExpresison) {
    // codes
    while (testExpression) {
        // codes
        if (condition for continue) {
            continue;
        }
        // codes
    }
    // codes
}
```

Labeled continue;

- In the case of nested loops, the continue skips the current iteration of the innermost loop.
- To skip the current iteration of the outer loop, labeled continue can be used.

```
label:
while (testExpression) {
    // codes
    while (testExpression) {
        // codes
        if (condition for continue) {
            continue label;
        }
        // codes
    }
    // codes
}
```

```
while(testExpresison) {
    // codes
    while (testExpression) {
        // codes
        if (condition for continue) {
            continue;
        }
        // codes
    }
    // codes
}
```

return statement

- The **return** statement exits from the current method, and control flow returns to where the method was invoked.
- The return statement has two forms: one that returns a value, and one that doesn't.
- To return a value, simply put the value (or an expression that calculates the value) after the return keyword.
- **return ++count;**
- The data type of the returned value must match the type of the method's declared return value.
- When a method is declared void, use the form of return that doesn't return a value.
- **return;**

```
public class ReturnValue
{
    public int addition(int a,int b)
    {
        int sum = a+b;
        return sum;
    }

    public static void main(String args[])
    {
        ReturnValue r = new ReturnValue();
        int answer;

        answer = r.addition(10,20);
        System.out.println("Addition = "+answer);
    }
}
```

Functions

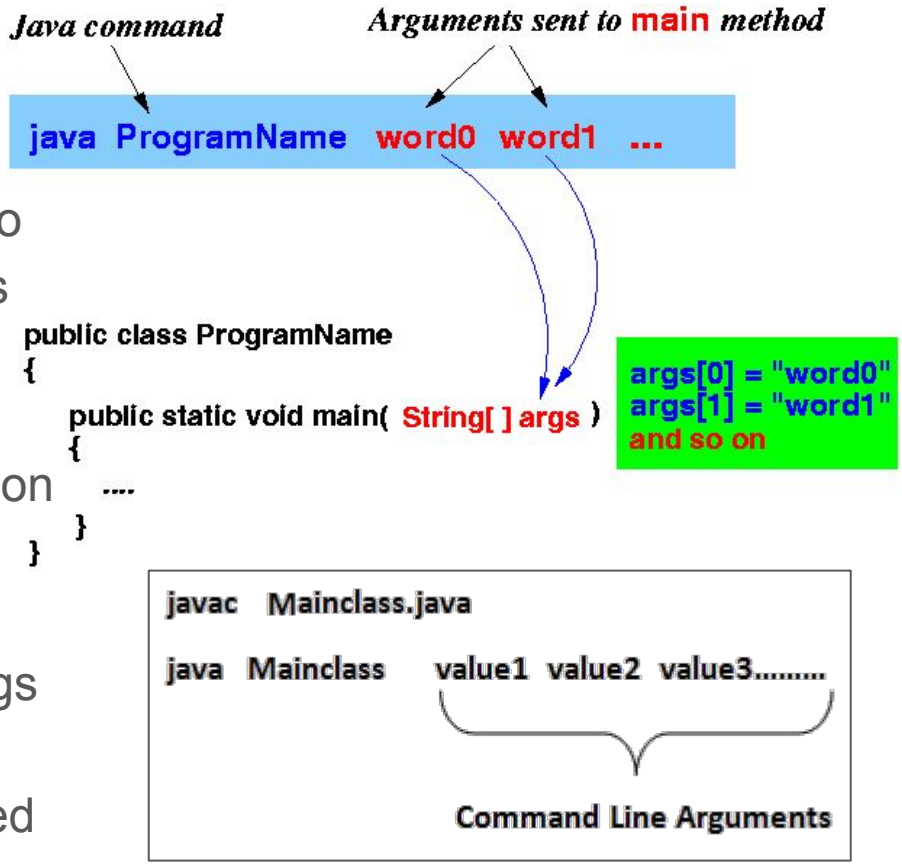
In Java, a function is referred to as a method.
A method is a block of code that performs a specific task and can be invoked or called to execute that task.
They are always defined within classes.
They promote code reusability, modularity, and organization.
Types of Methods in Java:

- **Pre-defined Methods (Standard Library Methods).**
- **User-defined methods.**

```
return_type function_name(arguments)
{    // body of the function
      return return_value;  }
```

Command-Line Arguments

- If we want to pass information into a program when you run it, then you can do this by passing command-line arguments to main( ).
- A command-line argument is the information that follows program's name on the command line when it is executed.
- Command-line arguments are stored as strings in a String array passed to the args parameter of main( ).
- The first command-line argument is stored at args[0] the second at args[1] so on.
- In C/C++ args[0] will be the executable name.



```
// Display all command-line arguments.
class CommandLine {
        public static void main(String args[]) {
            for(int i=0; i<args.length; i++)
              System.out.println("args[" + i + "]: " + args[i]);
}
}
```

- Compile this usig javac and execute this program as:-

  java CommandLine this is a test 100 -1

```
args[0]: this
args[1]: is
args[2]: a
args[3]: test
args[4]: 100
args[5]: -1
```

Variable length arguments

- In Java methods can take a variable number of arguments.
- This feature is called varargs or variable-length arguments.
- A method that takes a variable number of arguments is called a variable-arity method, or simply a varargs method.
- If the maximum number of arguments is small and known, then we can create overloaded versions of the method, one for each way the method could be called.
- If the maximum number of potential arguments is larger, or unknowable, then the arguments can be put into an array, and then the array can be passed to the method.

```
public static void mySum(double... arr) {
    double sum = 0.0;
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
    }
    System.out.println(sum);
}
```

Handling variable length arguments

- A variable-length argument is specified by three periods (...)
- E.g. static void test(int ... v) { //statemenst }
- This syntax tells the compiler that test( ) can be called with zero or more arguments.

```
//example method that takes a variable number of parameters

int sum(Integer... numbers) {

      for(int i: numbers)
      // do something
}

// Code fragment that calls the sum method
sum(12,13,20);
```

```
public static void mySum(double... arr) {
    double sum = 0.0;
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
    }
    System.out.println(sum);
}
```

```
class PassArray {
static void test(int ...v)
{
System.out.print("Number of args: " + v.length + " Contents: ");
for(int x : v)
  System.out.print(x + " ");
System.out.println();
}
public static void main(String args[])
{
   test(10);      // 1 arg
   test(1,2,3);   // 3 args
   test();        // no args
}
}
```

OUTPUT

```
Number of args: 1 Contents: 10
Number of args: 3 Contents: 1 2 3
Number of args: 0 Contents:
```

Handling variable length arguments

- A method can have “normal” parameters along with a variable-length parameter.
- However, the variable-length parameter must be the last parameter declared by the method.
- E.g: int test(int a, int b, double c, int ... vals) { //statements } //VALID
- E.g: int test(int a, int b, double c, int ... vals, boolean stopFlag) {} // ERROR!
- We can overload a method that takes a variable-length argument.
- There can be many functions with same name and having different type of variable length arguments.

- // Varargs and overloading.

```
class VarArgs3
{
    static void test(int ... v)
    {
    System.out.print("test(int ...): " + "Number of args: " + v.length);
    }
    static void test(boolean ... v)
    {
    System.out.print("test(boolean ...) " +"Number of args: " + v.length);
    }
  public static void main(String args[])
  {
        test(1, 2, 3);
        test(true, false);
  }
}
```

OUTPUT

test(int ...): Number of args: 3
test(boolean ...): Number of args: 2

Varargs and Ambiguity

- It is possible to create an ambiguous call to an overloaded varargs method.

```
class VarArgs3
{
    static void test(int ... v)
    {
    System.out.print("test(int ...): " + "Number of args: " + v.length);
    }
    static void test(boolean ... v)
    {
    System.out.print("test(boolean ...) " +"Number of args: " + v.length);
    }
  public static void main(String args[])
  {
        test(1, 2, 3);
        test();    // Error: Ambiguous!
  }
}
```

test() can call
test(int ...) or **test(boolean ...).**
Because both these functions
have varargs so they can accept
zero arguments .
System is confused which one to call
AMBIGUITY

Varargs and Ambiguity

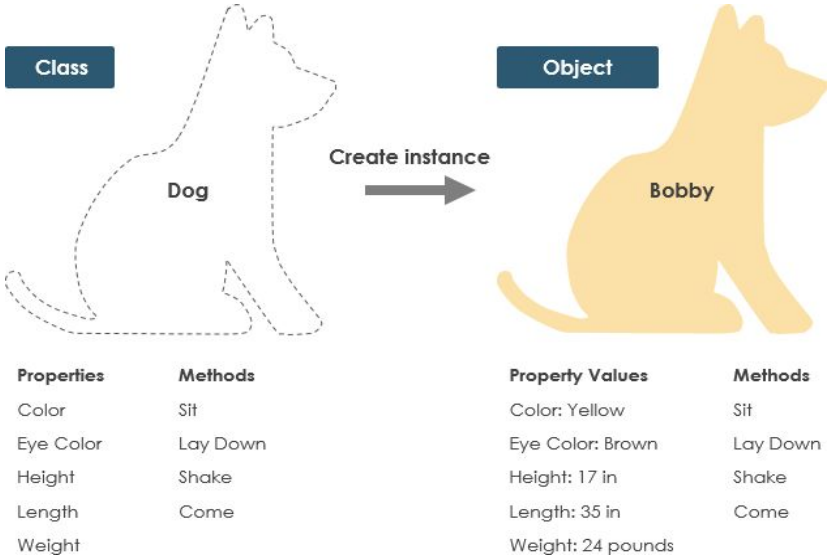
- Examine following methods
- static void test(int ... v) { // ... }
- static void test(int n, int ... v) { // ... }

If a call test(2); comes, then how it will be treated by the compiler?

Class & Object

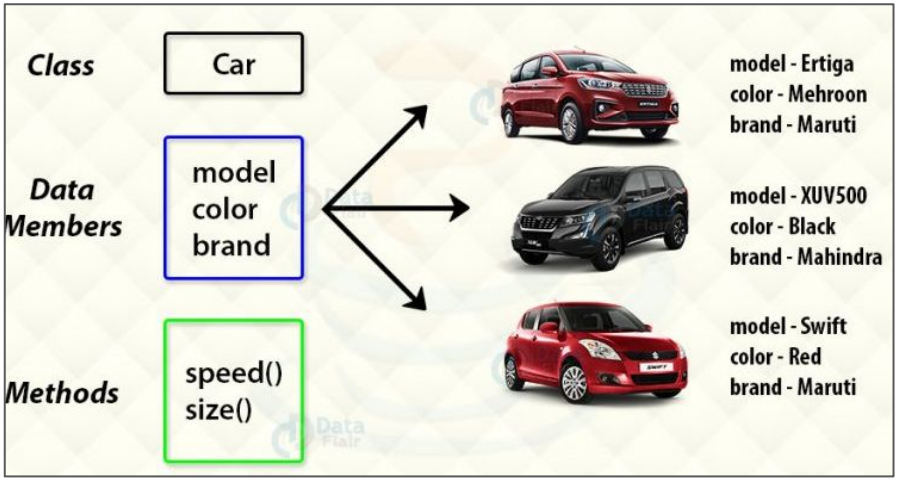
CLASS

- A class is a **blueprint** or prototype from which objects are created
- A class is a **generalized description** of an object. **No physical existence**.
- Acts as a type or category.



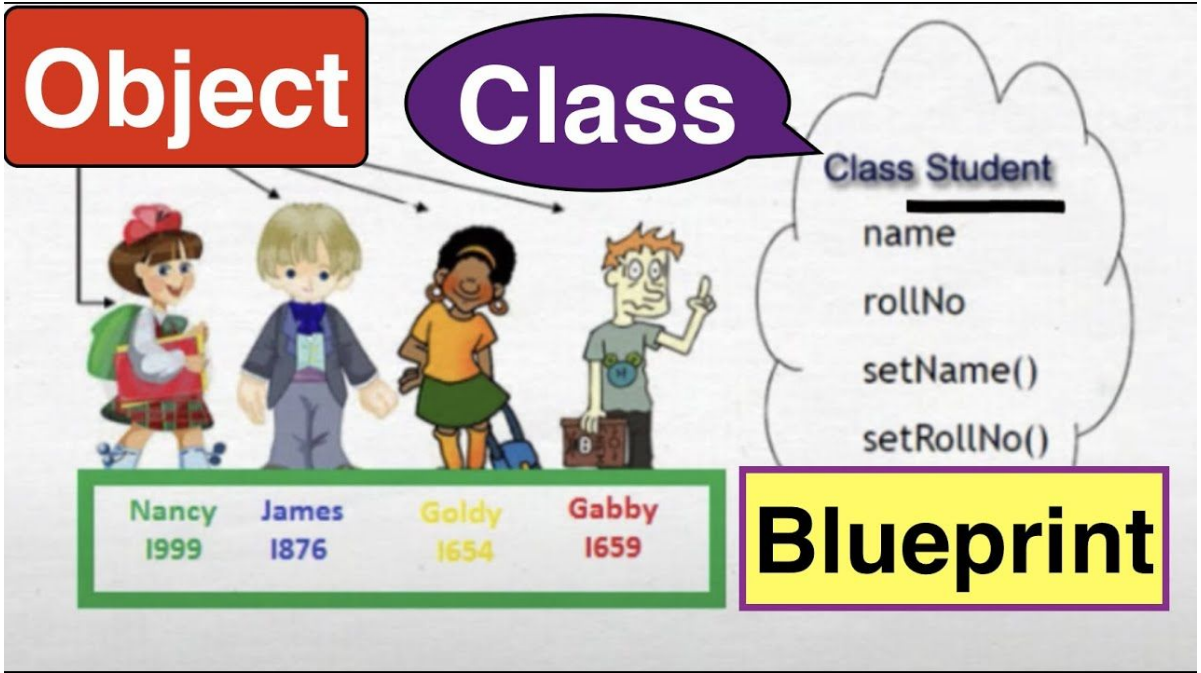
OBJECT

- An object is an instance of a class
- Objects are real-world entities that has their own **state** and **behavior**.
- An object has **physical existence**



Class & Object

- Let's take **Student** as a class. **James** is an instance / object of the class Student
- Object has a physical existence while a class is just a logical definition.

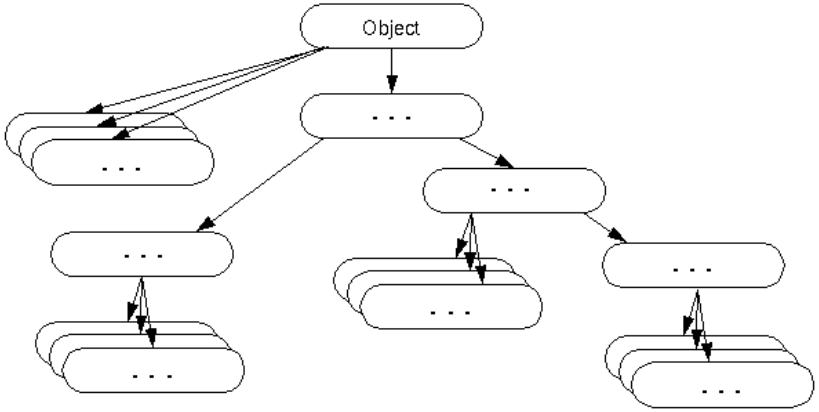


State => Variables
Name
RollNo

Behaviors => Functions
setName()
setRollNo()

Object class

The Object class is the parent class of all the classes in java by default. In other words, it is the topmost class of java.



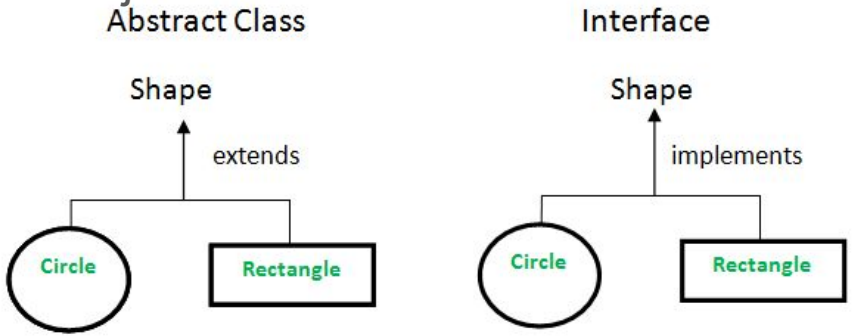
The Object class is beneficial if you want to refer any object whose type you don't know. Notice that parent class reference variable can refer the child class object, know as upcasting.

Object class methods

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.
public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

Abstract classes and methods

- Programming by contract
- Abstraction is a design level activity whereas encapsulation is implementation level
- Data abstraction is the process of hiding certain details and showing only essential information to the user.
- Abstraction can be achieved with either abstract classes or interfaces (Interface - which you will learn more about in the next chapter).
- Ways to achieve Abstraction
- There are two ways to achieve abstraction in java
- Abstract class (0 to 100%)
- Interface (100%)

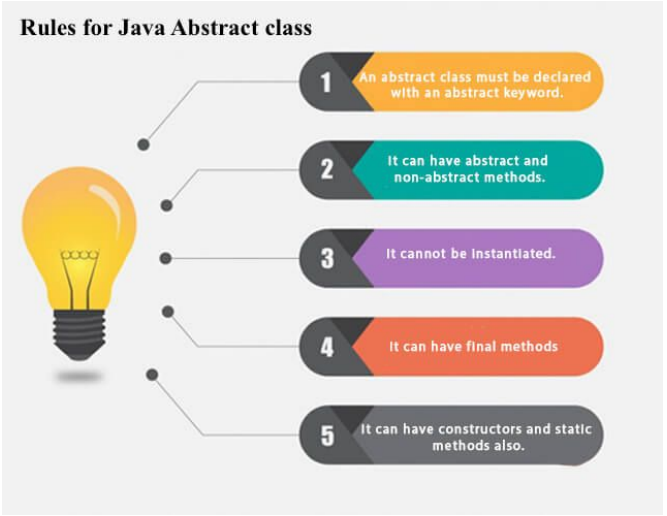


Abstract classes and methods

- The abstract keyword is a non-access modifier, used for classes and methods:
- Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

Abstract classes

- A class which is declared as abstract is known as an abstract class.
- Points to Remember
- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static metho also.
- It can have final methods which will force subclass not to change the body of the method.




```
abstract class ClassName
{
        ...
        ...
        abstract Type MethodName1();
        ...
        ...
        Type Method2()
        {
           // method body
        }
}
```

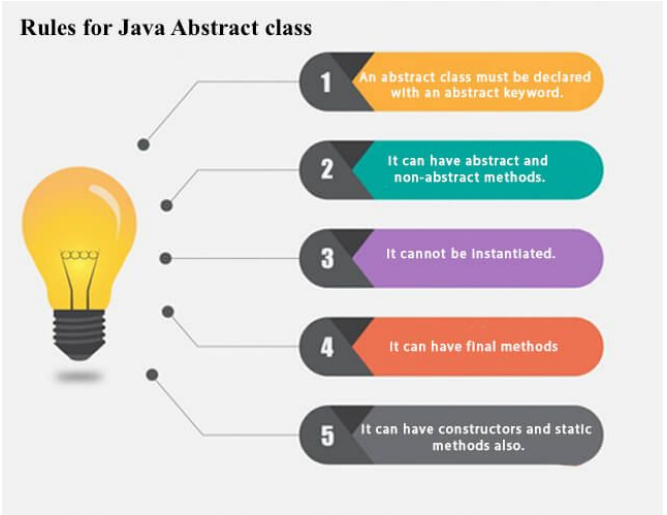
- When a class contains one or more abstract methods, it should be declared as abstract class.
- The abstract methods of an abstract class must be defined in its subclass.
- We cannot declare abstract constructors or abstract static methods.

Abstract classes

```
abstract class Vehicle
{
public abstract void start();
public abstract void gearShift();
}
class ElectricCarAT extends Vehicle
{
public void start(){      }

public void gearShift(){   }

}
```



Interfaces

An **interface** in Java is a reference type that acts as a formal contract or blueprint specifying what a class must do, without defining how it does it.

No Direct Instantiation: You cannot create an object directly from an interface.

Implicit Modifiers: All declared fields are implicitly public static final (constants).

Method Types Allowed:Abstract methods (default): Declared without a body.

Default methods (Java 8+): Include a fallback body using the default keyword.

Static methods (Java 8+): Belong to the interface itself and cannot be inherited.

Private methods (Java 9+): Used strictly for internal utility inside the interface.

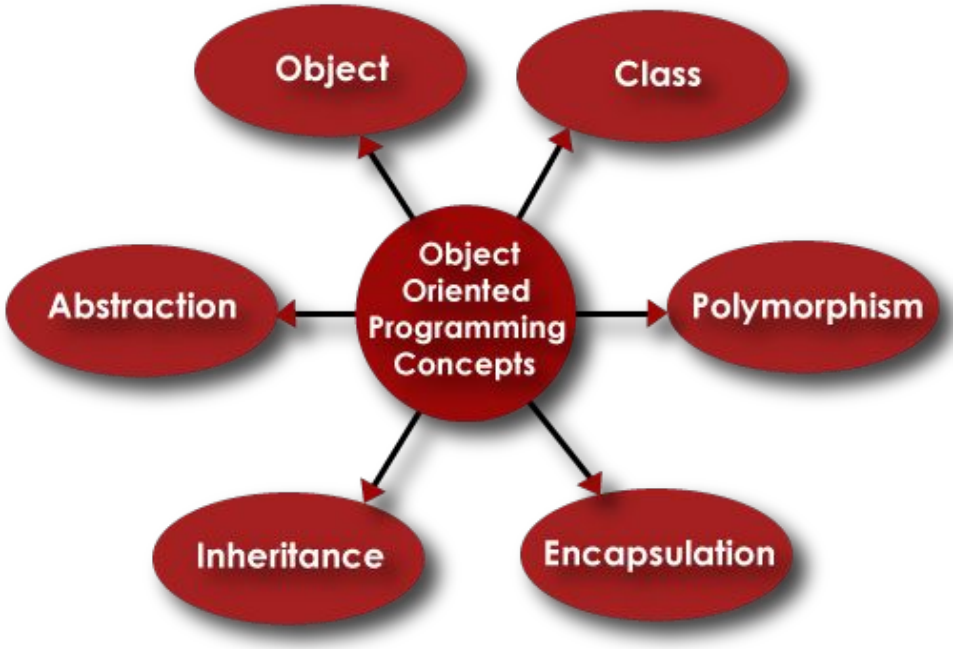
Interfaces

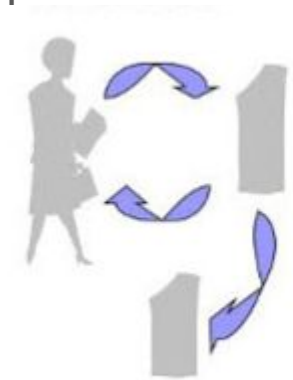
```
// Define the interface
interface Animal {
void makeSound(); // Abstract method (no body)
}
// Implement the interface
class Dog implements Animal
{
    public void makeSound()
    { System.out.println("Bark!");
    }
}
```

Basic Object Oriented Concepts

In OOD, implementation of a software is based on the concepts of **objects**.

This approach is very close to the real-world applications





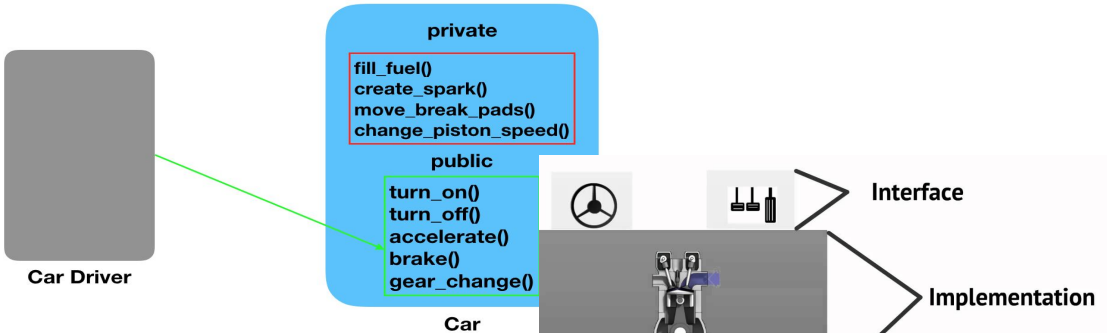
Withdraw, deposit, transfer



Customer, money, account

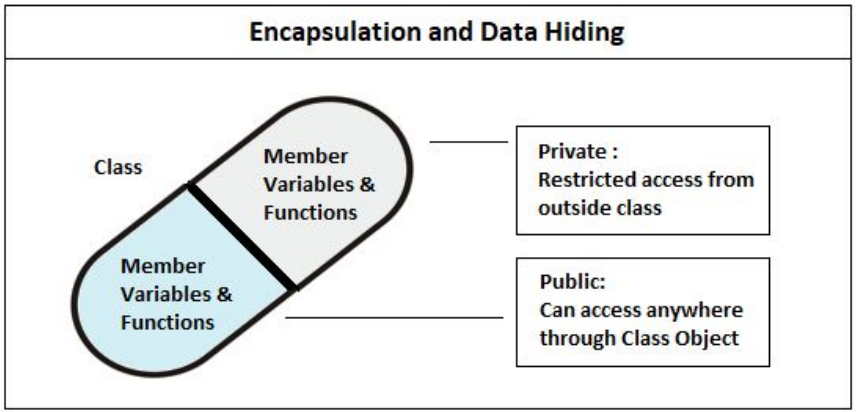
Abstraction

- Abstraction means displaying only essential information and hiding the internal details.
- It is a process of **generalizing** things at **design level**.
- Data abstraction refers to providing **only essential information** about the data to the outside world, hiding the background details or implementation.
- Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car.
- All about **showing necessary** details.



Encapsulation

- The **wrapping up** of data(variables) and functions (methods) that operates on the data, into a single unit (called class) is known as encapsulation.
- It is also called "**information hiding**“, hides the internal representation, or state, of an object from the outside through access modifies (Private / Public).
- **Hiding unnecessary** details, at implementation level.



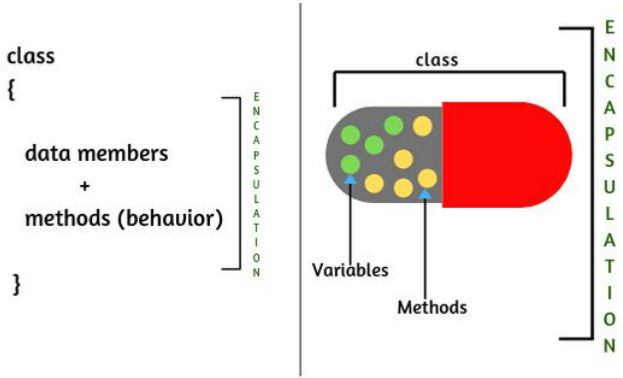
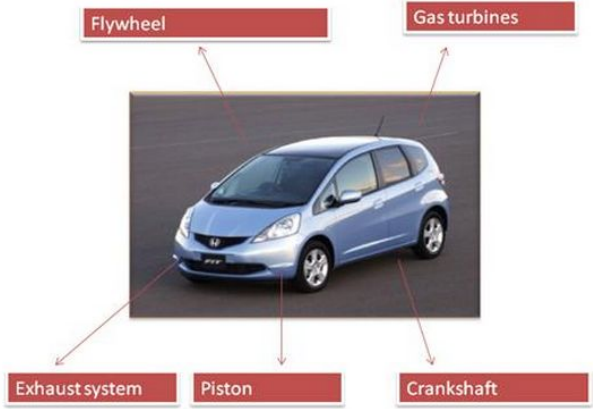
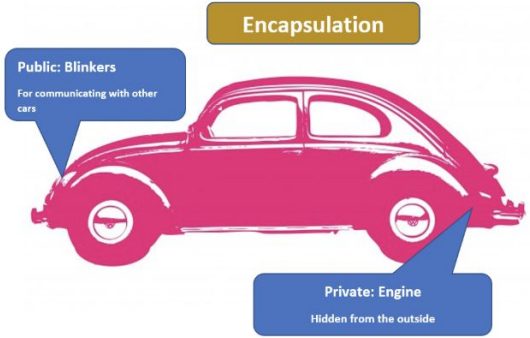


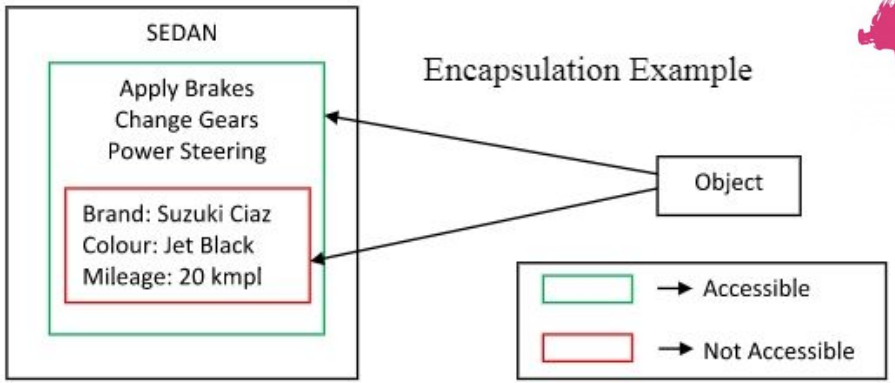
Fig: Encapsulation

Encapsulation...



- **Protection of data** from accidental corruption, by means of **access specifiers** (Private / Public), it **provides security.**
- Flexibility and extensibility of the code and reduction in complexity
- Encapsulation of a class can hide the internal details of how an object does something
- Encapsulation protects abstraction

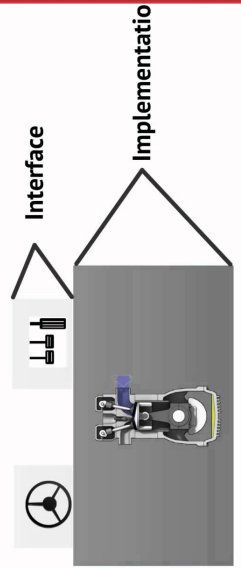




Abstraction & Encapsulation



Abstraction	Encapsulation
Abstraction is a general concept formed by extracting common features from specific examples or The act of withdrawing or removing something **unnecessary**.	Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both **safe from outside interference** and **misuse**.
You can use abstraction using **Interface** and **Abstract** Class	You can implement encapsulation using **Access Modifiers** (Public, Protected & Private)
Abstraction solves the problem in **Design** Level	Encapsulation solves the problem in **Implementation** Level
For simplicity, abstraction means hiding implementation using Abstract class and Interface	For simplicity, encapsulation means hiding data using getters and setters



Inheritance

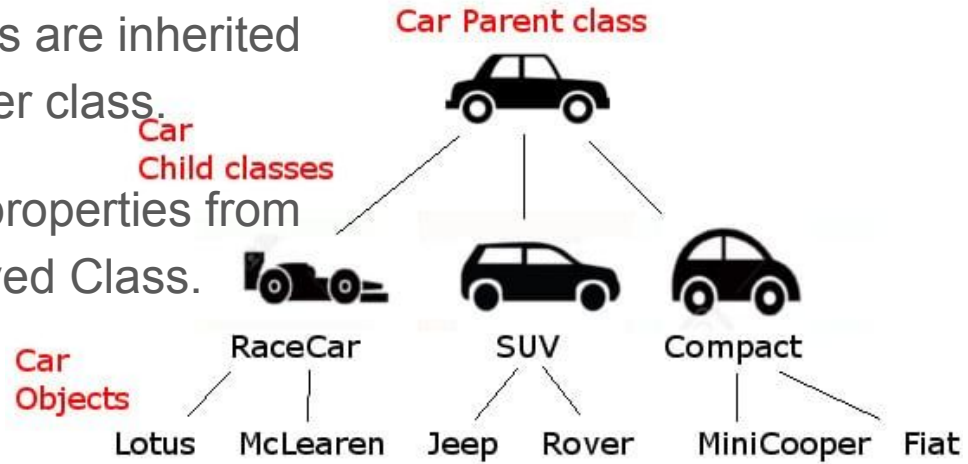


The capability of a class to derive properties and characteristics from another class is called Inheritance.

Inheritance is the process by which objects of one class acquired the properties of objects of another classes

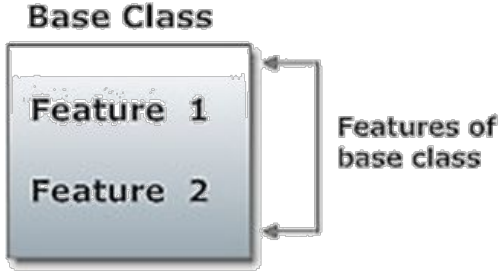
Base Class : The class whose properties are inherited by subclass is called Base Class or Super class.

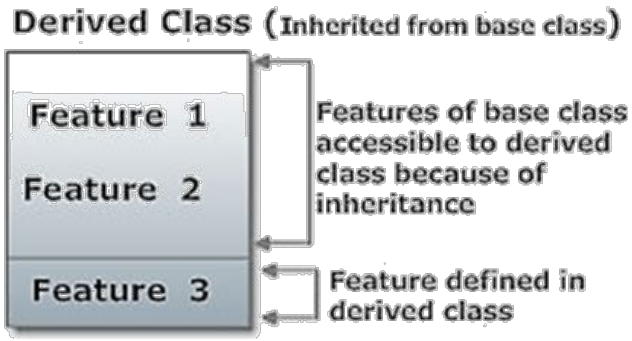
Derived Class : The class that inherits properties from another class is called Subclass or Derived Class.

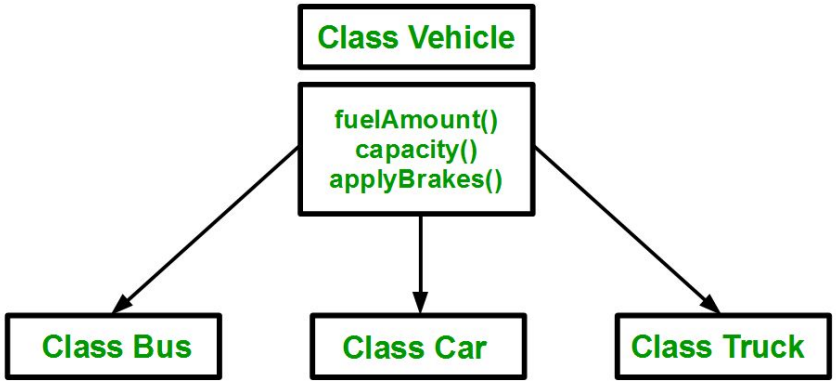


Inheritance

Reusability: Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.







Types of Inheritance

01 Single Inheritance

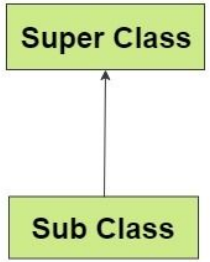
02 Multilevel Inheritance

03 Hierarchical Inheritance

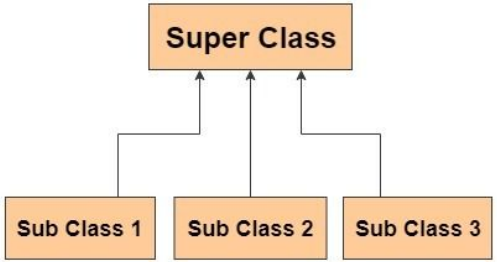
04 Multiple Inheritance

05 Hybrid Inheritance

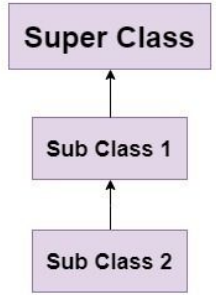
Single Inheritance



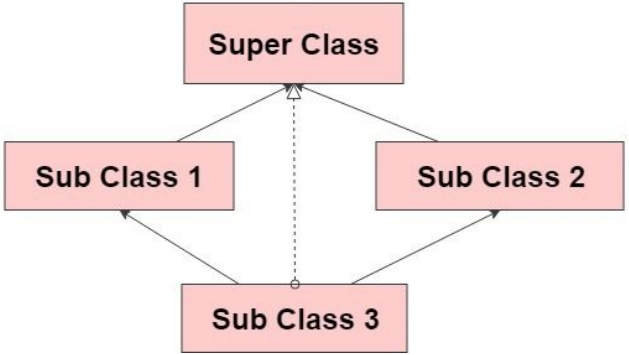
Hierarchial Inheritance



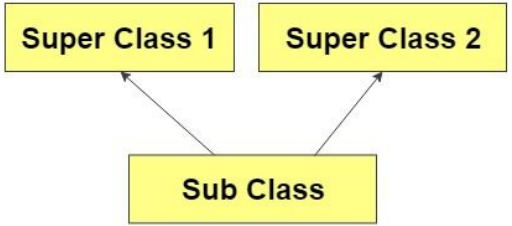
MultiLevel Inheritance



Hybrid Inheritance

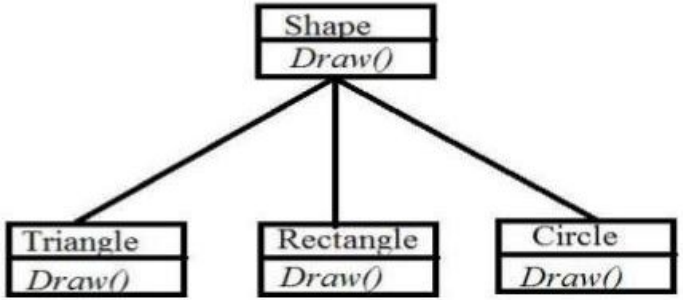


Multiple Inhertance



Polymorphism

- The word polymorphism means **having many forms.**
- Refers to a programming language's **ability to process objects differently** depending on their **data type or class**
- An operation may exhibit different behaviors in different instances. The behavior depends upon the **types of data used in the operation**.
- Ability for objects of different classes related by inheritance to **respond differently to the same member function** call





In Shopping malls behave like Customer

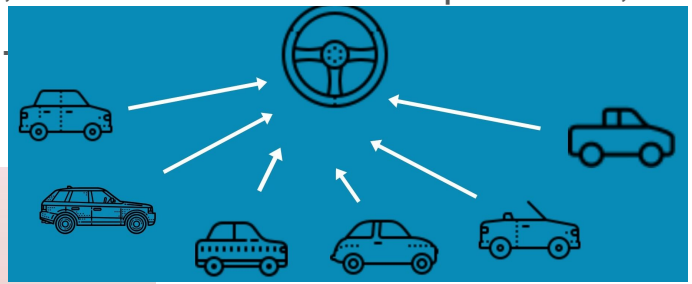
In Bus behave like Passenger

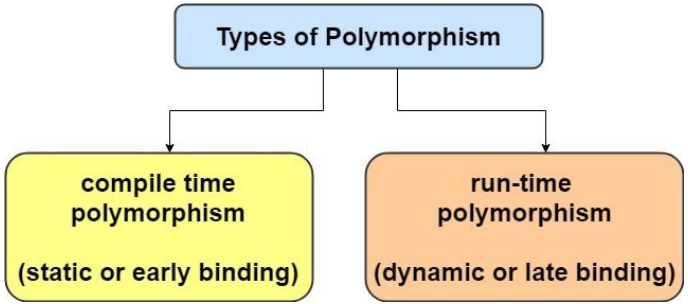
In School behave like Student

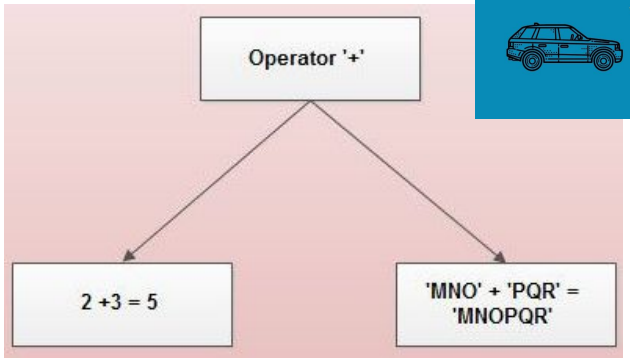
At Home behave like Son

Polymorphism...

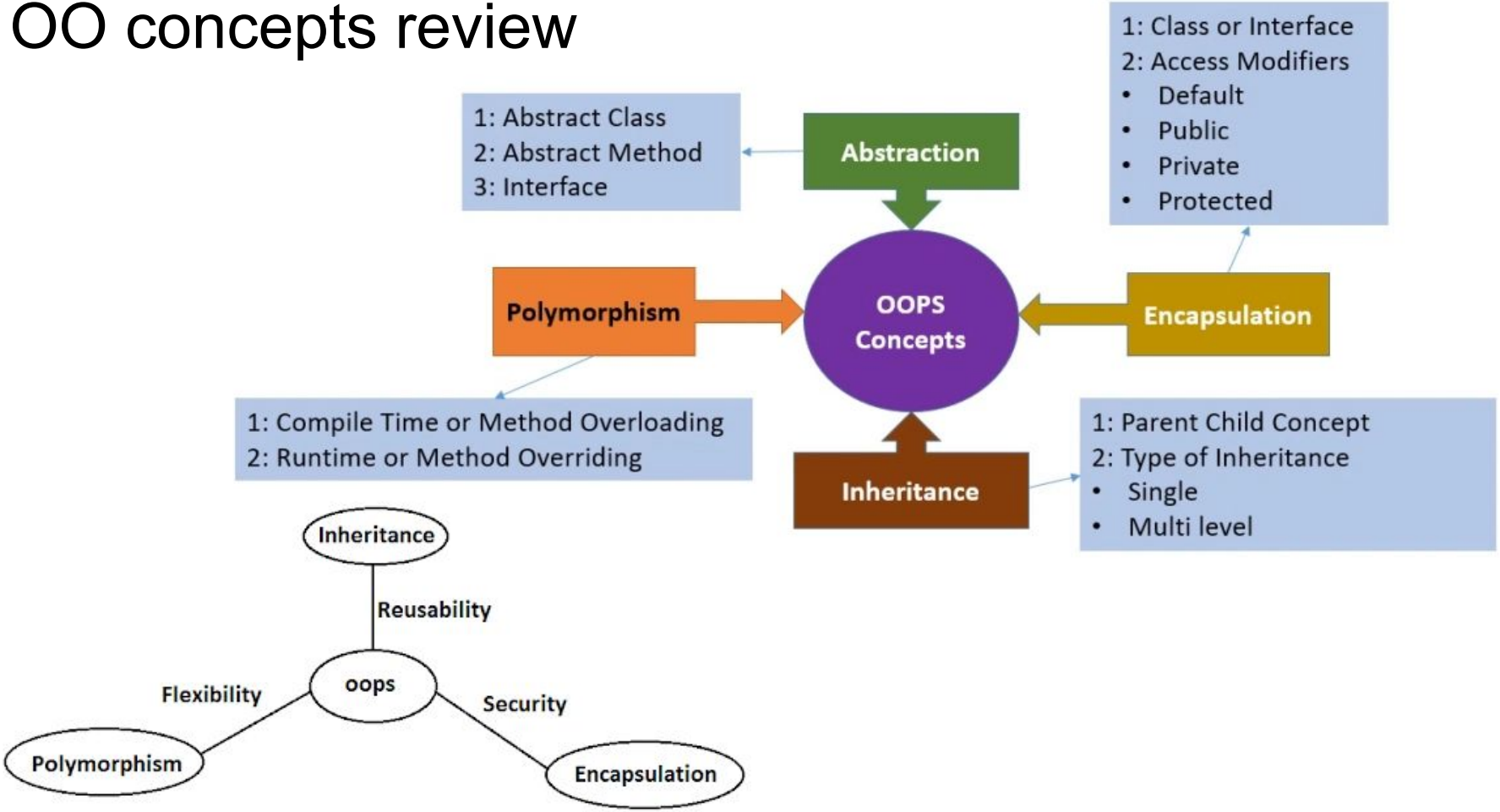
- **Static polymorphism**: the binding between the method call an the method body happens at the time of compilation and, this binding is known as static binding or early binding
- **Runtime polymorphism** or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime.
- Dynamic binding happens when all information needed for a function call cannot be determined at compile-time
- In the example, same rotation of steering wheel causes, different mechanical operations, corresponding to the steering type present in each cars.
- Brings **flexibility** to the implementations







OO concepts review



END OF LECTURE 2

Function Oriented Design (FOD)

These functional modules can share information among themselves by means of information passing and using information available **globally**.

Eg: Banking process

Here **withdraw**, **deposit**, **transfer** are functions and that can be divided into subfunctions again.

So, in FOD, the entire problem is divided in to number of functions and those functions are broken down into smaller functions and these smaller functions are converted in to software modules.



Object Oriented Design (OOD)



- OOD is based on **Objects** and **interaction between the objects.** Interaction between objects is called message communication.
- In the object-oriented design approach, the system is viewed as a collection of objects (i.e. entities).
- It involves the designing of **objects**, **classes** and the **relationship** between the classes
- The **state is decentralized** among the objects and each object manages its own state information.
- OOD follows bottom up approach

Consider the example of Banking process.

- Here, **customer**, **money** and **account** are objects

Object Oriented Design (OOD)

For example, in banking example,each customer may be a separate object with its **own data** and **functions to operate on these data**.

Functions defined for one object cannot refer or change data of other objects.

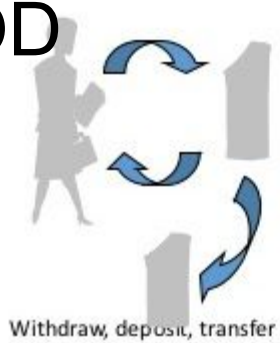
Objects have their **own internal data** which define their state.

Similar objects constitute a class. In other words, each object is an instance of some class.

Classes may **inherit** features from super class. Conceptually, objects communicate by message passing.

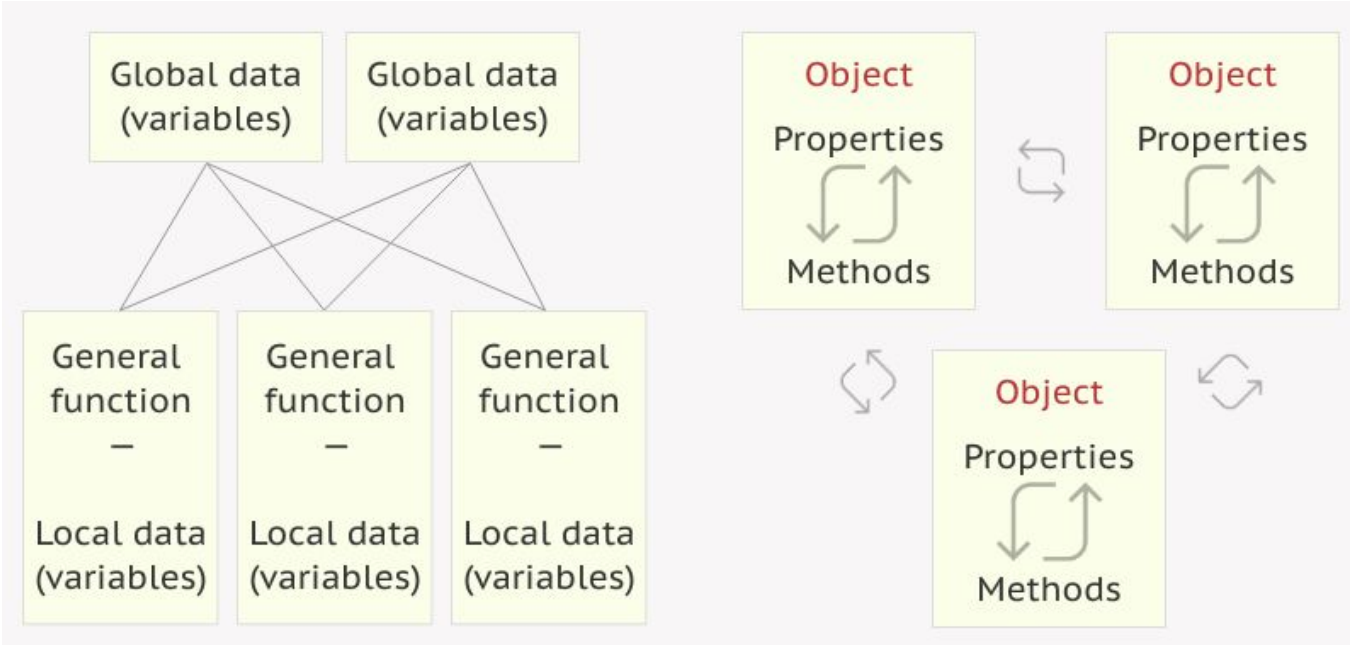
FOD vs OOD

- Procedural



- Object Oriented





Advantages of OOP

- Code and design reuse
- Increased productivity
- Ease of testing and maintenance
- Better code and design understandability enabling development of large programs

Disadvantages of OOD

- The principles of abstraction, data hiding, inheritance, etc. do incur runtime overhead.
- An important consequence of object-orientation is that the **data gets scattered across various objects** in an object-oriented implementation.

Java Micro Services

• A software architecture approach where applications are built as a collection of small, independently deployable services, each handling a specific business capability.

• Services, communicate with each other using protocols like HTTP, REST or gRPC.

• Think of it like a building constructed from Lego bricks, where each brick (microservice) has a specific function and can be assembled and reassembled to create different structures.

• Spring Boot, Jersey, and Swagger are some Popular Java frameworks used for building microservices.

Java Micro Services

Some Key Characteristics of Java Microservices:

- Modular architecture: The application is broken down into a set of loosely coupled services
- Language agnostic: Services can be written in different programming languages
- Scalability: Individual services can be scaled independently based on demand
- Resilience: Failure of one service does not impact others
- Flexibility: Services can be modified, updated or replaced independently

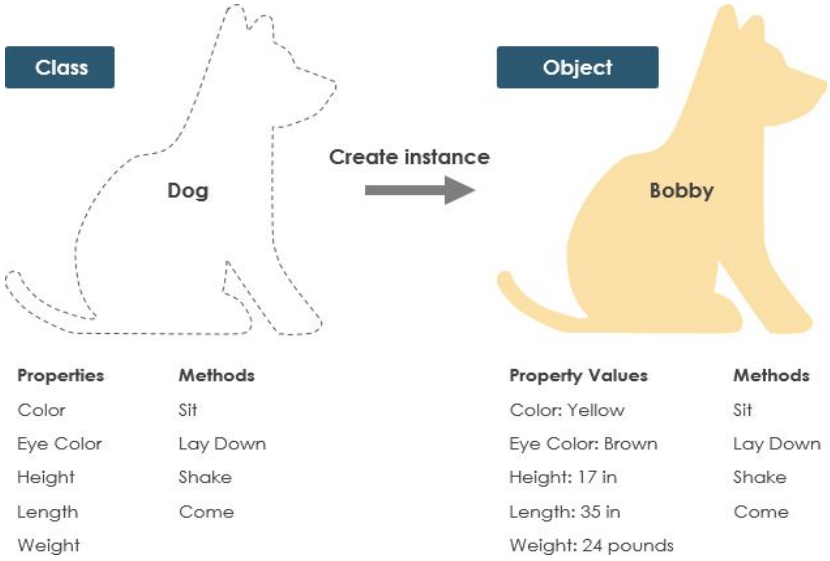
OO Concepts: Class & Object

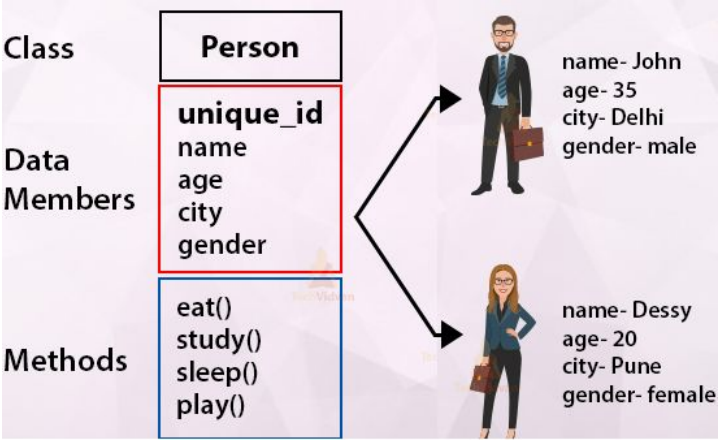
CLASS

- A class is a **blueprint** or prototype from which objects are created
- A class is a **generalized description** of an object. **No physical existence**.
- Acts as a type or category.

OBJECT

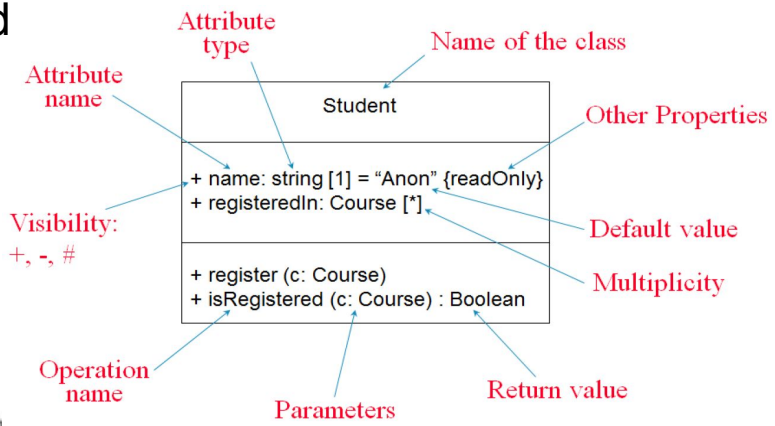
- An object is an instance of a class
- Objects are real-world entities that has their own **state** and **behavior**.
- An object has **physical existence**

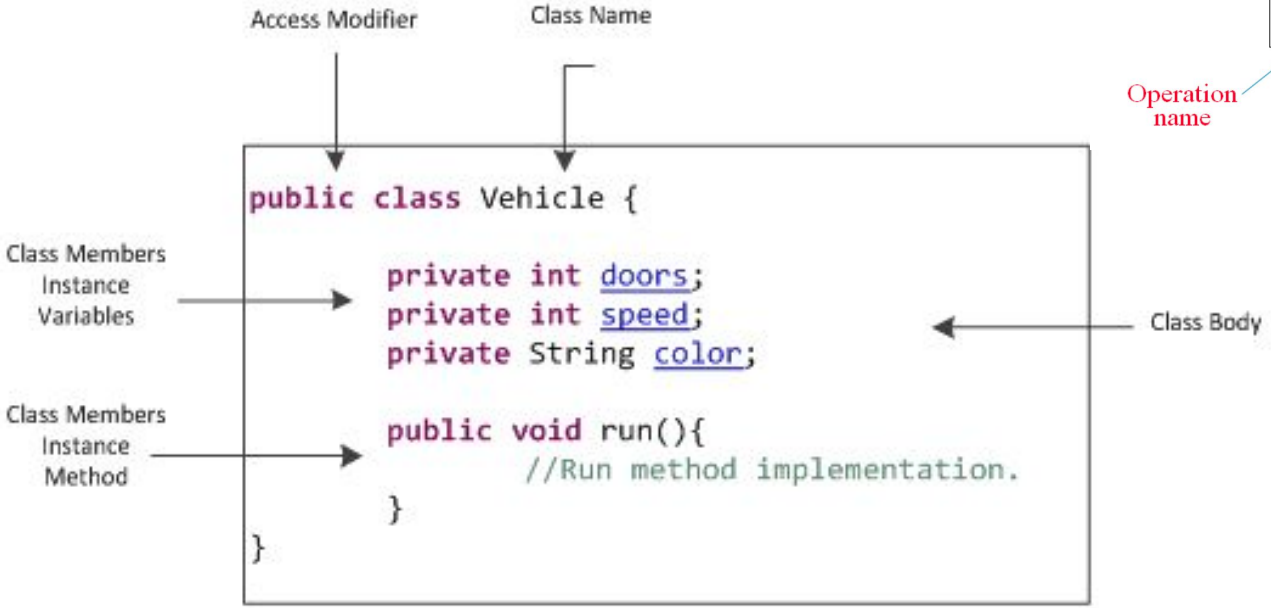




OOP in Java - Class Fundamentals

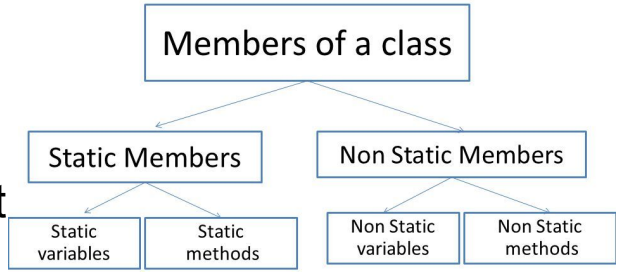
- Everything in Java is associated with classes and objects, along with its **attributes** and **methods**
- Class members can be either the part of instance(object) or Class (static members)





Class members

- A class can contain methods and member variables.
- Class members can be either the part of instance(object or Class (static members).
- Unless otherwise specified, a member declarec class is an instance member.

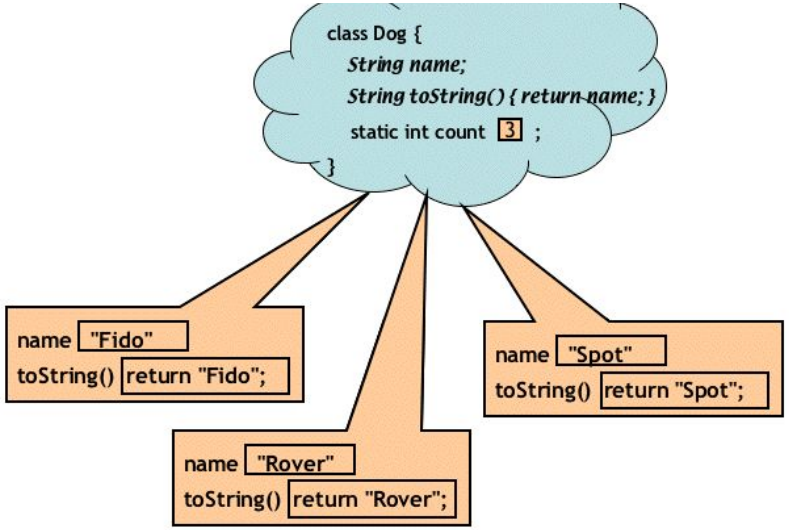



```
class Student{
   int rollno;
   String name;
   static String college = "SBC";

   static void change(){
   college = "AAA";
   }
   Student(int r, String n){
   rollno = r;
   name = n;
   }
```

Static variable

Static method



Creating Objects

- In Java, the **new** keyword is used to create new objects (instance of class).
- There are three steps when creating an object from a class −

1. **Declaration** − A variable declaration with a variable name with an object type.
2. **Instantiation** − The 'new' keyword is used to create the object.
3. **Initialization** − The 'new' keyword is followed by a call to a constructor. This call initializes the new object

```
public class Puppy {
   String m_name; // Member variable
   public Puppy() {   }
   public Puppy(String name) {
      m_name = name ;   }

   Public void whoareyou()
       { System.out.println("I am "+m_name);}

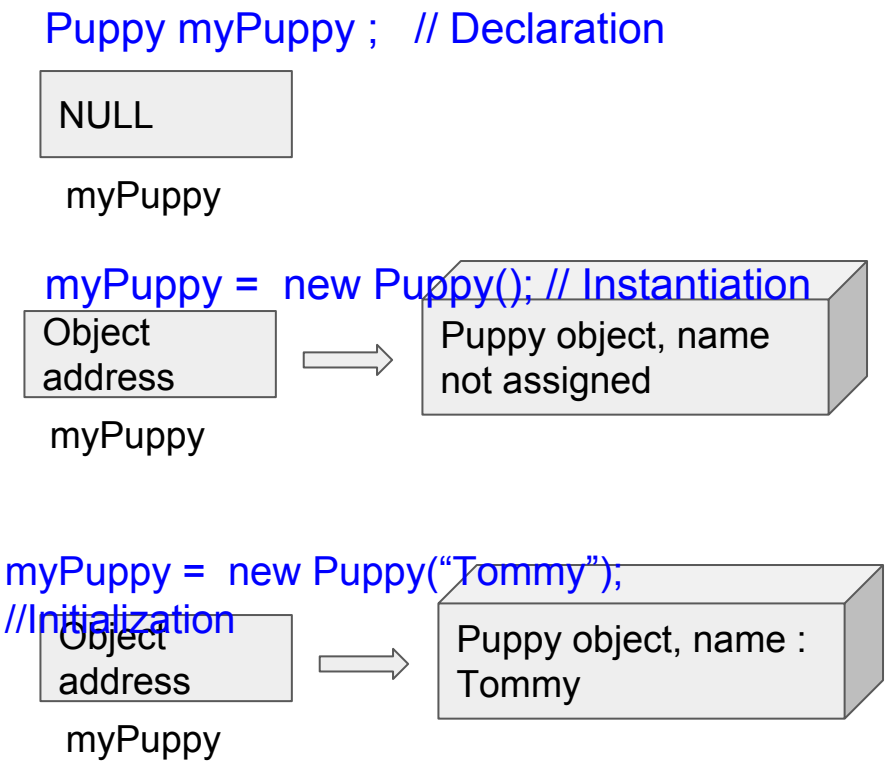
       public static void main(String args[]){
       Puppy myPuppy =  new
Puppy("Tommy");
      myPuppy.whoareyou();}
}
```

```
Puppy myPuppy ;   // Declaration
myPuppy =  new Puppy(); // Instantiation
myPuppy =  new Puppy("Tommy"); //Initialization
```

Creating Objects

- In Java, the **new** keyword is used to create new objects (instance of class).
- There are three steps when creating an object from a class −

1. **Declaration** − A variable declaration with a variable name with an object type.
2. **Instantiation** − The 'new' keyword is used to create the object.
3. **Initialization** − The 'new' keyword is followed by a call to a constructor. This call initializes the new object.



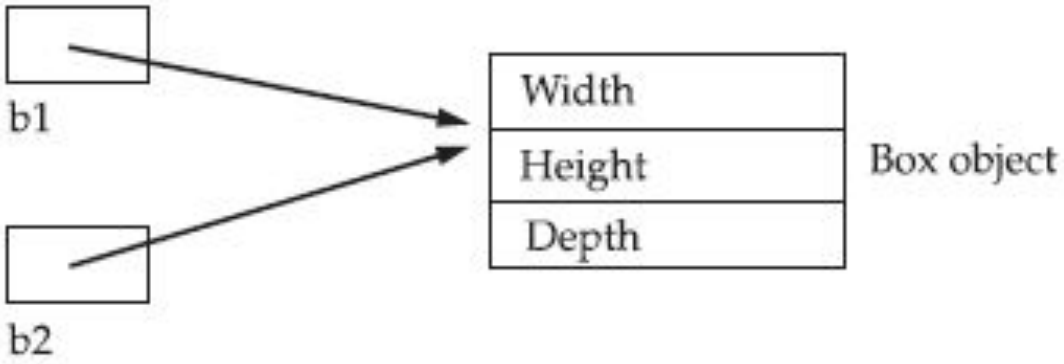
Assigning Object Reference Variables

- When you assign one object reference variable to another object reference variable, you are not creating a copy of the object, you are only making a copy of the reference.

```
Box b1 = new Box();
Box b2 = b1;
```

- When you assign null or another value to an object reference, you are not making any change to the object. Change is only to the reference (link).

```
b1 = null; // removes the link between the object and b1.
```



Accessing Instance Variables and Methods

- A class's methods can access the class's instance variables and call its methods. In particular, the main method for a class is often used to test the class's methods.

- An instance variable is accessed with the expression referenceVariable.instanceVariableName. Similarly, a method is called with the expression referenceVariable.methodName (parameters).

- Instance variables and methods are accessed via created objects
- /* First create an object */
- ObjectReference = new Constructor();
- /* Now call a variable as follows */
- ObjectReference.variableName;
- /* Now you can call a member method as follows */
- ObjectReference.MethodName();
- Default values for static and non-static variables are same.
- primitive integers(long, short etc): 0
- primitive floating points(float, double): 0.0
- boolean: false , object references: null

```java
public class Counter {
    int myCount = 0;
    void increment ( ) {
        myCount = myCount + 1;
        print();
    }
    void print ( ) {
        System.out.println ("count = " + myCount);
    }
    public static void main (String [ ] args) {
        Counter c1 = new Counter ( );
        Counter cx = c1 ;  // cx is refereing to c1
        c1.increment ( );  // c1's myCount is now 1
        c1.increment ( );  // c1's myCount is now 2
        c1 = null ;
        cx.increment ( );  // cx points to same obj, so 3
        cx.myCount = 0 ;   // resetting count 0
        cx.increment ( );  // cx's myCount is now 1
        cx = new Counter ( ); // new object
        cx.increment ( );  // c1's myCount is now 1
    }
}
```

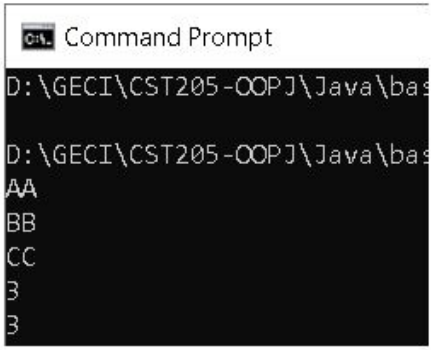
```
D:\GECI\CST205-OOPJ\Java\basics>java Counter
count = 1
count = 2
count = 3
count = 1
count = 1
```

Sample program

Static variables

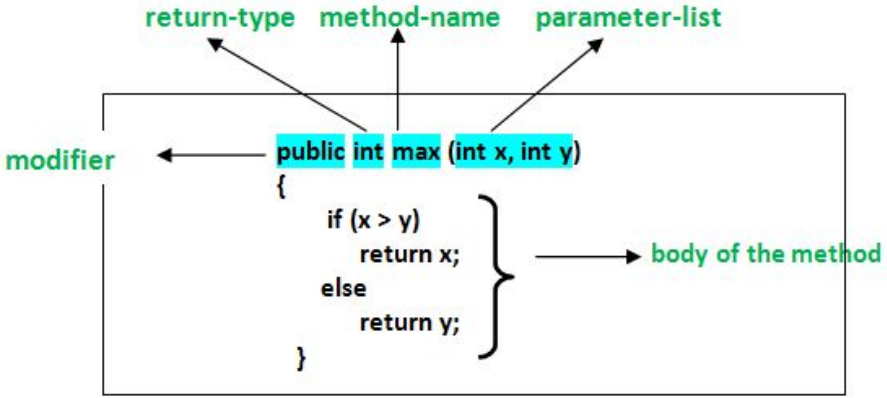
- A static variable is common to all the instances (or objects) of the class because it is a class level variable.
- Static variables can be accessed by calling with the class name **ClassName. VariableName.**
- Static variables are initialized when class is loaded.
- Static variables are initialized before any object of that class is created.
- Static variables are initialized before any static method of the class executes.

```
public class StaticCounter {
    static int myCount = 0;
    String name ;
    StaticCounter(String n)
    {
        name = n ;
        myCount++ ;
    }
    public static void main (String [ ] args) {
        StaticCounter c1 = new StaticCounter ( "AA");
        StaticCounter c2 = new StaticCounter ( "BB");
        StaticCounter c3 = new StaticCounter ( "CC");
        System.out.println(c1.name);
        System.out.println(c2.name);
        System.out.println(c3.name);
        System.out.println(StaticCounter.myCount);
        System.out.println(c1.myCount);
    }
}
```



methods

- Methods are same as that of the functions in C. But in Java they will be part of any class declaration.
- A method is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a method.
- Methods are used to perform certain actions, and they are also known as functions.




```java
public class MyClass {
  static void myMethod() {
    System.out.println("I just got executed!");
  }

  public static void main(String[] args) {
    myMethod();
    myMethod();
    myMethod();
  }
}

// I just got executed!
// I just got executed!
// I just got executed!
```

Instance methods (non static methods)

- Instance method are methods which require an object of its class to be created before it can be called.
- To invoke a instance method, we have to create an Object of the class in within which it defined.
- Memory allocation: These methods themselves are stored in heap but the parameters and their local variables and the value to be returned are allocated in stack.
- They can be invoked simply using the method name, by the instance methods within the same/derived class in which they reside.
- From unrelated classes or from static methods, instance methods can be called only by creating the class object. ie **objectname.methodname();**
- Instance method(s) belong to the Object of the class not to the class i.e. they can be called after creating the Object of the class.
- Every individual Object created from the class has its own copy of the instance method(s) of that class.
- They can be overridden since they are resolved using dynamic binding at run time.

Instance methods (non static methods)

```java
class UnrelatedClass
{
    int result ;
    void add(int a, int b) // instance method
    {
        result = a + b ;
        printResult(); // calling another instance metho
    }
    void printResult()
    { System.out.println("Result : " + result);
    }
}
public class InstanceMethod
{
    static void staticMethod()
    {
        System.out.println("I am static method");
    }
    void nonStaticMethod()
    {
        System.out.println("I am non-static method");
    }
    public static void main(String[] args)
    {
        UnrelatedClass uc = new UnrelatedClass();
        uc.add(4,5);
        staticMethod() ; // Calling static method
        InstanceMethod im = new InstanceMethod();
        im.nonStaticMethod();
    }
}
```

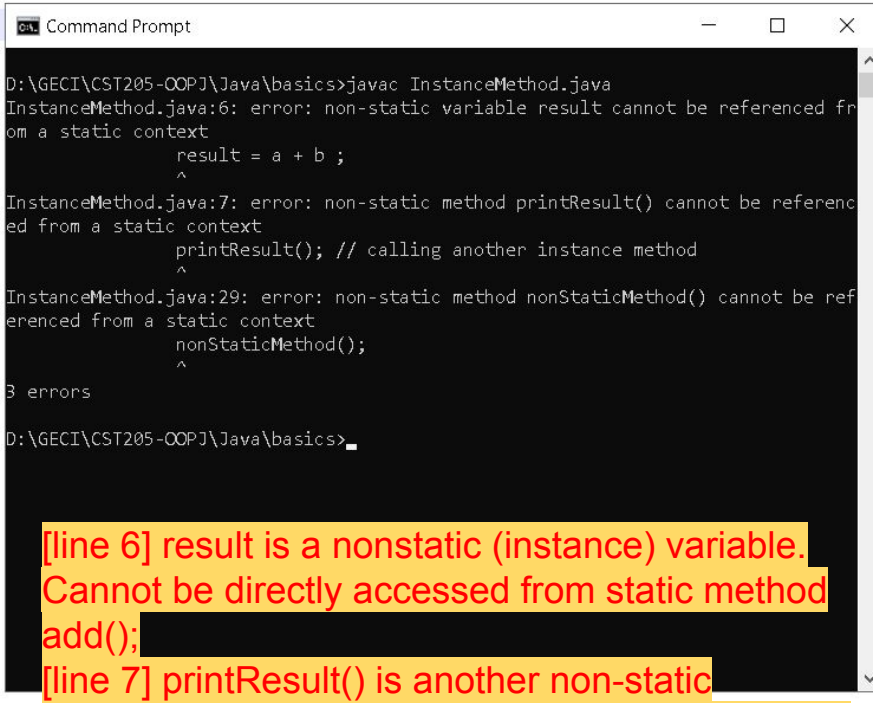
```
D:\GECI\CST205-OOPJ\Java
Result : 9
I am static method
I am non-static method

D:\GECI\CST205-OOPJ\Java
```

static methods

- These methods are declared with static modifier.
- Static methods are the methods in Java that can be called without creating an object of class.
- They are referenced by the **class name** itself or reference to the Object of that class.
- Static method(s) are associated to the class in which they reside i.e. they can be called even without creating an instance of the class i.e ClassName.methodName(args).
- They are designed with aim to be shared among all Objects created from the same class.
- Static methods can not be overridden. But can be overloaded since they are resolved using static binding by compiler at compile time.
- They can directly access only the static methods and static variables of their class

```java
class UnrelatedClass
{
    int result ;
    static void add(int a, int b) // instance method
    {
        result = a + b ;
        printResult(); // calling another instance method
    }
    void printResult()
    { System.out.println("Result : " + result);
    }
}
public class InstanceMethod
{
    static void staticMethod()
    {
        System.out.println("I am static method");
    }
    void nonStaticMethod()
    {
        System.out.println("I am non-static method");
    }
    public static void main(String[] args)
    {
        UnrelatedClass uc = new UnrelatedClass();
        uc.add(4,5);
        staticMethod() ; // Calling static method
        InstanceMethod im = new InstanceMethod();
        nonStaticMethod();
    }
}
```



1. [line 6] result is a nonstatic (instance) variable. Cannot be directly accessed from static method add();
2. [line 7] printResult() is another non-static member. Cannot be accessed directly from static method add()
3. [line 29] static method main cannot directly access a non-static method nonStaticMethod()

Constructor

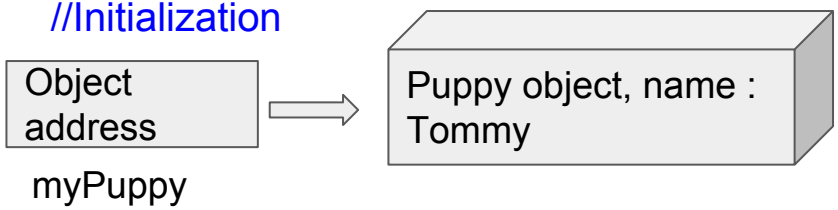
- A constructor is to initialize an object(give values to instance variables) immediately upon creation.
- Constructor is a special method inside the class.
- Constructor has the same name as the class in which it resides. No return type.
- Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.

```
public class Puppy {
   String m_name; // Member variable
   public Puppy() {   }
   public Puppy(String name) {
      m_name = name ;   }

   Public void whoareyou()
        { System.out.println("I am "+m_name);}

        public static void main(String args[]){
        Puppy myPuppy =  new
Puppy("Tommy");
      myPuppy.whoareyou();}
}
```

myPuppy = new Puppy("Tommy");
//Initialization



Constructor

- Constructors have no return type, not even void.
- This is because the implicit return type of a class' constructor is the class type itself.
- Two types of constructors

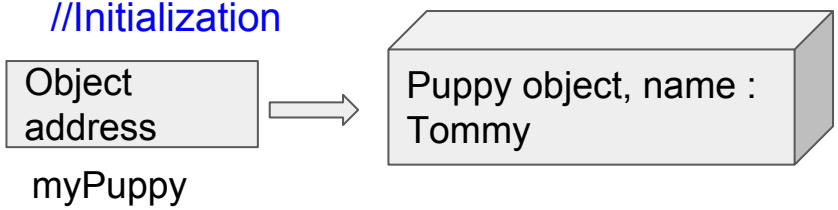
1. **Default constructor** – has no arguments
2. **Parameterized constructor** –has arguments(parameters)

```
public class Puppy {
   String m_name; // Member variable
   public Puppy() {   } // Default constructor
   public Puppy(String name) {
      m_name = name ;   }

   Public void whoareyou()
        { System.out.println("I am "+m_name);}

        public static void main(String args[]){
        Puppy myPuppy =  new
Puppy("Tommy");
      myPuppy.whoareyou();}
}
```

myPuppy = new Puppy("Tommy");
//Initialization



Default constructor

- The following statement creates an object of class Box.
- **Box mybox1 = new Box();**
- Here **new Box( )** is calling the **Box( )** constructor.
- When we do not explicitly define a constructor for a class, then Java creates a default constructor for the class.

```
1  class Box
2  {
3      int width ,length,height;
4      Box()
5      {
6          width=10;
7          length=10;
8          height=10;
9      }
10 }
11 public class BoxDemo
12 {
13     public static void main(String args[])
14     {
15         Box boxObj ; // Reference variable created
16         boxObj = new Box();
17         System.out.println("width " + boxObj.width);
18         System.out.println("length " + boxObj.length);
19         System.out.println("height " + boxObj.height);
20     }
21 }
```

```
D:\GECI\CST205-OOPJ\Java\basics>java BoxDemo
width 10
length 10
height 10
```

```
class Box {
    int length;
    int height;
    int width;
    Box()
    {
    System.out.println("Constructor");
    width = 10;
    length = 10;
    height= 10;
    }
    int volume()
    {
    return width * length * height;
    }
}
```

```
class BoxDemo {
public static void main(String args[])
{
Box mybox1 = new Box();
Box mybox2 = new Box();
int vol;
vol = mybox1.volume();
System.out.println("Volume is " + vol);

vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}
```

OUTPUT

```
Constructor
Constructor
Volume is 1000
Volume is 1000
```

Parameterized Constructors

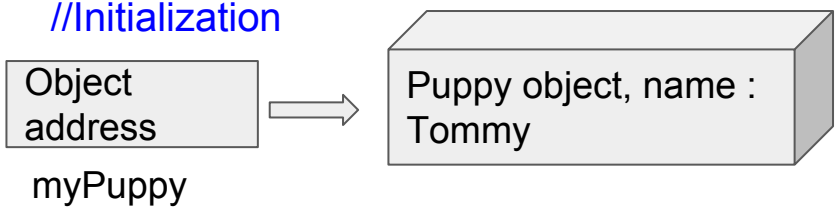
- Constructors with arguments are called parameterized constructors.
- Arguments can be of primitive data types or objects.

```
public class Puppy {
   String m_name; // Member variable
   public Puppy() {   }
   public Puppy(String name) {
      m_name = name ;   }

   Public void whoareyou()
        { System.out.println("I am "+m_name);}

        public static void main(String args[]){
        Puppy myPuppy =  new
Puppy("Tommy");
       myPuppy.whoareyou();}
}
```

myPuppy = new Puppy("Tommy");
//Initialization



```
class Box
{
double width;
double height;
double length;
Box(double w, double h, double l)
{
width = w;
height = h;
length = l;
}

double volume()
{
return width * height * length;
}
}
```

```
class BoxDemo {
public static void main(String args[]) {

Box mybox1  = new Box(10, 20, 15);
Box mybox2  = new Box(3, 6, 2);
double vol;
vol = mybox1.volume();
System.out.println("Volume  is " + vol);

vol = mybox2.volume();
System.out.println("Volume  is " + vol);
 }
}
```

OUTPUT
Volume is 3000
Volume is 36

Parameterized constructor(contd.)

Box mybox1 = new **Box(**10, 20, 15);

•Here the values 10, 20, and 15 are passed to the **Box( )** constructor when new creates the object mybox1.

•The parameterized constructor is

```
Box(double w, double h, double l)
{
width = w;
height = h;
length = l;
}
```

•Thus, value of mybox1 object's width, height, and depth will be set as 10, 20, and 15 respectively.

```java
class Box
{
    int width ,length,height;
    Box(int width, int length, int height)
    {
        width=width;
        length=length;
        height=height;
    }
    Box()
    {
        System.out.print("Default constructor");
    }
    void printVolume()
    {
        System.out.print("Volume = " + (width * length * height));
    }
}
public class BoxDemoParam
{
    public static void main(String args[])
    {
        Box boxObj ; // Reference variable created
        boxObj = new Box(10,10,10);
        boxObj.printVolume();
    }
}
```

Check the output of this program??

- Instance variables are hidden (not accessible) due to the function parameters with same name.

```
D:\GECI\CST205-OOPJ\Java\basics>javac BoxDemoParam.java

D:\GECI\CST205-OOPJ\Java\basics>java BoxDemoParam
Volume = 0
D:\GECI\CST205-OOPJ\Java\basics>
```

Instance variable hiding

- We can have local variables, including formal parameters to methods, with same **same name** of the class' instance variables(attributes).
- But when a local variable has the same name as that of an instance variable, the local variable hides the instance variable.
- **this** keyword helps to solve this situation.
- Use **this.** along with instance variables allow as to explicitly access the instance variable.

```
class Box
{
    int width ,length,height;
    Box(int width, int length, int height)
    {
        width=width;
        length=length;
        height=height;
    }
```

The **this** Keyword

- The **this** keyword can be used inside any method to refer to the current object.
- this is always a reference to the object on which the method was invoked.
- this can be used to refer current class instance variable.
- this can be used to invoke current class method (implicitly)
- this() can be used to invoke current class constructor.
- this can be passed as an argument in the method call.
- this can be passed as argument in the constructor call.

this -Example

```
Box(int w, int h, int l)
 {
this.width = w;
this.height = h;
this.length = l;
}
```

Here **this** will always refer to the object invoking the method

```
class Box
{
    int width ,length,height;
    Box(int width, int length, int height)
    {
        this.width=width;
        this.length=length;
        this.height=height;
    }
    Box(){ System.out.print("Default constructor"); }
    void printVolume()
    {
        System.out.print("Volume = " + (width * length * height));
    }
}
public class BoxDemoParam
{
    public static void main(String args[])
    {
        Box boxObj ; // Reference variable created
        boxObj = new Box(10,10,10);
        boxObj.printVolume();
    }
}
```

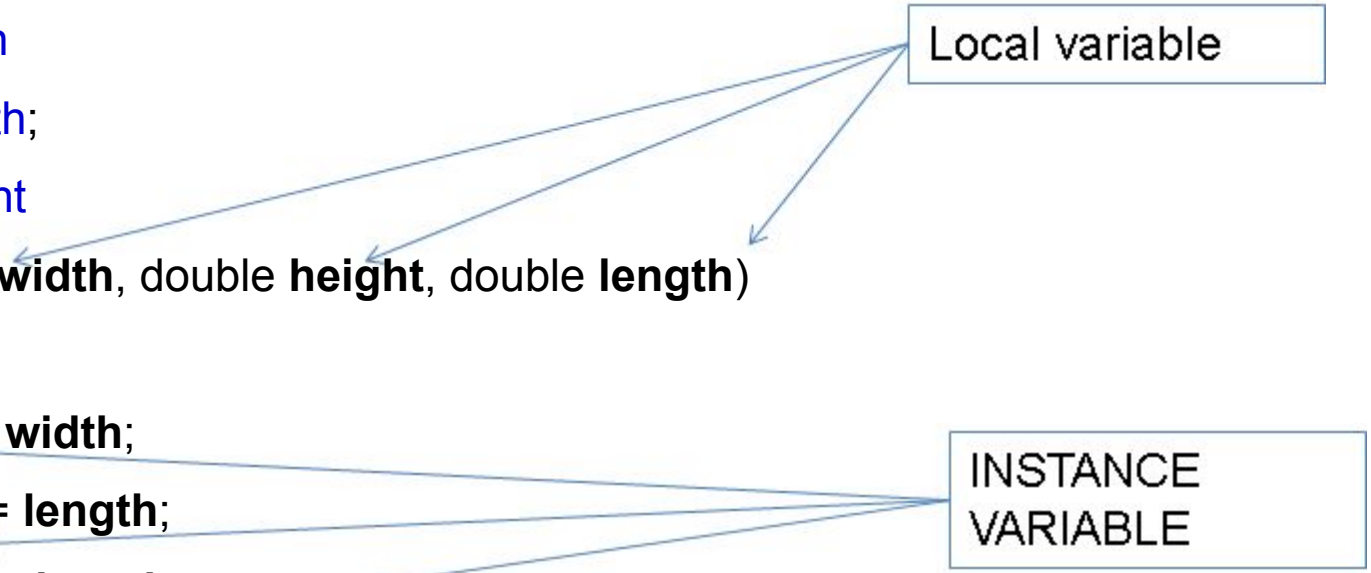
- Check the output of this program??
  - Instance variables are hidden (not accessible) due to the function parameters with same name. But they are accessed using **this** keyword

```
D:\GECI\CST205-OOPJ\Java\basics>java BoxDemoParam
Volume = 1000
D:\GECI\CST205-OOPJ\Java\basics>
```

Instance variable hiding-using this (contd.)

•// Use **this** to resolve namespace collisions.

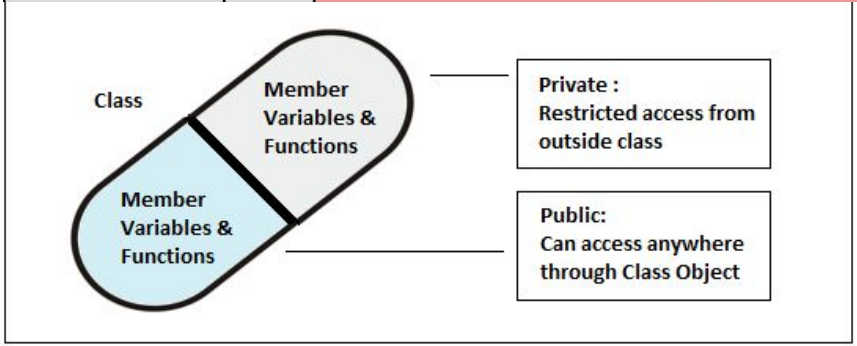
```
class Box
{
double width
double length;
double height
Box(double width, double height, double length)
{
this.width = width;
this.length = length;
this. height = length;
} }
```



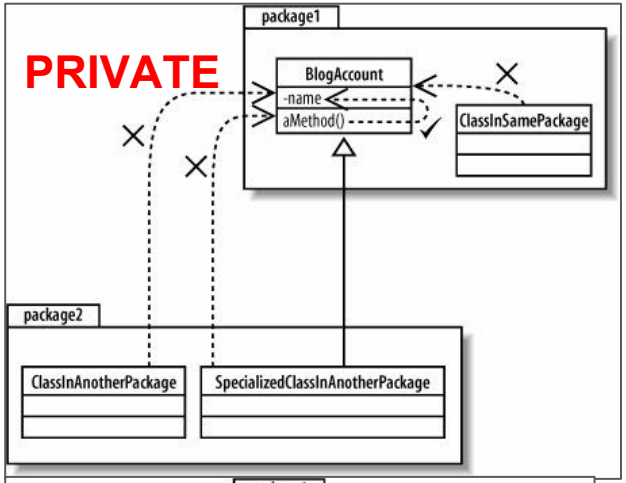
Access Control

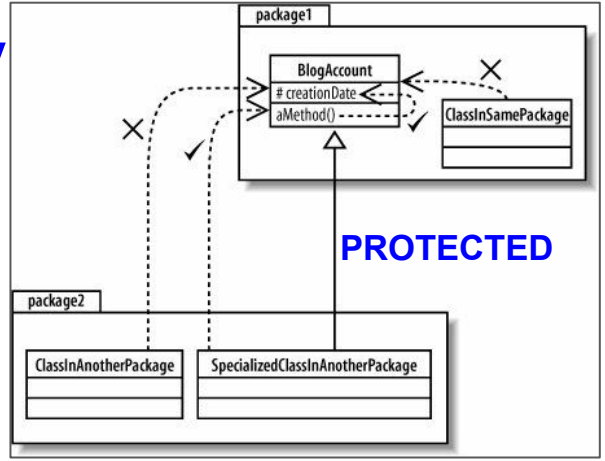
- Through encapsulation, we can control what parts of a program can access the members of a class.
- By controlling access, you can prevent misuse.
- How a member can be accessed is determined by the access specifier that modifies its declaration
- Java's access specifiers are
- **public**
- **private**
- **protected**
- **default**

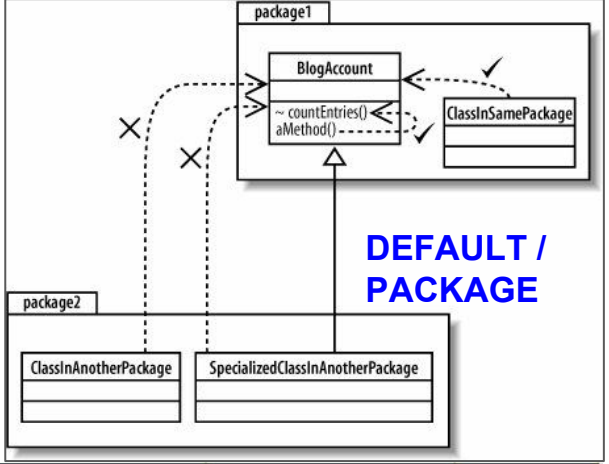
public	+	anywhere in the program and may be called by any object within the system
private	-	the class that defines it
protected	#	(a) the class that defines it or (b) a subclass of that class
package	~	instances of other classes within the same package

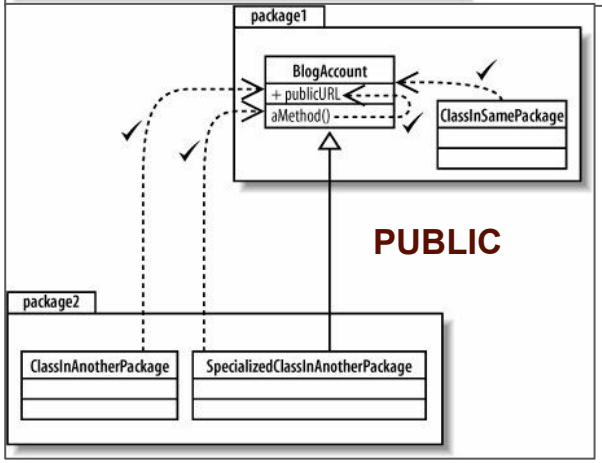


[UML]Member visibility

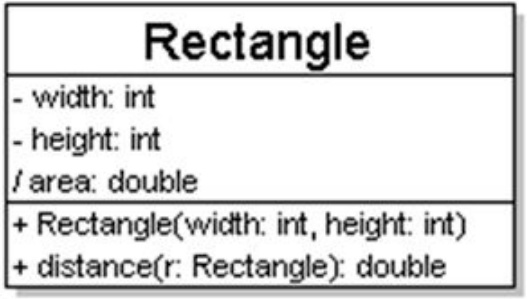






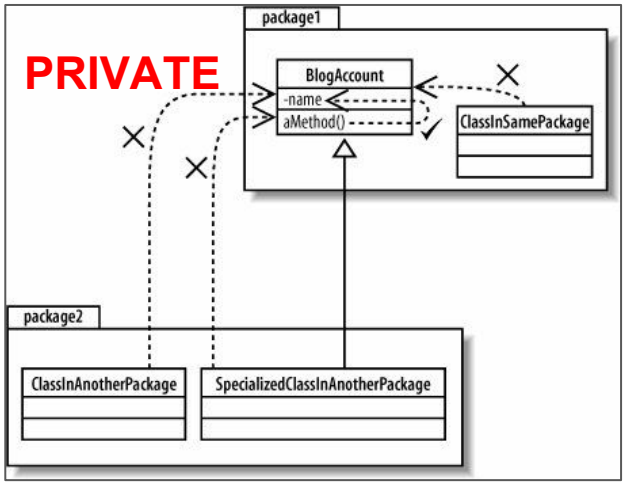


Access Modifier	Within Class	Within Package	Same Package by subclasses	Outside Package by subclasses	Global
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No



Private

- The private access modifier is specified using the keyword private.
- The methods or data members declared as private are accessible only within the class in which they are declared.
- Any other class of same package will not be able to access these members.
- Top level Classes or interface can not be declared as private because
- private means “only visible within the enclosing class”.
- Objectname.variable name is not allowed for private members



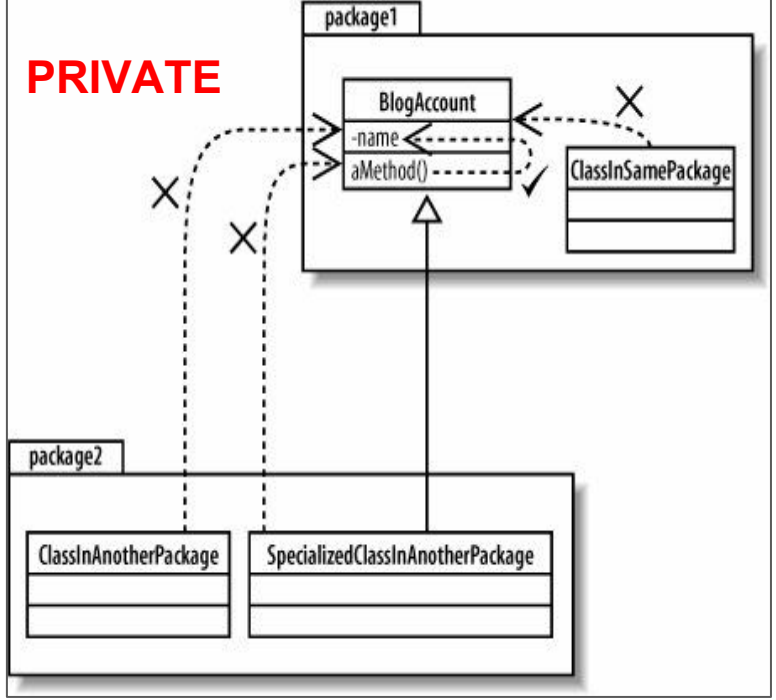
public	+	anywhere in the program and may be called by any object within the system
private	-	the class that defines it
protected	#	(a) the class that defines it or (b) a subclass of that class
package	~	instances of other classes within the same package

```java
//Java program to illustrate error while
//using class from different package with
//private modifier
package p1;

class A
{
    private void display()
     {
         System.out.println("GeeksforGeeks");
     }
}

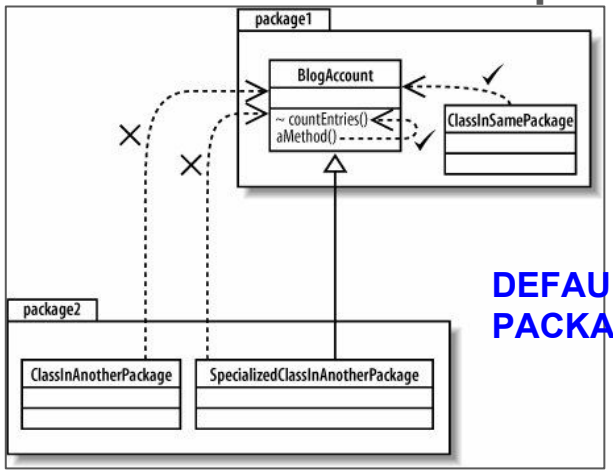
class B
{
    public static void main(String args[])
       {
           A obj = new A();
           //trying to access private method of another class
           obj.display();
       }
}
```

```
error: display() has private access in A
        obj.display();
```



Default

- Default: no modifier used [UML terminology: package]
- When no access modifier is specified for a class , method or data member – It is said to be having the default access modifier by default.
- The data members, class or methods which are not declared using any access modifiers i.e. having default access modifier are accessible only **within the same package**.



DEFAULT PACKAGE

public	+	anywhere in the program and may be called by any object within the system
private	-	the class that defines it
protected	#	(a) the class that defines it or (b) a subclass of that class
package	~	instances of other classes within the same package

```java
//Java program to illustrate default modifier
package p1;

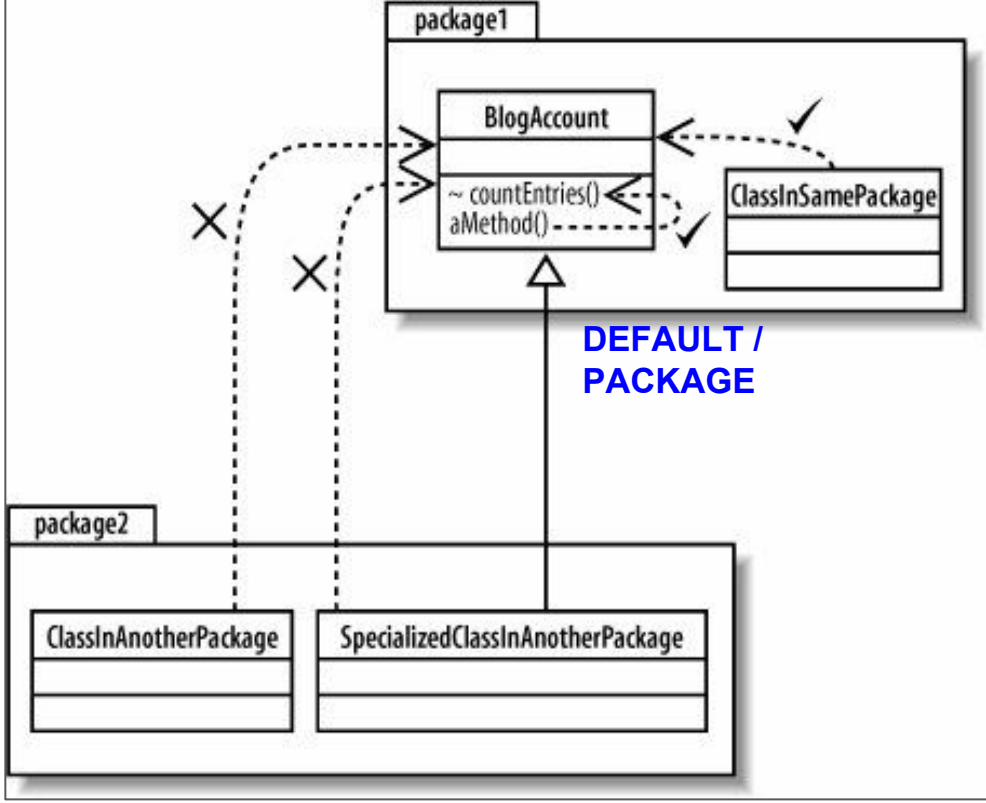
//Class Geeks is having Default access modifier
class Geek
{
    void display()
       {
           System.out.println("Hello World!");
       }
}
```

```java
//Java program to illustrate error while
//using class from different package with
//default modifier
package p2;
import p1.*;

//This class is having default access modifier
class GeekNew
{
    public static void main(String args[])
       {
          //accessing class Geek from package p1
          Geek  obj = new Geek();

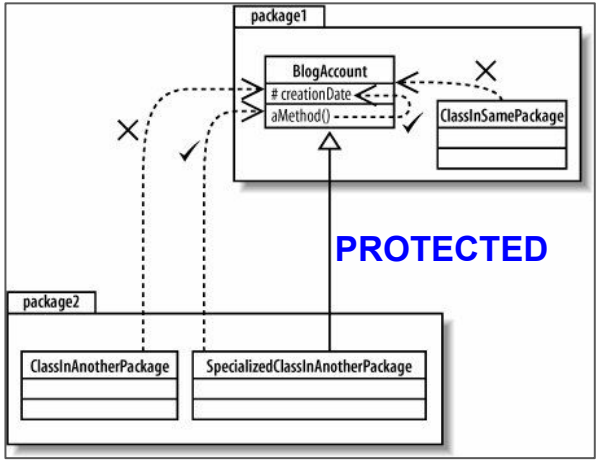
          obj.display();
       }
}
```

Compile time error



Protected

- The protected access modifier is specified using the keyword protected.
- The methods or data members declared as protected are accessible within same package or sub classes in different package.
- Top level Classes or interface can not be declared as protected because
- protected means “only visible within the enclosing class and any subclasses”
- Hence private/protected modifiers in terms of application to classes, they apply only to nested classes and not on top level classes



public	+	anywhere in the program and may be called by any object within the system
private	-	the class that defines it
protected	#	(a) the class that defines it or (b) a subclass of that class
package	~	instances of other classes within the same package

protected members in a class

- Variables, methods, and constructors, which are declared protected in a superclass can be accessed only by the subclasses in other package or any class within the package of the protected members' class.
- The protected access modifier cannot be applied to class and interfaces. Methods, fields can be declared protected, however methods and fields in a interface cannot be declared protected.
- Protected access gives the subclass a chance to use the helper method or variable, while preventing an unrelated class from trying to use it.

```java
//Java program to illustrate
//protected modifier
package p1;

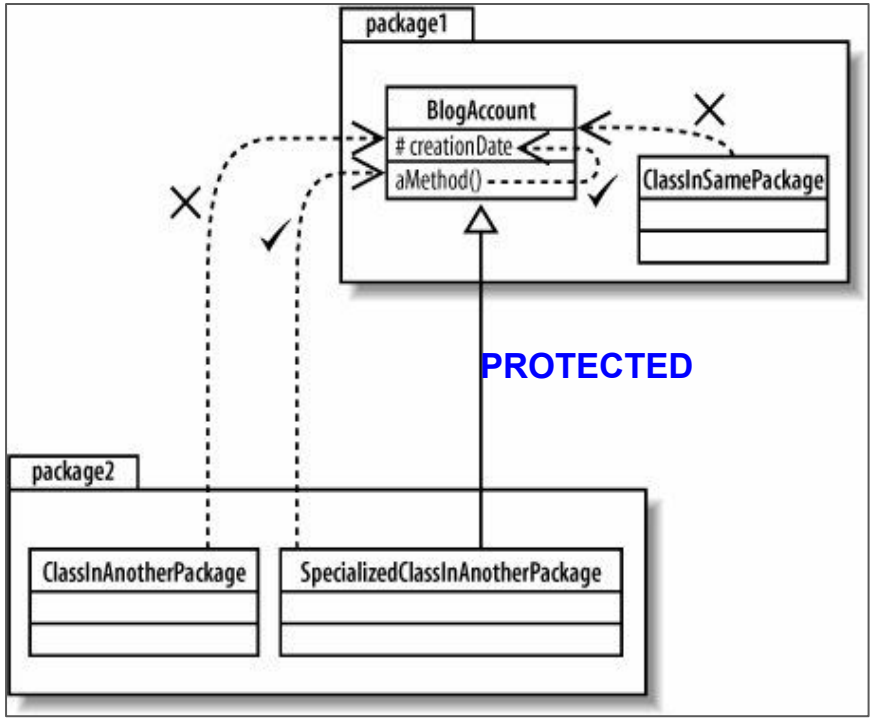
//Class A
public class A
{
   protected void display()
    {
        System.out.println("GeeksforGeeks");
    }
}
```

```java
//Java program to illustrate
//protected modifier
package p2;
import p1.*; //importing all classes in package p1

//Class B is subclass of A
class B extends A
{
   public static void main(String args[])
   {
       B obj = new B();
       obj.display();
   }

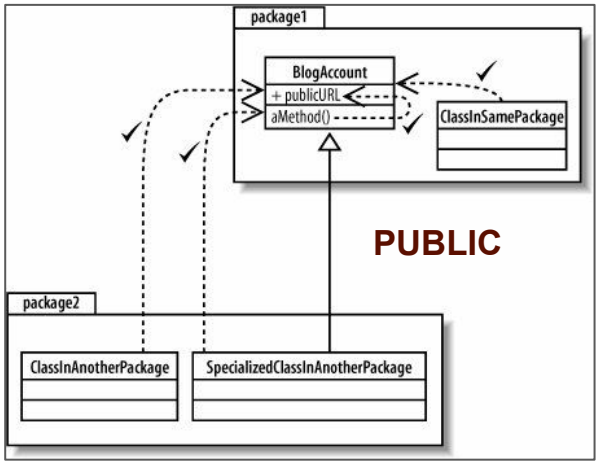
}
```

```
GeeksforGeeks
```



Public

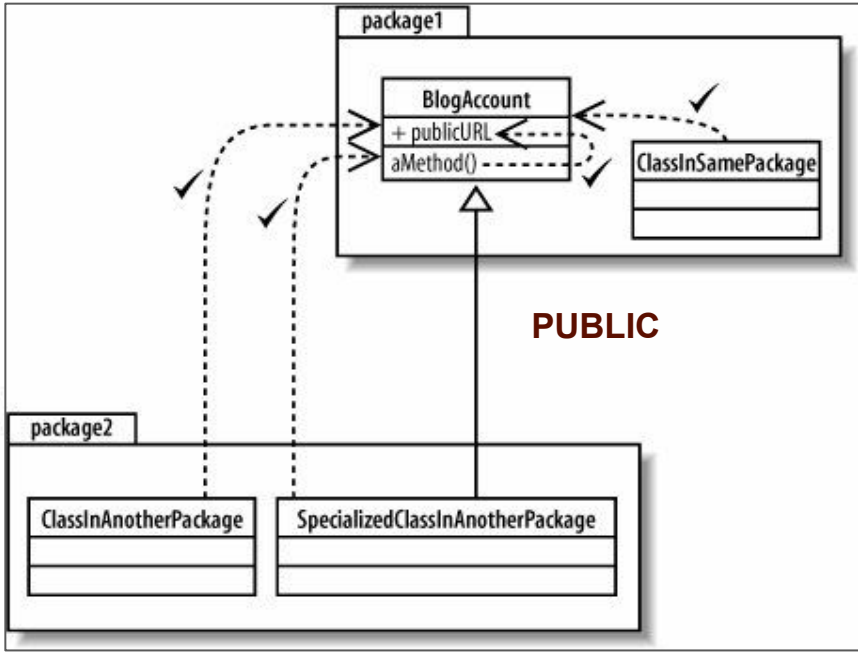
- The public access modifier is specified using the keyword public.
- The public access modifier has the widest scope among all other access modifiers.
- Classes, methods or data members which are declared as public are accessible from everywhere in the program. There is no restriction on the scope of a public data members.



public	+	anywhere in the program and may be called by any object within the system
private	-	the class that defines it
protected	#	(a) the class that defines it or (b) a subclass of that class
package	~	instances of other classes within the same package

```java
//Java program to illustrate
//public modifier
package p1;
public class A
{
    public void display()
        {
            System.out.println("GeeksforGeeks");
        }
}
package p2;
import p1.*;
class B
{
     public static void main(String args[])
        {
            A obj = new A;
            obj.display();
        }
}
```

```
GeeksforGeeks
```



Access Control

```
class Test
{
int a;              // default access
public int b;       // public access
private int c;      // private access

void setc(int i)            //setter
{
c = i;
}

int getc()          //getter
{
return c;
} }
```

```
class AccessTest {
public static void main(String args[]) {
Test ob = new Test();
ob.a = 10;
ob.b = 20;
// ob.c = 100; // Error! // PRIVATE
// You must access private variable c
    //through its methods
ob.setc(100);          // OK
System.out.println("a="+ob.a);
System.out.println("b="ob.b");
System.out.println("c= " + ob.getc() );
}
}
```

Access Control

	PRIVATE	DEFAULT	PROTECTED	PUBLIC
Same class	Yes	Yes	Yes	Yes
Same package Subclass	No	Yes	Yes	Yes
Same package Non-subclass	No	Yes	Yes	Yes
Different package Subclass	No	No	Yes	Yes
Different package Non-subclass	No	No	No	Yes

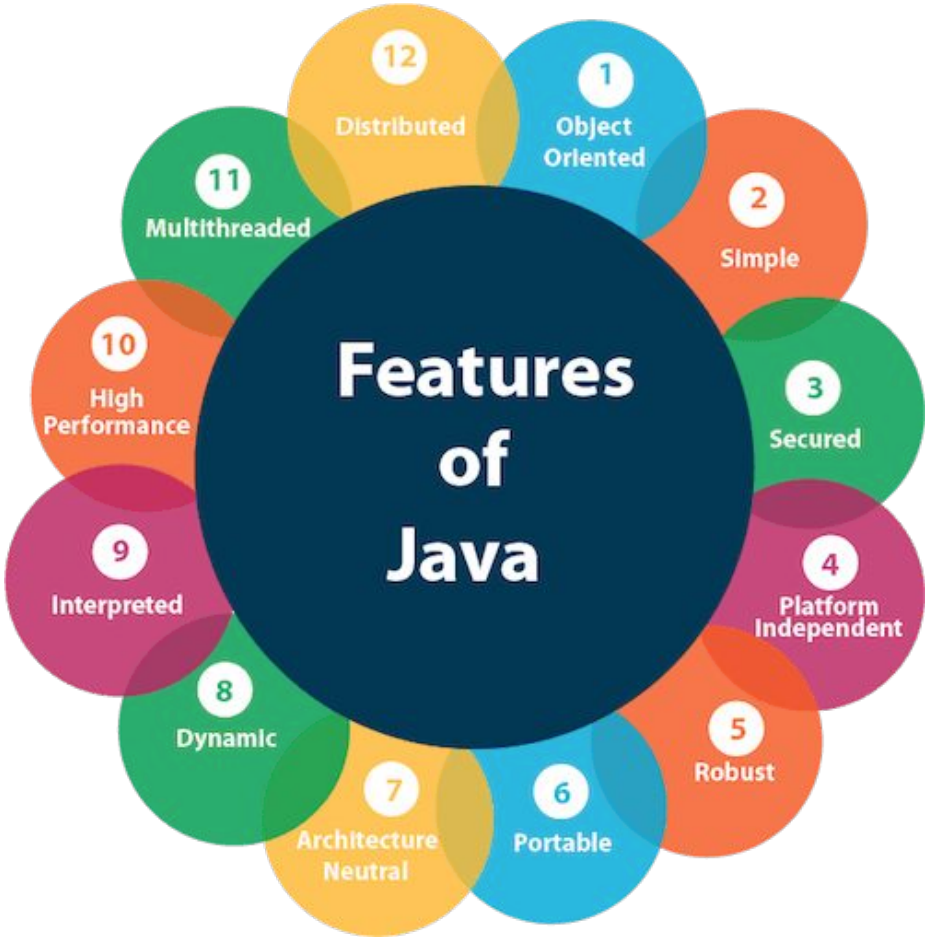
SAME CLASS

SAME PACKAGE,

SAME PACKAGE , ANY SUBCLASS

ALL

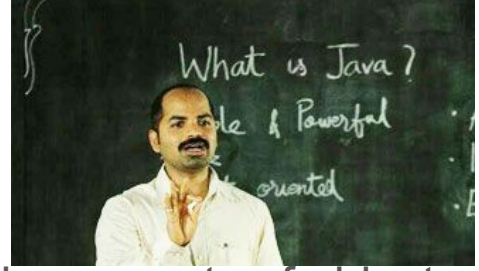
Java Buzzwords



Important features of Java language.

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

Java features



1. **Simple**

- Java **syntax** is based on C/C++.
- If you already understand the basic concepts of object oriented programming, learning Java will be even easier.
- Java has removed many complicated and rarely-used features, for example, explicit **pointers**, **operator overloading**, etc.
- There is no need to remove unreferenced objects because there is an **Automatic Garbage Collection** in Java.

1. **Object-oriented**

- Java is an object-oriented programming language.
- OOP means we organize our software as a combination of different types of **objects with both data and behavior**.
- Basic concepts of OOPs are: Object, Class, Inheritance Polymorphism, Abstraction, Encapsulation

Java features

3. Platform Independent

- Java is platform independent because java bytecode (class file) can run on any platform, just copy the class file to new platform and run using **java** interpreter
- Class file contains the platform independent bytecode which can run on any platform with Java VM.

4. Robust

- Robustness is the property of being **strong** and **healthy.**
- Java is robust because:
  1. There is **automatic memory management** (garbage collection) in java which runs on the JVM to get rid of objects which are not being used anymore.
  2. There is a **lack of explicit pointers** that avoids security problems.
  3. There are **exception handling** and the type checking mechanism.

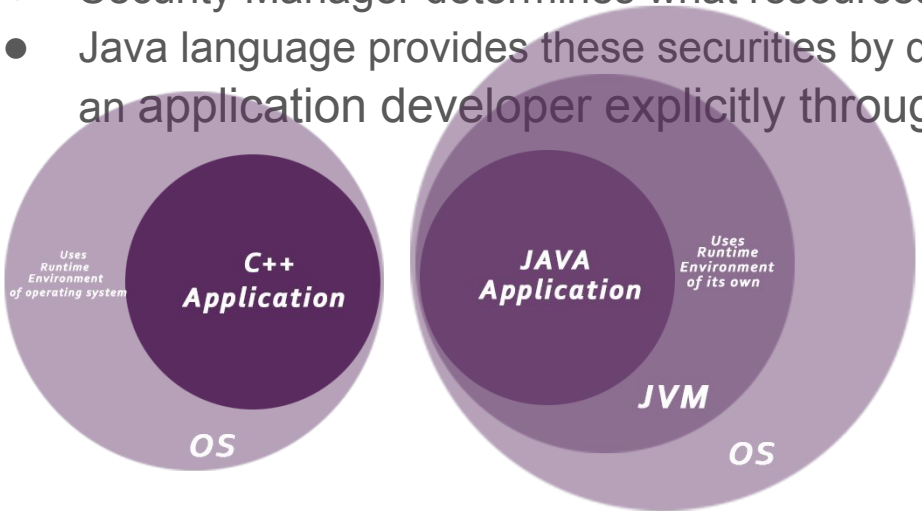
5. Interpreted

- Java programs are compiled into an intermediate representation called Java bytecode.
- The **byte code is interpreted** to any machine code so that it runs on the native machine.

Java features

6. Secured

- Java is secure, we can develop **virus-free systems**. It is secured because,
- No explicit pointer, Java Programs run inside a virtual machine sandbox
- JVM has Classloader to load Java classes dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network.
- Bytecode Verifier checks the code fragments for illegal code that can violate access right to objects.
- Security Manager determines what resources a class can access.
- Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.



Java features

7. Architecture-neutral

- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture.
- However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

8. Portable

- Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

9. High-performance

- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.
- It is still a little bit slower than a compiled language (e.g., C++).

Java features

10. Distributed

- Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

11. Multi-threaded

- A thread (light weight process) is a single sequential flow of control within a program.
- Java supports multiple concurrent threads with APIs to manage the thread states.
- We can write Java programs that deal with many tasks at once by defining multiple threads.

12. Dynamic

- Java supports dynamic loading of classes.
- It means classes are loaded on demand at run time.
- Java supports dynamic compilation and automatic memory management (garbage collection) at JVM level.

Garbage Collection

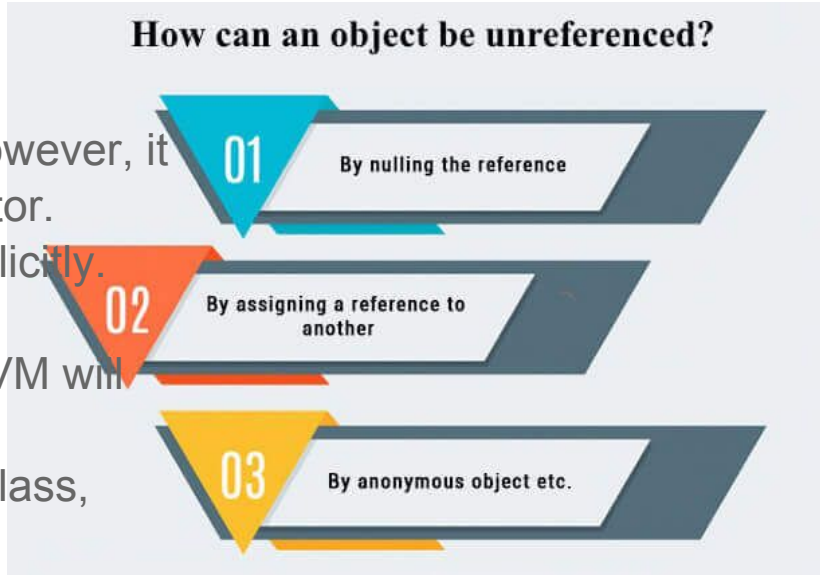
- **Memory leak** is a type of resource leak that occurs when computer programs fails to release the allocated memory after use. This is a common issue in C/C++ programs.
- Java's answer to this issue is Automatic Garbage Collection
- When JVM starts up, it creates a heap area which is known as runtime data area. This is where all the objects (instances of class) are stored
- The process of removing unused objects from heap memory is known as Garbage collection and this is a part of memory management in Java.
- In java, garbage means unreferenced objects.
- In Java responsibility of releasing unused memory is with JVM.
- In C/C++.. Programmer is responsible for memory release and it is the major reason for memory leaks and program crashes.

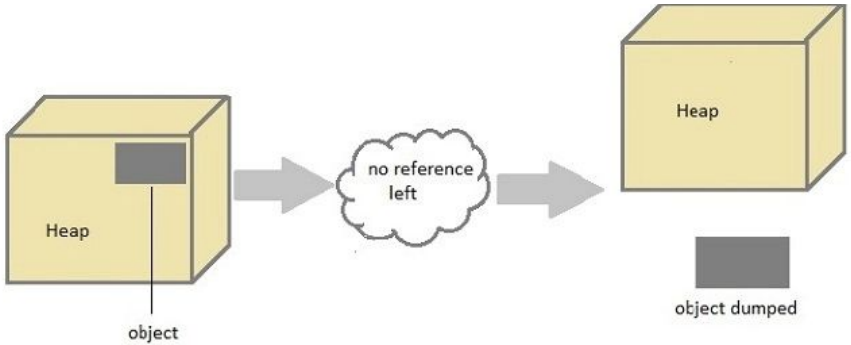
Garbage Collection

- In java, garbage means unreferenced objects.
- Main objective of Garbage Collector is to free heap memory by destroying unreachable objects.
- Unreachable objects : An object is said to be unreachable when there is no valid references to that object exist.
- Eligibility for garbage collection : object must be unreachable.
- **finalize()** method – This method is invoked each time before the object is garbage collected and it is intended to perform cleanup processing.
- Garbage collector releases only those objects that are created by **new** keyword.
- So if we have created any object without new, we can use finalize method to perform cleanup processing

Request for Garbage Collection

- We can request to JVM for garbage collection however, it is upto the JVM when to start the garbage collector.
- gc() method is used to call garbage collector explicitly.
- It only request the JVM for garbage collection.
- However gc() method does not guarantee that JVM will perform the garbage collection.
- This method is present in System and Runtime class, functionally both are same.





MCQ - 1

Which component of the Java environment is responsible for executing bytecodes directly on the host machine?

A) Java Development Kit (JDK)

B) Java Virtual Machine (JVM)

C) Java Compiler (javac)

D) Integrated Development Environment (IDE)

MCQ - 2

What is the correct file extension generated by the Java compiler (javac) after successfully compiling a .java source file?

A) .exe

B) .jar

C) .class

D) .obj

MCQ - 3

What is the required method signature for the main entry point of a standard Java application?

A) public static void main(String args[])

B) public void main(String[] args)

C) static void main(String[] args)

D) public static int main(String[] args)

MCQ - 4

Which of the following data types is a primitive type in Java?

A) String

B) Integer

C) double

D) Vector

MCQ - 5

Automatic conversion of a primitive data type into its corresponding wrapper object is known as what?

A) Explicit Casting

B) Unboxing

C) Autoboxing

D) Widening Conversion

MCQ - 6

What will be the result of executing (int) 9.85 in Java?

A) 10

B) 9

C) 9.85

D) Compilation Error

MCQ - 7

How is memory allocated for an array in Java?

A) Statically at compile time on the Stack

B) Dynamically at runtime on the Heap using new

C) Automatically in register memory

D) Using the malloc() function

MCQ - 8

Why are Java String objects referred to as "immutable"?

A) They cannot be assigned to another variable.

B) Their values cannot be modified once created.

C) They can only hold numerical characters.

D) They do not inherit from Object.

MCQ - 9

Which feature distinguishes the java.util.Vector class from a standard array?

A) It stores only primitive data types.

B) It is fixed in size upon instantiation.

C) It is dynamic in size and thread-safe (synchronized).

D) It cannot hold duplicate elements.

MCQ - 10

What is the output of the expression 10 + 3 * 2 in Java?

A) 26

B) 16

C) 30

D) 20

MCQ - 11

Which operator is used to evaluate a condition and return one of two values depending on whether the condition is true or false?

A) &&

B) instanceof

C) ?:

D) ==

MCQ - 12

What does the bitwise XOR operator (^) evaluate to when comparing two identical bits (e.g., 1 ^ 1)?

A) 1

B) 0

C) -1

D) True

MCQ - 13

Which control statement is used to immediately exit a loop or switch block?

A) continue

B) return

C) break

D) skip

MCQ - 14

How many times will a do-while loop execute its body at a minimum?

A) 0 times

B) 1 time

C) 2 times

D) Depends on the initial condition

MCQ - 15

Which variable type was traditionally NOT supported in a Java switch statement before Java 7?

A) int

B) char

C) byte

D) String

MCQ - 16

How are command-line arguments passed into a Java main method accessible inside the program?

A) As an array of integers

B) As an array of String objects

C) Via system environment variables only

D) As a single concatenated string

MCQ - 17

Which syntax correctly declares a variable-length argument (varargs) parameter in a Java method?

A) void printValues(int... numbers)

B) void printValues(int[]... numbers)

C) void printValues(var int numbers)

D) void printValues(int numbers[])

MCQ - 18

Where must a variable-length argument (varargs) parameter be placed in a method signature?

A) At the beginning of the parameter list

B) Anywhere in the parameter list

C) As the last parameter in the signature

D) Varargs cannot be combined with other parameters

MCQ - 19

Wrapping data (fields) and methods into a single unit while restricting direct access to components is known as:

A) Inheritance

B) Encapsulation

C) Polymorphism

D) Abstraction

MCQ - 20

Which core OOP concept allows a single method call to exhibit different behaviors based on the object instance receiving the call?

A) Encapsulation

B) Polymorphism

C) Aggregation

D) Serialization

MCQ - 21

In software architecture, what characterizes the Microservices paradigm compared to a Monolithic architecture?

A) Building an application as a single tightly-coupled binary

B) Structuring an application as a collection of small, independent, deployable services

C) Eliminating the use of database instances

D) Replacing object-oriented programming with procedural programming

MCQ - 22

What keyword is used to allocate memory and instantiate an object of a class?

A) class

B) create

C) new

D) alloc

MCQ - 23

What is a key requirement when declaring a constructor in Java?

A) It must return an int.

B) It must have the same name as the class and no return type.

C) It must be declared static.

D) It must accept at least one argument.

MCQ - 24

Inside an instance method, what does the this keyword refer to?

A) The superclass of the current object

B) The current object instance executing the method

C) The main method of the application

D) A static reference to the class

MCQ - 25

Which access modifier makes a class member accessible ONLY within its own class?

A) public

B) protected

C) default (package-private)

D) private

MCQ - 26

Which point is not correct about the static keyword?

A) Variable with static prefix are not part of the object.

B) static keyword is used to create instance variable

C) A static method can access only the other static members of the same class

D) Static variables are automatically initialized to the default values.

MCQ - 27

According to Java naming conventions, interface names and class names follow which casing style?

A) lowercase

B) UpperCamelCase

C) SCREAMING_SNAKE_CASE

D) kebab-case

MCQ - 28

Which keyword is used by a Java class to inherit features from an abstract class?

A) implements

B) extends

C) uses

D) exports

MCQ - 29

What is true about StringBuffer in Java?

A) Used to create immutable string objects.

B) Used to create mutable string objects

C) Used as a temporary storage

D) Used an a primitive string data type

MCQ - 30

Which keyword does a class use to declare that it fulfills an interface contract in Java?

A) extends

B) implements

C) interface

D) abstract

Answer Key (Questions 1 – 15)

1	B	The JVM executes the compiled bytecode on the targeted OS platform.
2	C	javac produces .class files containing Java bytecode.
3	A	public static void main(String args[]) is the exact required entry point.
4	C	double is a primitive type; String, Integer, and Vector are reference objects.
5	C	Converting a primitive to a wrapper class object automatically is Autoboxing.
6	B	Explicit cast (int) truncates decimal values to 9.
7	B	Java arrays are objects created dynamically on the heap using new.
8	B	Strings are immutable; any change produces a new String instance.
9	C	Vector is dynamic in capacity and synchronized for thread safety.
10	B	Multiplication * has higher precedence than addition +: 10+(3×2)=16.
11	C	The ternary operator ?: is a shorthand conditional statement.
12	B	XOR outputs 0 when both comparison bits are identical (1 ^ 1 = 0).
13	C	break exits the nearest enclosing loop or switch block immediately.
14	B	A do-while loop evaluates condition at the end, so it executes at least once.
15	D	String support in switch statements was added in Java 7.

Answer Key (Questions 16 – 30)

16	B	Command line inputs pass to main(String[] args) as an array of String.
17	A	Varargs syntax uses type... variableName.
18	C	Varargs must be the last parameter in a method signature.
19	B	Encapsulation bundles data and logic while restricting direct variable access.
20	B	Polymorphism enables unified interfaces with dynamic runtime behavior.
21	B	Microservices break applications into small, independently deployable services.
22	C	The new keyword allocates heap memory for new objects.
23	B	Constructors match the class name exactly and have no return type.
24	B	this is a reference variable pointing to the current object instance.
25	D	private limits access strictly to within the defining class body.
26	B	static keyword will make the variable to class variable.
27	B	Interfaces and classes both use UpperCamelCase.
28	B	Classes extend other classes (including abstract classes) using extends.
29	B	StringBuffer objects are mutable.
30	B	Classes implement interfaces using the implements keyword.